Heap Thrust Reference

ICPC Reference

Team Members:

Enrique Job Calderón (ksobrenat32)
Luis Daniel Salazar (luisdakan)
Gustavo Valenzuela (GusTimeTraveler)

Institution:
Facultad de Ingeniería, UNAM - CPCFI

September 12, 2026

Contents

1 Python stuff **2**
- 1.1 Regular input . . . 2
- 1.2 Fast I/O . . . 2
- 1.3 List comprehensions . . . 2
- 1.4 Strings . . . 2
- 1.5 Regex . . . 3
  - 1.5.1 Regex notes . . . 3
- 1.6 Itertools . . . 4
- 1.7 Scripts . . . 5
  - 1.7.1 Combinatorics calculator . . . 5
  - 1.7.2 Random array generator . . . 5
  - 1.7.3 Generate all subsets . . . 5

2 C++ utils **5**
- 2.1 Common STL Algorithms . . . 5
- 2.2 Random number generator . . . 7
  - 2.2.1 Graph generator . . . 7
- 2.3 Comparators for Structs . . . 7
- 2.4 Lambda Functions for Sort . . . 7

3 Binary Search in the Answer **8**
- 3.1 Parallel Binary Search . . . 9

4 Bitwise Techniques **9**
- 4.1 All Subsets . . . 9
- 4.2 FWHT . . . 9
- 4.3 Gray Code . . . 10
- 4.4 XOR Basis . . . 10
- 4.5 Hopcroft-Karp . . . 10
- 4.6 Min-Cost Max-Flow . . . 11
- 4.7 Push-Relabel . . . 12
- 4.8 0-1 BFS . . . 13
- 4.9 Biconnected Components . . . 13
- 4.10 Johnson's Algorithm . . . 14

5 Graph Theory **15**
- 5.1 Bellman-Ford . . . 15
- 5.2 Dijkstra . . . 15
- 5.3 Layered Dijkstra . . . 15
- 5.4 Dinic . . . 16
- 5.5 Directed Cycles . . . 17
- 5.6 Edmonds-Karp . . . 18
- 5.7 Floyd-Warshall . . . 18
- 5.8 Kahn Topological Sort . . . 18
- 5.9 Kruskal . . . 19
- 5.10 Lava Flow . . . 19
- 5.11 Negative Cycles . . . 20
- 5.12 Prim Algorithm . . . 21
- 5.13 Strongly Connected Components . . . 21
- 5.14 Topological Sort . . . 22
- 5.15 2-SAT . . . 22
- 5.16 Undirected Cycles . . . 23
- 5.17 Articulation Points . . . 23
- 5.18 Bridges . . . 24
- 5.19 Bridges Online . . . 24
- 5.20 Bridge Tree . . . 25
- 5.21 Dynamic Connectivity . . . 26
- 5.22 Eulerian Path . . . 27
- 5.23 Gale-Shapley . . . 28
- 5.24 Hungarian Algorithm . . . 28
- 5.25 SPFA . . . 29
- 5.26 Tarjan . . . 29
- 5.27 Sqrt Decomposition . . . 30

6 Data Structures **30**
- 6.1 2D Segment Tree . . . 30
- 6.2 Bit Trie . . . 31
- 6.3 Disjoint Set Union . . . 32
- 6.4 Fenwick Tree . . . 32
- 6.5 GP Hash Table . . . 33
- 6.6 Implicit Treap . . . 33
- 6.7 Iterative Segment Tree . . . 34
- 6.8 Lazy Segment Tree (Max) . . . 35
- 6.9 Lazy Segment Tree (Sum) . . . 36
- 6.10 Lazy Treap . . . 36
- 6.11 Merge Sort Tree . . . 38
- 6.12 Ordered Set . . . 38
- 6.13 Recursive Segment Tree . . . 39
- 6.14 Treap . . . 39
- 6.15 Cycle Detection Online . . . 40
- 6.16 DSU with Rollbacks . . . 41

6.17 Fenwick Lazy . . . 41
6.18 Implicit Treap 2 . . . 41
6.19 Online Bipartiteness . . . 42
6.20 Path Copy Persistent Array . . . 43
6.21 Persistent Array . . . 43
6.22 Persistent Segment Tree . . . 43
6.23 Prefix Suffix Segment Tree . . . 44
6.24 Wicho Lazy Segment Tree . . . 45
6.25 Li Chao Tree . . . 46
6.26 Mo's Algorithm . . . 47
7 Range Queries **47**
7.1 Sparse Table . . . 47
8 Mathematics **48**
8.1 Formula Sheet (Contest) . . . 48
8.1.1 Combinatorics . . . 48
8.2 Cayley's Formula . . . 48
8.3 Geometric series . . . 48
8.4 Divisors . . . 48
8.5 Number of primes between 1 and n . . . 49
8.6 Pythagorean triplets . . . 49
8.7 Derangements . . . 49
8.7.1 Sums . . . 49
8.7.2 Geometry . . . 49
8.7.3 Probability . . . 50
8.7.4 Number Theory and Counting Essentials . . . 50
8.8 Manhattan Distance on a Grid . . . 50
8.9 Binary Exponentiation . . . 50
8.10 Binomial Coefficients . . . 50
8.11 Extended Euclidean . . . 51
8.12 FFT . . . 51
8.13 Big Integer . . . 52
8.14 NTT . . . 52
8.15 Graph Paths . . . 54
8.16 Graph Paths Min-Plus . . . 54
8.17 Log Fibonacci . . . 55
8.18 Matrix Determinant . . . 55
8.19 Matrix Rank . . . 55
8.20 Gauss-Jordan . . . 56
8.21 Miller-Rabin (Primality Test) . . . 56
8.22 Linear Sieve . . . 57
8.23 Euler Totient Function . . . 57
8.24 Fast Totient Function . . . 57
8.25 Diophantine Equations . . . 58
8.26 Mod Struct . . . 58
8.27 Factorials . . . 59
8.28 Number of Divisors . . . 59
8.29 Modular Inverse . . . 59
8.30 Discrete Logarithm . . . 59
8.31 Segmented Sieve . . . 59
8.32 Sieve of Eratosthenes . . . 60
8.33 Simpson Method . . . 60
8.34 Precompute Pascal Triangle . . . 60
8.35 Sqrt and Inverse Sqrt . . . 60
8.36 Sum of Divisors . . . 61
8.37 Ternary Search . . . 61
8.38 Trivial Factorization . . . 61
8.39 Chinese Remainder Theorem . . . 61
8.40 Garner's Algorithm . . . 62
8.41 Polynomial Operations . . . 63
8.42 Linear Recurrence . . . 65
8.43 Matrix Exponentiation . . . 67
8.44 Modular Square Root . . . 67
8.45 Simplex . . . 68
9 String Algorithms **70**
9.1 Aho-Corasick . . . 70
9.2 DP on Trie . . . 71
9.3 KMP . . . 72
9.4 Manacher Algorithm . . . 72
9.5 Palindromic Tree . . . 73
9.6 Polynomial Hash . . . 74
9.7 Rabin-Karp . . . 74
9.8 Suffix Array + LCP . . . 75
9.9 Suffix Automaton . . . 76
9.10 Suffix Automata 2 . . . 76
9.11 String Stream . . . 77
9.12 XOR Hash . . . 77
9.13 Z Algorithm . . . 78

10 Computational Geometry **78**
10.1 Convex Hull . . . 78
10.2 Complex Point . . . 79
10.3 Line Segment Intersection . . . 80
10.4 Minimum Euclidean Distance . . . 81
10.5 Minimum Manhattan Distance . . . 81
10.6 Max Nearest Distance . . . 81
10.7 Point In Polygon . . . 82
10.8 Point Location Test . . . 83
10.9 Polar Sort . . . 84
10.10Polygon Area . . . 84
10.11Closest Pair of Points . . . 84
10.12Half-Plane Intersection . . . 85
10.13Smallest Enclosing Circle . . . 87
10.14Rotating Calipers . . . 87
10.15Pick's Theorem . . . 88
11 Dynamic Programming **89**
11.1 Coin Problem . . . 89
11.2 Coin Problem (Min Coins) . . . 89
11.3 Coin Problem (Sorted) . . . 89
11.4 Knapsack . . . 89
11.5 LCS . . . 90
11.6 LIS . . . 90
11.7 Two Sets . . . 90
11.8 Bitmask DP . . . 90
11.9 Digit DP . . . 91
11.10Edit Distance . . . 91
11.11Retrieve LIS . . . 92
11.12SOS DP . . . 92
11.13XOR Subset DP . . . 92
11.14Convex Hull Trick . . . 93
11.15Knuth's Optimization . . . 94
11.16Divide and Conquer DP . . . 94
12 Trees **95**
12.1 Diameter . . . 95
12.2 Euler Tour . . . 95
12.3 Tree Isomorphism . . . 96
12.4 Tree Isomorphism (Centroid Hash) . . . 97
12.5 Max Distance . . . 97
12.6 Binary Lifting . . . 98
12.7 Centroid Decomposition . . . 98
12.8 Heavy-Light Decomposition . . . 99
12.9 Lowest Common Ancestor . . . 100
12.10Binary Lifting LCA . . . 101
12.11Successor DP . . . 101
13 Miscellaneous **101**
13.1 2D Prefix Sum . . . 101
13.2 Kadane . . . 102
13.3 Index Compression . . . 102
13.4 Berlekamp-Massey . . . 102
13.5 Moore's Voting . . . 103
13.6 Compiler Optimizations . . . 103
13.7 Sprague-Grundy . . . 103

1 Python stuff

1.1 Regular input

```python
s = input() # This is a string
n = int(input()) # This is an integer

arr = input().split() # This is an array of strings

# Array of integers (list comprehension)
a = [int(x) for x in input().split()]
x,y,z = [int(i) for i in input().split()] # Multiple integers
```

1.2 Fast I/O

```python
import sys
input = sys.stdin.readline
n = 220
sys.stdout.write(str(n) + '\n')
```

1.3 List comprehensions

```python
# 1 to N
nums = [i for i in range(1, N+1)]

# Filter values
evens = [x for x in arr if x%2 == 0]

# Filter primes
primes = [x for x in arr if all(x%d for d in range(2,int(x**0.5)+1))
    ]

# Square every element
sqr = [x*x for x in arr]

# Convert string digits to int
digits = [int(c) for c in s]

# Nested loops
pairs = [(i,j) for i in range(N) for j in range(M)]

# Generate NxM matrix
matrix = [[i*j for j in range(M)] for i in range(N)]

# Generate all combinations of two lists
combs = [(a,b) for a in list1 for b in list2]

# Prefix Zum
from itertools import accumulate
pref = list(accumulate(arr))

# Count occurences of a value (without count)
cnt = sum(1 for x in arr if x == target)
```

1.4 Strings

```python
s = "  Hola reference  "
# Remove trailing/whitespaces
print(s.strip()) # also .lstrip and .rstrip
print(s.split()) # Separates by spaces on list

s = "hola,reference,icpc,2026"
print(s.split(',')) # Separate by character

strs = ['vamos', 'a','ganar','icpc']

# Join a list of strings
joined = ' '.join(strs)
print(joined)

print(s.replace('hola','adios')) # Replace substrings
s.find("reference") # Finds the first index of the substring
s.rfind("reference") # Finds the last index of substring

s.startswith('hola') # Checks for a prefix
s.endswith('2026') # Checks for a suffix

s.count('icpc') # Counts the number of non overlapping occurrences
s.lower() # to lower
s.upper() # to upper

s.capitalize() # Capitalizes the word

c = 'a'
c.isalpha() # Is from alphabet

c='2'
c.isdigit() # Checks if it's a digit

c = '69'
c.isnumeric() # Checks if it is a number

# Slice a string (substring)

subs = s[2:5] # Substring from indices 2 to 5
rev = s[::-1] # Reverse a substring
```

```
step = s[::2] # Transform into every second character

c = 'z'
ord(c) # Unicode code point of the character
chr(65) # Reverse of ord
```

1.5 Regex

We must be careful with these functions since they could blow into $O(2^n)$ complexity.

```
import re

# 1. Basic matching
re.match(r'foo', 'foobar')          # <re.Match object>
re.search(r'bar', 'foobar')         # <re.Match object>
re.fullmatch(r'foo', 'foo')        # <re.Match object>

# 2. Find all / finditer
re.findall(r'\d+', 'a12b34')        # ['12', '34']
re.finditer(r'\d+', 'a12b34')       # iterator over <Match> objects

# 3. Replace
re.sub(r'\d+', '#', 'a12b34')       # 'a#b#'
re.subn(r'\d+', '#', 'a12b34')     # ('a#b#', 2)

# 4. Split
re.split(r'\W+', 'a,b;c')           # ['a', 'b', 'c']

# 5. Escape a literal
re.escape(r'foo.bar*')              # 'foo\\.bar\\*'
```

1.5.1 Regex notes

The "must-know" Python `re` *functions (and their runtimes).*

`re.match(pattern, string, flags=0)` Test start of `string`. *O(n)* avg, exponential worst case.

`re.search(pattern, string, flags=0)` Find first match anywhere in `string`. *O(n)* avg, exponential worst case.

`re.fullmatch(pattern, string, flags=0)` Match entire string. *O(n)* avg, exponential worst case.

`re.compile(pattern, flags=0)` Pre-compile pattern. *O(m)* pattern length.

`re.findall(pattern, string, flags=0)` All non-overlapping matches. *O(n)* avg.

`re.finditer(pattern, string, flags=0)` Iterator of matches. *O(n)* avg.

`re.split(pattern, string, maxsplit=0, flags=0)` Split at matches. *O(n)* avg.

`re.sub(pattern, repl, string, count=0, flags=0)` Replace matches. *O(n)* avg.

`re.subn(pattern, repl, string, count=0, flags=0)` Replace + count. *O(n)* avg.

`re.escape(string)` Escape metacharacters. *O(m)* m = input length.

`re.finditer` + `match.group()` Extract captured groups.

`re.Scanner` **(rare)** Multi-pattern scanner for tokenizers.

General rule of thumb: *If your pattern does not contain nested quantifiers or back-references, the engine runs in* ***linear time*** *in the length of the input string. If you do have nested* `*`, `+`, `{m,n}` *or back-references, the worst-case becomes* ***exponential*** *- this is the "catastrophic backtracking" phenomenon.*

Why the exponential blow-up happens

Python's regex engine is a **back-tracking NFA**. When the engine tries a branch (e.g., a `.` that can match any character) it keeps a stack of alternatives. If the branch fails later, it **pops** to the previous branch and tries the next alternative. If a pattern has k nested alternations, the number of states explored can be as high as $\mathbf{2}^k$.

Typical examples:

```
pattern = r'^(a+)+$'          # matches only "a", but the engine
    explores 2^(len(s)) possibilities
```

Tip: If you ever hit a "too slow" regex, try to rewrite it so that the number of back-tracking branches is bounded or replace it with a deterministic automaton (e.g., use `re.fullmatch(r'[a-z]+')` instead of `re.match(r'.*')`).

Pre-compilation matters

```
pat = re.compile(r'\bfoo\b')
for line in sys.stdin:
    if pat.search(line):
        ...
```

Compiling once keeps the **O(m)** cost to a single place, then every call to `search`, `match`, or `sub` runs in $O(n)$.

Quick-reference cheat-sheet

```
import re

# 1. Basic matching
re.match(r'foo', 'foobar')          # <re.Match object>
re.search(r'bar', 'foobar')         # <re.Match object>
re.fullmatch(r'foo', 'foo')        # <re.Match object>

# 2. Find all / finditer
re.findall(r'\d+', 'a12b34')        # ['12', '34']
re.finditer(r'\d+', 'a12b34')       # iterator over <Match> objects

# 3. Replace
re.sub(r'\d+', '#', 'a12b34')       # 'a#b#'
re.subn(r'\d+', '#', 'a12b34')     # ('a#b#', 2)

# 4. Split
re.split(r'\W+', 'a,b;c')           # ['a', 'b', 'c']

# 5. Escape a literal
re.escape(r'foo.bar*')              # 'foo\\.bar\\*'

# 6. Pre-compile
pat = re.compile(r'\bfoo\b')
for line in sys.stdin:
    if pat.search(line):
        ...
```

Summary of complexities

Complexity summary (all worst cases are 2^k where k is the number of nested quantifiers):

- `match`, `search`, `fullmatch`: $O(n)$ avg. Watch for `.*`, `.+`, or back-references in the pattern.
- `findall`, `finditer`: $O(n)$ avg. Non-overlapping matches.
- `split`: $O(n)$ avg. Avoid `.+?` inside the separator.
- `sub`, `subn`: $O(n)$ avg. Replacement string or callable.
- `escape`: $O(m)$ avg and worst. Any literal.
- `compile`: $O(m)$ avg. Pre-compile once per program run.

Bottom line: For the vast majority of CP problems you will never hit the exponential worst-case. Just avoid nested quantifiers or back-references, and you're safely in linear-time territory. When you do need them, test the regex on a few worst-case inputs to be sure it runs in time.

1.6 Itertools

```
import itertools

a = [1,2,3,4]
b = [5,6,7,8]

# All 2 digit numbers composed of digits 1,2,3
# Cartesian product
for a, b in itertools.product('123', repeat=2):
    print(a+b)

# All k-subsets (combinations)
for team in itertools.combinations(['A','B','C','D','E'],3):
    print(team)

# Subsets with repetitions (combinations_with_replacement)
# All ways to choose 3 balls from 2 colours where each colour can be
    used any number of times
for bag in itertools.combinations_with_replacement(['red','blue'],3)
    :
    print(bag)

# All ways to order 3 letters O(n!)
for order in itertools.permutations('abc'):
    print(''.join(order))

# Prefix XOR
pref_xor = list(itertools.accumulate(a, func=lambda x,y : x^y))

# Group consecutive equal keys

s = "AAABBBCCDA"
for key, group in itertools.groupby(s):
```

```
    print(key, list(group))
# A ['A','A','A']
# B ['B','B','B']
# C ['C','C']
# D ['D']
# A ['A']
```

1.7 Scripts

1.7.1 Combinatorics calculator

```
a,b = [int(x) for x in input.split()]

MAX = 1000001
MOD = 1000000007

fact = [0] * MAX
i_fact = [0] * MAX

def inv(x : int):
    return pow(x, MOD-2,MOD)

def factorial():
    fact[0] = fact[1] = 1
    for i in range(2,MAX):
        fact[i] = (fact[i-1] * i) % MOD
    i_fact[MAX-1] = inv(fact[MAX-1])
    for i in range(MAX-2, -1, -1):
        i_fact[i] = (i_fact[i+1]*(i+1))%MOD

def comb(n : int, k: int):
    return fact[n] * i_fact[n-k]%MOD * i_fact[k] % MOD

factorial()
print(comb(a,b))
```

1.7.2 Random array generator

```
import random

with open("in.txt", 'w') as f:
    n = random.randint(1,100000)
    f.write(str(n)+'\n')
    for i in range(n):
        f.write(str(random.randint(1,100))+' ')
```

1.7.3 Generate all subsets

```
k = int(input())
S = [int(x) for x in input().split()]
subsets = [[S[j] for j in range(k) if (i >> j) & 1] for i in range(1
    << k)]
print(subsets)
```

2 C++ utils

2.1 Common STL Algorithms

Sorting Algorithms

Function	Parameters	Description
sort	begin, end, [comp]	Standard unstable sort (O(n log n))
stable_sort	begin, end, [comp]	Stable sort preserves element order
is_sorted	begin, end, [comp]	Checks if range is sorted (returns bool)
nth_element	begin, nth, end, [comp]	Partitions around nth element

Searching Functions

Function	Parameters	Description
lower_bound	begin, end, val, [comp]	First element $\leq$ value
upper_bound	begin, end, val, [comp]	First element ¿ value
binary_search	begin, end, val, [comp]	Existence check in sorted range
find	begin, end, val	Linear search for value
find_if	begin, end, pred	Find first matching predicate

Sequence Operations

Function	Parameters	Description
reverse	begin, end	Reverse elements in-place
rotate	begin, mid, end	Rotate elements left
next_permutation	begin, end	Generate next permutation
unique	begin, end, [pred]	Remove consecutive duplicates
remove	begin, end, val	Remove elements equal to value

Numerical Functions

Function	Parameters	Description
accumulate	begin, end, init, [op]	Sum/accumulate elements
partial_sum	begin, end, dest, [op]	Compute prefix sums
__gcd	a, b	Greatest common divisor (GCC ext.)
lcm	a, b	Least common multiple (C++17)
iota	begin, end, val	Fill with consecutive values

Memory/Array Operations

Function	Parameters	Description
memset	ptr, value, count	Fill memory with byte value
fill	begin, end, value	Fill range with value
fill_n	begin, count, value	Fill N elements with value
copy	src_b, src_e, dest	Copy range to destination
copy_if	src_b, src_e, dest, pred	Copy elements matching predicate

Utility Functions

Function	Parameters	Description
swap	a, b	Swap two values
max_element	begin, end, [comp]	Find maximum element
min_element	begin, end, [comp]	Find minimum element
count	begin, end, val	Count element occurrences
all_of	begin, end, pred	Check all elements satisfy condition

Mathematical / Bitwise Builtins

Function	Parameters	Description
__builtin_popcount	x (int)	Count number of 1-bits
__builtin_popcountll	x (long long)	Count number of 1-bits (64-bit)
__builtin_clz	x (unsigned int)	Count leading zeros
__builtin_clzll	x (unsigned long long)	Leading zeros (64-bit)
__builtin_ctz	x (unsigned int)	Count trailing zeros
__builtin_ctzll	x (unsigned long long)	Trailing zeros (64-bit)
__builtin_parity	x	Return 1 if #bits is odd
__builtin_ffs	x	Position of least significant 1-bit (1-indexed)
__lg	x	Floor of $\log_2(x)$ (index of highest bit)

Priority Queues and Heaps

Function	Parameters	Description
priority_queue	[type], [container], [comp]	Max-heap by default
make_heap	begin, end, [comp]	Turn range into heap
push_heap	begin, end, [comp]	Push element into heap
pop_heap	begin, end, [comp]	Pop max element into end
sort_heap	begin, end, [comp]	Heap sort

Set / Map Utilities

Operation	Usage	Description
s.lower_bound(x)	set/map	First element $\geq$ x
s.upper_bound(x)	set/map	First element $>$ x
s.equal_range(x)	multiset/map	Pair of lower/upper bound
s.erase(it)	iterator	Erase element at iterator
s.find(x)	key	Iterator to key or end

String Functions

Function	Parameters	Description
stoi, stol, stoll	string, [pos], [base]	Convert string → int/long/ll
stoul, stoull	string, [pos], [base]	Convert string → unsigned
stod, stof, stold	string	Convert string → double/float/long double
to_string	value	Convert number → string
substr	pos, len	Substring
find	str, pos	Find first occurrence
rfind	str, pos	Find last occurrence

Random Number Utilities

Type / Function	Usage	Description
mt19937 rng	chrono::steady_clock::now()	Fast random generator
uniform_int_distribution	dist(a,b)(rng)	Random int in [a,b]
shuffle	begin, end, rng	Random shuffle

Other Useful Utilities

Function	Parameters	Description
chrono::high_resolution_clock	now()	Get precise current time
__int128	value	128-bit integer (manual I/O needed)
bitset $< N >$	ops: &, —, ˆ, ¡¡, ¿¿	Fixed-size bitset manipulation
tuple	get$< i >$(t)	Store and access heterogenous data
pair	first, second	Store pair of values

2.2 Random number generator

```
mt19937 rng(chrono::steady_clock::now().time_since_epoch().count());
```

2.2.1 Graph generator

```
import random
n,m = [int(x) for x in input().split()]

class Edge:
    def __init__(self, u:int, v:int):
        self.u = u
        self.v = v

    def __eq__(self, o):
        return o.u == self.u and o.v == self.v

    def __hash__(self):
        return hash((self.u, self.v))


with open("in.txt",'w') as f:
    f.write(str(n)+" "+str(m)+'\n')
    edges = set()
    for i in range(m):
        u,v = [0,0]
        while u == v or Edge(u,v) in edges:
            u = random.randint(1,n)
            v = random.randint(1,n)
        edges.add(Edge(u,v))
        f.write(str(u)+" "+str(v)+"\n")
    print(edges)
```

2.3 Comparators for Structs

```
bool operator<(const Point& a, const Point& b) {
    if (a.x != b.x) return a.x < b.x;
    return a.y < b.y;
}

struct CompareX {
    bool operator()(const Point& a, const Point& b) const {
        return a.x < b.x;
    }
};

struct Descending {
    bool operator()(const int& a, const int& b) const { return a > b
        ; }
};
```

2.4 Lambda Functions for Sort

```cpp
sort(v.begin(), v.end(), [](const Point& a, const Point& b) {
    return a.x < b.x;
});

auto cmp = [](const auto& a, const auto& b) { return a.x < b.x; };
sort(v.begin(), v.end(), cmp);

auto it = lower_bound(v.begin(), v.end(), target, [](const Point& a,
     const Point& b) {
    return a.x < b.x;
});

partition(v.begin(), v.end(), [](const Point& p) { return p.x < 3;
    });
int cnt = count_if(v.begin(), v.end(), [](const Point& p) { return p
    .x == 1; });

auto mn = min_element(v.begin(), v.end(), [](const Point& a, const
    Point& b) {
    return a.x + a.y < b.x + b.y;
});
auto it2 = find_if(v.begin(), v.end(), [](const Point& p) { return p
    .x == 3; });

for_each(v.begin(), v.end(), [](const Point& p) {
    cout << p.x << "," << p.y << "\n";
});
```

3 Binary Search in the Answer

```cpp
// Standard binary search (iterative)
int binary_search(vector<int>& arr, int target) {
    int left = 0, right = arr.size() - 1;
    while (left <= right) {
        int mid = left + (right - left) / 2;
        if (arr[mid] == target) return mid;
        if (arr[mid] < target) left = mid + 1;
        else right = mid - 1;
    }
    return -1;
}

// Lower bound (first element >= target)
int lower_bound(vector<int>& arr, int target) {
    int left = 0, right = arr.size();
    while (left < right) {
        int mid = left + (right - left) / 2;
        arr[mid] < target ? left = mid + 1
                          : right = mid;
    }
    return left;
}

// Upper bound (first element > target)
int upper_bound(vector<int>& arr, int target) {
    int left = 0, right = arr.size();
    while (left < right) {
        int mid = left + (right - left) / 2;
        arr[mid] <= target ? left = mid + 1
                           : right = mid;
    }
    return left;
}

// Binary search on real numbers (e.g. sqrt)
double sqrt_precision(double n, double eps=1e-6) {
    double left = 0, right = n;
    for (int i = 0; i < 100; ++i) { // or while (right-left > eps)
        double mid = (left + right) / 2;
        if (mid*mid < n) left = mid;
        else right = mid;
    }
    return left;
}

// Binary search on answer space (monotonic condition)
int find_min_valid(vector<int>& nums, int k) {
    auto is_valid = [&](int x) {
        /* condition check */
    };

    int left = 0, right = 1e9; // adjust bounds
    while (left < right) {
        int mid = left + (right - left) / 2;
        is_valid(mid) ? right = mid
                      : left = mid + 1;
    }
    return left;
}

// Binary search using for loop

int binary_search_for_loop(vector<int>& arr, int target) {
    int left = 0, right = arr.size() - 1;
    for (; left <= right; ) {
        int mid = left + (right - left) / 2;
        if (arr[mid] == target) return mid;
        if (arr[mid] < target) left = mid + 1;
        else right = mid - 1;
    }
    return -1;
}
```

3.1 Parallel Binary Search

```

int n,m,q; cin>>n>>m>>q;
vector<vector<query>> timeline(m);
vector<vector<query>> nxt(m);

FO(i,q){
    int a,b; cin>>a>>b;
    timeline[(m-1)/2].push_back({query{}});
}
FO(i,BS){

    FO(i,m) nxt[i].clear();
    for(int j=0;j<m;j++)
    {

        for(query &q:timeline[j])
        {
            if(q.l>q.r) continue;
            if(possible()){

                int mid=(j-1+q.l)/2;
                nxt[mid].push_back(query(q.citya,q.cityb,q.l,j-1,q.
                    id));
            }
            else{
                int mid=(j+1+q.r)/2;
                nxt[mid].push_back(query(q.citya,q.cityb,j+1,q.r,q.
                    id));
            }
        }
    }
    timeline.swap(nxt);
}
```

4 Bitwise Techniques

4.1 All Subsets

```
void minSumSubset(vector<int> &a, ll total_zum){
        int n = (int)a.size();
  ll min_diff = LLONG_MAX;

        for(int i=1; i<(1<< n); i++){
                ll zum = 0;
                for(int j=0; j<n; j++){
                        if((i & (1<<j)) != 0){
                                zum+=a[j];
                        }
                }
    ll diff = abs(total_zum - 2LL * zum);
                min_diff = min(diff,min_diff);
        }

}
```

4.2 FWHT

```
const ll MOD = 1e9+7;

// MAX should be a power of 2 and >= 2*10^5
// FFT works with sizes that are powers of 2
// 2^18 = 262144 > 200000
const int MAX = 1 << 20;

typedef complex<double> cd;
double PI = acos(-1);

// Transformada de Wicho Cachondo
vector<ll> fwht(vector<ll> a, int d = 1) {
    int n = a.size();
    vector<ll> r = a;


    for (int m = 2; m <= n; m *= 2) {
        for (int k = 0; k < n; k += m) {
            for (int j = 0; j < m / 2; j++) {
                ll u = r[k + j];
                ll t = r[k + j + m / 2];

    r[k + j] = u + t;
                r[k + j + m / 2] = u - t;
            }
        }
    }

    if (d == -1) {
        for (int i = 0; i < n; i++)
            r[i] /= n;
    }

    return r;
```

```
}
```

4.3 Gray Code

```
vector<string> grayCode(int n){
    vector<string> arns;
    for(int i=0; i<(1<<n); i++){
  int g = i^(i>>1);
  bitset<32> r(g);
  string arnsito = r.to_string().substr(32-n, 32);
  arns.push_back(arnsito);
    }
    return arns;
}
```

4.4 XOR Basis

```
const int D = 32;
ll basis[D];
ll bs_len = 0;

void add(int x){
    for(int i=0; i<D; i++){
  if((x & 1 << i) == 0) continue;

  if(!basis[i]){
      basis[i] = x;
      bs_len++;
      return;
  }

  x ^= basis[i];
    }
}

void add2(ll x){
    for(int i=D-1; i>=0; i--){
  if((x>>i) & 1){
      if(!basis[i]){
    basis[i] = x;
    return;
      }
      x^=basis[i];
  }
    }
}

```

```
inline void solve(){
    int n; cin>>n;
    vector<int> st(n);
    for(int &x : st) cin>>x;

    for(int vec : st){
  add2(vec);
    }

    ll max_xor = 0;

    for(int i=D-1; i>=0; i--){
  max_xor = max(max_xor, max_xor^basis[i]);
    }

    cout<<max_xor<<endl;
}
```

4.5 Hopcroft-Karp

```
#include <bits/stdc++.h>
using namespace std;

// Hopcroft-Karp: Maximum bipartite matching in O(E * sqrt(V))
// adj[i] contains right-side neighbors of left vertex i (0-indexed)
// Returns size of maximum matching.
// Left vertices: 0..n_l-1, Right vertices: 0..n_r-1

int hopcroft_karp(int n_l, int n_r, const vector<vector<int>>& adj)
    {
    const int INF = 1e9;

    vector<int> pair_l(n_l, -1), pair_r(n_r, -1), dist(n_l);

    auto bfs = [&]() -> bool {
        queue<int> q;
        for (int u = 0; u < n_l; ++u) {
            if (pair_l[u] == -1) {
                dist[u] = 0;
                q.push(u);
            } else {
                dist[u] = INF;
            }
        }

        bool reachable_free = false;
        while (!q.empty()) {
            int u = q.front(); q.pop();
            for (int v : adj[u]) {
                int pu = pair_r[v];
```

```
                if (pu != -1 && dist[pu] == INF) {
                    dist[pu] = dist[u] + 1;
                    q.push(pu);
                }
                if (pu == -1) {
                    reachable_free = true;
                }
            }
        }
        return reachable_free;
    };

    function<bool(int)> dfs = [&](int u) -> bool {
        for (int v : adj[u]) {
            int pu = pair_r[v];
            if (pu == -1 || (dist[pu] == dist[u] + 1 && dfs(pu))) {
                pair_l[u] = v;
                pair_r[v] = u;
                return true;
            }
        }
        dist[u] = INF;
        return false;
    };

    int matching = 0;
    while (bfs()) {
        for (int u = 0; u < n_l; ++u) {
            if (pair_l[u] == -1 && dfs(u))
                ++matching;
        }
    }
    return matching;
}
```

4.6 Min-Cost Max-Flow

```
#include <bits/stdc++.h>
using namespace std;
using ll = long long;

const ll LINF = numeric_limits<ll>::max() / 4;

struct MCMFEdge {
    int to, rev;
    ll cap, cost;
};

struct MinCostMaxFlow {
    int n, s, t;
    ll flow_ = 0, cost_ = 0;
    vector<vector<MCMFEdge>> adj;
    vector<ll> pot, dist;
    vector<int> prevnode, prevedge;

    MinCostMaxFlow(int n, int s, int t) : n(n), s(s), t(t) {
        adj.assign(n, {});
        pot.resize(n, 0);
        dist.resize(n);
        prevnode.resize(n);
        prevedge.resize(n);
    }

    void add_edge(int from, int to, ll cap, ll cost) {
        MCMFEdge a{to, (int)adj[to].size(), cap, cost};
        MCMFEdge b{from, (int)adj[from].size(), 0, -cost};
        adj[from].push_back(a);
        adj[to].push_back(b);
    }

    bool dijkstra() {
        fill(dist.begin(), dist.end(), LINF);
        fill(prevnode.begin(), prevnode.end(), -1);
        priority_queue<pair<ll, int>, vector<pair<ll, int>>, greater
            <pair<ll, int>>> pq;

        dist[s] = 0;
        pq.emplace(0, s);

        while (!pq.empty()) {
            auto [d, v] = pq.top();
            pq.pop();
            if (d != dist[v]) continue;
            for (int i = 0; i < (int)adj[v].size(); i++) {
                MCMFEdge &e = adj[v][i];
                if (e.cap <= 0) continue;
                ll ndist = dist[v] + e.cost + pot[v] - pot[e.to];
                if (ndist < dist[e.to]) {
                    dist[e.to] = ndist;
                    prevnode[e.to] = v;
                    prevedge[e.to] = i;
                    pq.emplace(ndist, e.to);
                }
            }
        }

        if (dist[t] == LINF) return false;

        for (int i = 0; i < n; i++) {
            if (dist[i] < LINF) pot[i] += dist[i];
        }

        return true;
```

```
    }

    void augment() {
        ll addflow = LINF;
        for (int v = t; v != s; v = prevnode[v]) {
            MCMFEdge &e = adj[prevnode[v]][prevedge[v]];
            addflow = min(addflow, e.cap);
        }

        for (int v = t; v != s; v = prevnode[v]) {
            MCMFEdge &e = adj[prevnode[v]][prevedge[v]];
            e.cap -= addflow;
            adj[e.to][e.rev].cap += addflow;
        }

        flow_ += addflow;
        cost_ += addflow * pot[t];
    }

    pair<ll, ll> min_cost_flow(int s, int t) {
        this->s = s;
        this->t = t;
        flow_ = 0;
        cost_ = 0;
        fill(pot.begin(), pot.end(), 0);

        while (dijkstra()) {
            augment();
        }
        return {flow_, cost_};
    }

    ll flow() const { return flow_; }
    ll cost() const { return cost_; }
};
```

4.7 Push-Relabel

```
/**
 * Push-Relabel Max-Flow (Highest-Label + Gap Relabeling)
 * Time: O(V^2 sqrt(E)) worst case, faster than Dinic on dense
     graphs
 * Based on KACTL PushRelabel.h
 */

#include <bits/stdc++.h>
using namespace std;

using ll = long long;
const ll INF = LLONG_MAX;

struct PushRelabel {
    struct Edge {
        int dest, back;
        ll f, c;
    };

    int n;
    vector<vector<Edge>> g;
    vector<ll> ec;        // excess capacity
    vector<Edge*> cur;    // current edge pointer
    vector<vector<int>> hs;
    vector<int> H;

    PushRelabel(int n) : n(n), g(n), ec(n), cur(n), hs(2 * n), H(n)
        {}

    void add_edge(int u, int v, ll cap) {
        if (u == v) return;
        g[u].push_back({v, (int)g[v].size(), 0, cap});
        g[v].push_back({u, (int)g[u].size() - 1, 0, 0});
    }

    void addFlow(Edge& e, ll f) {
        Edge& back = g[e.dest][e.back];
        if (!ec[e.dest] && f) hs[H[e.dest]].push_back(e.dest);
        e.f += f;
        e.c -= f;
        ec[e.dest] += f;
        back.f -= f;
        back.c += f;
        ec[back.dest] -= f;
    }

    ll max_flow(int s, int t) {
        int v = n;
        H[s] = v;
        ec[t] = 1;
        vector<int> co(2 * v);
        co[0] = v - 1;
        for (int i = 0; i < v; i++) cur[i] = g[i].data();

        for (Edge& e : g[s]) addFlow(e, e.c);

        for (int hi = 0;;) {
            while (hs[hi].empty()) if (hi-- == 0) return -ec[s];
            int u = hs[hi].back();
            hs[hi].pop_back();

            while (ec[u] > 0) {  // discharge u
                if (cur[u] == g[u].data() + (int)g[u].size()) {
                    H[u] = INT_MAX;
                    for (Edge& e : g[u])
```

```
                        if (e.c && H[u] > H[e.dest] + 1)
                            H[u] = H[e.dest] + 1, cur[u] = &e;
                    if (++co[H[u]], !--co[hi] && hi < v)
                        for (int i = 0; i < v; i++)
                            if (hi < H[i] && H[i] < v)
                                --co[H[i]], H[i] = v + 1;
                    hi = H[u];
                } else if (cur[u]->c && H[u] == H[cur[u]->dest] + 1)
                    addFlow(*cur[u], min(ec[u], cur[u]->c));
                else
                    ++cur[u];
            }
        }
    }

    // Returns true if vertex is on the source side of min cut
    bool left_of_min_cut(int a) { return H[a] >= n; }
};
```

4.8 0-1 BFS

```
#include <bits/stdc++.h>
using namespace std;
using ll = long long;

vector<ll> zero_one_bfs(int n, int src, const vector<vector<pair<int
    ,int>>>& adj) {
    const ll INF = (ll)1e17;
    vector<ll> dist(n, INF);
    deque<int> dq;
    dist[src] = 0;
    dq.push_back(src);

    while (!dq.empty()) {
        int u = dq.front();
        dq.pop_front();
        for (auto [v, w] : adj[u]) {
            if (dist[u] + w < dist[v]) {
                dist[v] = dist[u] + w;
                if (w == 0) {
                    dq.push_front(v);
                } else {
                    dq.push_back(v);
                }
            }
        }
    }
    return dist;
}
```

4.9 Biconnected Components

```
// Biconnected Components (vertex-biconnected components)
// Uses DFS with edge stack; articulation points split components

#include <algorithm>
#include <vector>
#include <utility>

using namespace std;

int dfsTime = 0;
vector<int> disc, low;
vector<vector<int>> comps;
vector<pair<int,int>> edgeStack;
vector<vector<int>> adj;

// disc[u] = discovery time of u
// low[u] = 'low' node of u (lowest discovery time reachable from u)

void dfsBCC(int u, int parent) {
  disc[u] = low[u] = ++dfsTime;
  for (int v : adj[u]) {
    if (v == parent) continue;
    if (!disc[v]) {
      edgeStack.emplace_back(u, v);
      dfsBCC(v, u);
      low[u] = min(low[u], low[v]);
      if (disc[u] <= low[v]) {
        // u is an articulation point; pop edges to form a component
        comps.push_back({});
        while (edgeStack.back() != make_pair(u, v)) {
          int a = edgeStack.back().first, b = edgeStack.back().
              second;
          comps.back().push_back(a);
          comps.back().push_back(b);
          edgeStack.pop_back();
        }
        // pop the (u,v) edge
        comps.back().push_back(u);
        comps.back().push_back(v);
        edgeStack.pop_back();
      }
    } else if (disc[v] < disc[u]) {
      // back edge to ancestor (avoid duplicates in undirected graph
          )
      low[u] = min(low[u], disc[v]);
      edgeStack.emplace_back(u, v);
    }
  }
}

```

```
vector<vector<int>> biconnected_components(int n, const vector<
    vector<int>>& adj_) {
  adj = adj_;
  disc.assign(n, 0);
  low.assign(n, 0);
  comps.clear();
  edgeStack.clear();
  dfsTime = 0;

  for (int u = 0; u < n; u++) {
    if (!disc[u]) {
      dfsBCC(u, u);
      // remaining edges on stack form a component
      if (!edgeStack.empty()) {
        comps.push_back({});
        while (!edgeStack.empty()) {
          comps.back().push_back(edgeStack.back().first);
          comps.back().push_back(edgeStack.back().second);
          edgeStack.pop_back();
        }
      }
    }
  }

  // remove duplicate vertices in each component
  for (auto& comp : comps) {
    sort(comp.begin(), comp.end());
    comp.erase(unique(comp.begin(), comp.end()), comp.end());
  }

  return comps;
}
```

4.10 Johnson's Algorithm

```
#include <bits/stdc++.h>
using namespace std;
using ll = long long;
const ll INF = 1e17;

vector<ll> dijkstra(int n, int source, const vector<vector<pair<int,
    ll>>>& adj, const vector<ll>& h) {
    vector<ll> dist(n, INF);
    priority_queue<pair<ll,int>, vector<pair<ll,int>>, greater<pair<
        ll,int>>> pq;
    dist[source] = 0;
    pq.push({0, source});

    while (!pq.empty()) {
        auto [d, u] = pq.top();
        pq.pop();
        if (d > dist[u]) continue;

        for (auto [v, w] : adj[u]) {
            ll reweighted = w + h[u] - h[v];
            if (dist[u] + reweighted < dist[v]) {
                dist[v] = dist[u] + reweighted;
                pq.push({dist[v], v});
            }
        }
    }

    return dist;
}

vector<vector<ll>> johnson(int n, const vector<vector<pair<int,ll
    >>>& adj) {
    vector<vector<pair<int,ll>>> aug = adj;
    for (int i = 0; i < n; i++) {
        aug[i].push_back({n, 0});
    }

    vector<ll> dist(n + 1, INF);
    dist[n] = 0;
    for (int i = 0; i < n; i++) {
        bool updated = false;
        for (int u = 0; u <= n; u++) {
            if (dist[u] == INF) continue;
            for (auto [v, w] : aug[u]) {
                if (dist[u] + w < dist[v]) {
                    dist[v] = dist[u] + w;
                    updated = true;
                }
            }
        }
        if (!updated) break;
    }

    vector<ll> h(n);
    for (int i = 0; i < n; i++) h[i] = (dist[i] == INF) ? 0 : dist[i
        ];

    vector<vector<ll>> result(n, vector<ll>(n, INF));
    for (int s = 0; s < n; s++) {
        vector<ll> d = dijkstra(n, s, adj, h);
        for (int t = 0; t < n; t++) {
            if (d[t] != INF) {
                result[s][t] = d[t] - h[s] + h[t];
            }
        }
    }
    return result;
}
```

5 Graph Theory

5.1 Bellman-Ford

```cpp
const ll INF = 1e17;
const ll MIN_INF = (-1)*INF;

vector<ll> dist(MAX, INF);

struct Edge {
    int u, v;
    ll w;
};

void bellmanFord(int V, int E, vector<Edge>& edges, int start) {
    dist[start] = 0;
    vector<int> prev(V, -1);

    for (int i = 0; i < V-1; i++){
        for (int j = 0; j < E; j++){
            int u = edges[j].u;
            int v = edges[j].v;
            ll w = edges[j].w;
            if (dist[u] == INF) continue;
            dist[v] = min(dist[v], w+dist[u]);
            if (dist[v] > dist[u] + w){
                dist[v] = max(MIN_INF, dist[u] + w);
            }
        }
    }

    for (int i = 0; i < V-1; i++){
        for (int j = 0; j < E; j++){
            int u = edges[j].u;
            int v = edges[j].v;
            ll w = edges[j].w;
            if (dist[u] == INF) continue;
            dist[v] = max(MIN_INF,dist[v]);
            if (dist[u] == MIN_INF or dist[v] > dist[u] + w){
                dist[v] = MIN_INF;
            }
        }
    }

}
```

5.2 Dijkstra

```cpp
// Old implementation, use global variables.

const int MAX = 2e5+3;

#define INF LLONG_MAX
typedef pair<ll, ll> pll;

vector<ll> dijkstra(int n, int source, vector<vector<pll>> &adj) {
    vector<ll> dist(n, INF);
    priority_queue<pll, vector<pll>, greater<pll>> pq;
    dist[source] = 0;
    pq.push({0, source});

    while (!pq.empty()) {
        ll d = pq.top().first;
        ll u = pq.top().second;
        pq.pop();

        if (d > dist[u]) continue;

        for (auto &edge : adj[u]) {
            ll v = edge.first;
            ll weight = edge.second;

            if (dist[u] + weight < dist[v]) {
                dist[v] = dist[u] + weight;
                pq.push({dist[v], v});
            }
        }
    }

    return dist;
}
```

5.3 Layered Dijkstra

```cpp
vector<vector<long long>> dist(n + 1, vector<long long>(max_k + 1,
    INF));
    priority_queue<State, vector<State>, greater<State>> pq;

    // Assuming 1 is the source node
    dist[1][0] = 0;
    pq.push({0, 1, 0});

    while (!pq.empty()) {
        auto [d, u, fast_used] = pq.top();
        pq.pop();

```

```
        // Skip outdated states (lazy deletion)
        if (d > dist[u][fast_used]) continue;

        for (const auto& edge : adj[u]) {
            int v = edge.to;
            long long w = edge.weight;

            if (!edge.is_fast) {
                // Transition 1: Take a normal edge
                if (dist[u][fast_used] + w < dist[v][fast_used]) {
                    dist[v][fast_used] = dist[u][fast_used] + w;
                    pq.push({dist[v][fast_used], v, fast_used});
                }
            } else {
                // Transition 2: Take a fast edge (only if we haven't hit the limit)
                if (fast_used < max_k && dist[u][fast_used] + w < dist[v][fast_used + 1]) {
                    dist[v][fast_used + 1] = dist[u][fast_used] + w;
                    pq.push({dist[v][fast_used + 1], v, fast_used + 1});
                }
            }
        }
    }
```

5.4 Dinic

```
const int MAX = 503;

#define INF LLONG_MAX
#define pii pair<int,int>
#define fi first
#define sc second
#define all(x) x.begin(),x.end()

struct FlowEdge{
    int u,v;
    ll flow=0, cap;
    FlowEdge(int u, int v, ll cap) : u(u), v(v), cap(cap){};
};

struct Dinic{
    vector<FlowEdge> edges;
    vector<vector<int>> adj;
    int n, s, t; // Vertices, Source, Sink
    int id = 0;
    vector<int> level, next;
    queue<int> q;

    Dinic(int n, int s, int t) : n(n), s(s), t(t){
        adj.resize(n);
        level.resize(n);
        next.resize(n);
        fill(all(level), -1);
        level[s] = 0;
        q.push(s);
    }


    void addEdge(int u, int v, ll cap){
        edges.emplace_back(u,v,cap);
        edges.emplace_back(v,u,0);
        adj[u].push_back(id++);
        adj[v].push_back(id++);
    }

    bool bfs(){
        while(!q.empty()){
            int u = q.front();
            q.pop();
            for(auto e : adj[u]){
                // Saturated
                if(edges[e].cap - edges[e].flow < 1) continue;
                // Visited
                if(level[edges[e].v] != -1) continue;

                level[edges[e].v] = level[edges[e].u] +1;
                q.push(edges[e].v);
            }
        }

        return level[t] != -1;
    }

    ll dfs(int u, ll flow){
        if(flow == 0) return 0;
        if(u == t) return flow;
        // cid is current id
        for(int &cid = next[u]; cid<adj[u].size(); cid++){
            int e = adj[u][cid];
            int v = edges[e].v;

            if(level[edges[e].u] + 1 != level[v] or edges[e].cap - edges[e].flow < 1)
                continue;
            ll f = dfs(v, min(flow, edges[e].cap - edges[e].flow));
            if(f == 0) continue;

            edges[e].flow += f;
            edges[e^1].flow -= f;
            return f;
        }
```

```
        return 0;
    }

    ll maxFlow(){
        ll flow = 0;
        while(bfs()){ // While there are aug paths
            fill(all(next), 0);
            for(ll f = dfs(s, INF); f!=0ll; f = dfs(s,INF)) flow+=f;
            fill(all(level), -1);
            level[s] = 0;
            q.push(s);
        }

        return flow;
    }

    vector<pii> minCut(){
        vector<pii> arns;
        fill(all(level), -1);
        level[s] = 0;
        q.push(s);
        while(!q.empty()){
            int u = q.front();
            q.pop();
            for(int id = 0; id<adj[u].size(); id++){
                int e = adj[u][id];
                if(level[edges[e].v] == -1 and edges[e].cap - edges[e].flow > 0){
                    q.push(edges[e].v);
                    level[edges[e].v] = level[edges[e].u] + 1;
                }
            }
        }

        for(int i=0; i<level.size(); i++){
            if(level[i] != -1){
                for(int id=0; id<adj[i].size(); id++){
                    int e = adj[i][id];
                    if(level[edges[e].v] == -1 and edges[e].cap - edges[e].flow == 0)
                        arns.emplace_back(edges[e].u, edges[e].v);
                }
            }
        }

        return arns;
    }

    vector<pii> maxMatching(){
        vector<pii> arns;
        fill(all(level), -1); // Reset levels.
        level[s] = 0;
        q.push(s);
        while(!q.empty()){
            int u = q.front();
            q.pop();
            for(int id = 0; id < adj[u].size(); id++){
                int e = adj[u][id];
                if(level[edges[e].v] == -1 and edges[e].cap - edges[e].flow == 0ll and edges[e].flow != 0ll){
                    q.push(edges[e].v);
                    level[edges[e].v] = level[edges[e].u] +1;
                }
            }
        }

        for(int i=0; i<level.size(); i++){
            if(level[i] != -1){
                for(int id=0; id<adj[i].size(); id++){
                    int e = adj[i][id];
                    if(edges[e].u != s and edges[e].v != t and edges[e].cap - edges[e].flow == 0 and edges[e].flow != 0ll)
                        arns.emplace_back(edges[e].u, edges[e].v);
                }
            }
        }
        return arns;
    }

};
```

5.5 Directed Cycles

```
int n; // number of vertices
vector<vector<int>> adj(MAX);
stack<int> st;
vector<bool> visited(MAX);
vector<bool> stFlg(MAX);
vector<int> arns;
int depth = 0;

bool dfs(int v) {
    visited[v] = true;
    st.push(v);
    stFlg[v] = true;
    for(auto &z : adj[v]){
        if(!visited[z])
            if(dfs(z)) return true;

        if(stFlg[z]){
            st.push(z);
```

```
            return true;
        }
    }

    st.pop();
    stFlg[v] = false;
    return false;
}

void printCycle(){
  int tmp = st.top();
  while(!st.empty()){
    arns.push_back(st.top());
    st.pop();
    if(arns.back() == tmp and arns.size() != 1) break;
  }
  reverse(arns.begin(), arns.end());

  cout<<arns.size()<<endl;
  for(auto v: arns) cout<<v<<" ";
  cout<<endl;
}
```

5.6 Edmonds-Karp

```
vector<vector<int>> adj(MAX);
vector<vector<int>> cap(MAX, vector<int>(MAX));

int bfs(int s, int t, vector<int> &parent){
    parent[s] = -2;
    queue<pii> q;
    q.push({s,INF});

    while(!q.empty()){
        int u = q.front().fi;
        int flow = q.front().sc;
        q.pop();

        for(int v : adj[u]){
            if(parent[v] == -1 and cap[u][v]){
                parent[v] = u;
                int new_flow = min(flow, cap[u][v]);
                if(v == t) return new_flow;
                q.push({v, new_flow});
            }
        }
    }
    return 0;
}

int edmond_karp(int s, int t, int n){
    vector<int> parent(n+1,-1);
    int flow = 0;
    int new_flow;
    while(new_flow = bfs(s,t,parent)){
        flow += new_flow;
        int cur = t;
        while(cur != s){
            int prev = parent[cur];
            cap[prev][cur] -=  new_flow;
            cap[cur][prev] += new_flow;
            cur = prev;
        }
        fill(parent.begin(), parent.end(), -1);
    }

    return flow;
}
```

5.7 Floyd-Warshall

```
 void floydWarshall(vector<vector<ll>> &d, int n){
    for (int k = 0; k < n; ++k) {
        for (int i = 0; i < n; ++i) {
            for (int j = 0; j < n; ++j) {
                d[i][j] = min(d[i][j], d[i][k] + d[k][j]);
            }
        }
    }
}
```

5.8 Kahn Topological Sort

```
def kahnTopoSort(self,adj: List[List[int]]) -> List[int]:
        #print(adj)
        in_deg = [0] * len(adj)
        for i in range(len(adj)):
            for u in adj[i]:
                in_deg[u]+=1

        q = []
        for i in range(len(in_deg)):
            if in_deg[i] == 0:
                q.append(i)

        arns = []
        while len(q)>0:
```

```
            u = q[0]
            q.pop(0)
            arns.append(u)

            for v in adj[u]:
                in_deg[v]-=1
                if in_deg[v] == 0:
                    q.append(v)

        print(str(len(arns))+" "+str(len(adj)))
        if(len(arns) != len(adj)):
            return []

        return arns
```

5.9 Kruskal

```
struct Edge {
    int u, v, w;
    bool operator<(Edge const& other) {
        return w < other.w;
    }
};

class DisjointSets {
private:
    vector<int> parents;
    vector<int> rank; // Use rank instead of size

public:
    DisjointSets(int size) : parents(size), rank(size, 0) {
        for (int i = 0; i < size; i++) parents[i] = i;
    }

    int find(int x) {
        if (parents[x] != x) parents[x] = find(parents[x]);
        return parents[x];
    }

    bool unite(int x, int y) {
        int x_root = find(x);
        int y_root = find(y);
        if (x_root == y_root) return false;

        if (rank[x_root] < rank[y_root]) swap(x_root, y_root);
        parents[y_root] = x_root;
        if (rank[x_root] == rank[y_root]) rank[x_root]++;
        return true;
    }
};
```

```


int kruskal(int n, vector<Edge> &edges, vector<Edge> &ans){
    ll cost = 0;
    int ed_cnt = 0;
    DisjointSets dsu(n);
    sort(edges.begin(), edges.end());

    for(Edge e : edges){
        if(dsu.unite(e.u, e.v)){
            ans.push_back(e);
            cost += e.w;
            ed_cnt++;
        }
    }

    if (ed_cnt != n - 1) return -1;
    return cost;
}
```

5.10 Lava Flow

```
struct Cell{
    int x,y,t;
};

const int MAX = 1005;
int n,m;

char maze[MAX][MAX];
int vis[MAX][MAX];
int player[MAX][MAX];
char path[MAX][MAX];
set<pii> isExit;
queue<Cell> q;
string res;

bool isValid(int x, int y){
    if(x < 0 || x >= n || y < 0 || y >= m) return false;
    if(maze[x][y] == '#') return false;
    return true;
}

bool isSafe(int x, int y, int u, int v){
    return player[x][y] == -1 and maze[x][y] != 'M' and (vis[x][y]
        == -1 or player[u][v] + 1 < vis[x][y]);
}


void restorePath(int u, int v, int x, int y){
```

```
    while (x != u || y != v) {
        res.push_back(path[u][v]);

        if (res.back() == 'U') u++;
        if (res.back() == 'D') u--;
        if (res.back() == 'L') v++;
        if (res.back() == 'R') v--;
    }
}

bool lavaFlow(int x,int y){
    q.push({x,y,1});
    player[x][y] = 0;

  while(!q.empty()){
    int u =  q.front().x;
    int v = q.front().y;
        int t = q.front().t;

    q.pop();

        vector<pii> dir = {{1, 0}, {-1, 0}, {0, 1}, {0, -1}};

    for(auto it: dir){
            int i = u+it.fs;
            int j = v+it.sc;

            if(isValid(i,j)){
                if(t == 0){
                    if(vis[i][j] == -1){
                        vis[i][j] = vis[u][v]+1;
                        q.push(Cell{i,j,0});
                    }
                }else{
                    if(isSafe(i,j,u,v)){
                        path[i][j] = (it.fs == 1) ? 'D' : (it.fs ==
                            -1) ? 'U' : (it.sc == 1) ? 'R' : 'L';
                        player[i][j] = player[u][v]+1;
                        q.push(Cell{i,j,1});
                        if (isExit.find({i,j}) != isExit.end()) {
                            if (player[i][j] < vis[i][j] || vis[i][j
                                ] == -1) {
                                restorePath(i, j, x, y);
                                return true;
                            }
                        }
                    }
                }
            }

        }
  }

  return false;
}
```

5.11 Negative Cycles

```
struct Edge {
    int u, v;
    ll w;
};

void bellmanFord(int V, int E, vector<Edge>& edges, int start) {
    vector<ll> dist(V, INF);
    dist[start] = 0;
    vector<int> prev(V, -1);

    int x;
    for (int i = 0; i < V; i++){
        x = -1;
        for (int j = 0; j < E; j++){
            int u = edges[j].u;
            int v = edges[j].v;
            ll weight = edges[j].w;
            if (dist[u] != INF && dist[u] + weight < dist[v]) {
                dist[v] = max(-INF, (ll) dist[u] + weight);
                prev[v] = u;
                x = v;
            }
        }
    }

    if(x == -1){
        cout<<"NO"<<endl; // No negative Cycle
    }else{
        cout<<"YES"<<endl;
        int curr = x;
        for (int i = 0; i < V; ++i)
            curr = prev[curr];

        vector<int> cycle;
        for (int x = curr;; x = prev[x]) {
      cycle.push_back(x);
      if (x == curr and cycle.size() > 1) break;
    }


        reverse(cycle.begin(), cycle.end());

        for(int v : cycle)
            cout<<v+1<<" ";
        //cout<<cycle[0]+1<<endl;
```

```
    }


}


void solve(){
    int n,m; cin>>n>>m;
    vector<Edge> edges;
    for(int i=0; i<m; i++){
        int u,v; ll w; cin>>u>>v>>w;
        edges.push_back({u-1,v-1,w});
    }


    int dummyNode = n;
    for (int i = 0; i < n; ++i) {
        edges.push_back({dummyNode, i, 0LL});
    }

    bellmanFord(n + 1, m + n, edges, dummyNode);

}
```

5.12 Prim Algorithm

```
const int MAX = 2e5+3;

#define INF LLONG_MAX
typedef pair<ll, ll> pll;

ll prim(int V, int E, vector<vector<pll>> &adj) {

    priority_queue<pll, vector<pll>, greater<pll>> pq;

    vector<bool> visited(V, false);

    ll res = 0;

    pq.push({0, 0});

    while(!pq.empty()){
        auto p = pq.top();
        pq.pop();

        int wt = p.first;
        int u = p.second;

        if(visited[u] == true){
            continue;
        }

        res += wt;
        visited[u] = true;

        for(auto v : adj[u]){
            if(visited[v.first] == false){
                pq.push({v.second, v.first});
            }
        }
    }

    for(int i=0; i<V; i++){
        if(!visited[i])
            return -1;
    }

    return res;
}
```

5.13 Strongly Connected Components

```

const int MAX = 2e5+3;

#define INF LLONG_MAX
typedef pair<ll, ll> pll;

vector<vector<int>> adj(MAX);
vector<vector<int>> t_adj(MAX);
vector<int> v_id(MAX);
bool visited[MAX];
vector<int> order; // Append the respective nodes with dfsA order

// Calculate vertex order
void dfsA(int s){
    visited[s] = true;
    for(int u : adj[s]){
        if(!visited[u])
            dfsA(u);
    }
    order.push_back(s);
}

// Set its component id to every vertex
void dfsB(int s, int c){
    visited[s] = true;
    for(int u: t_adj[s]){
        if(!visited[u])
            dfsB(u,c);
```

```
    }
    v_id[s] = c;
}


void solve(){
    int n,m; cin>>n>>m;

    for(int i=0; i<m; i++){
        int u,v; cin>>u>>v;
        adj[u].push_back(v);
        t_adj[v].push_back(u);
    }

    for(int i=1; i<=n; i++){
        if(!visited[i])
            dfsA(i);
    }

    for(int i=1; i<=n; i++)
        visited[i] = false;

    reverse(order.begin(),order.end());

    int c = 0;
    for(int i=0; i<order.size(); i++){
        if(!visited[order[i]]){
            c++;
            dfsB(order[i],c);
        }
    }

    if(c == 1){
        cout<<"YES"<<endl;
    }else{
        bool flag1 = false, flag2 = false;
        cout<<"NO"<<endl;
        int arns[2];
        for(int i=1; i<=n; i++){
            if(v_id[i] == 1 and flag1 == false){
                flag1 = true;
                arns[0] = i;
            }

            if(v_id[i] == 2 and flag2 == false){
                flag2 = true;
                arns[1] = i;
            }

            if(flag1 == true and flag2 ==true) break;
        }
        cout<<arns[1]<<" "<<arns[0]<<endl;
    }
}
```

5.14 Topological Sort

```
void dfs(int v) {
    visited[v] = true;
    for (int u : adj[v]) {
        if (!visited[u])
            dfs(u);
    }
    ans.push_back(v);
}

void topological_sort() {
    visited.assign(n+1, false);
    ans.clear();
    for (int i = 1; i <= n; ++i) {
        if (!visited[i]) {
            dfs(i);
        }
    }
    reverse(ans.begin(), ans.end());
}
```

5.15 2-SAT

```
// Kosaraju Algorithm
struct TwoSAT{
    int vars;
    int n;
    vector<vector<int>> adj, adj_t;
    vector<bool> used;
    vector<int> order, comp;
    vector<bool> values;

    TwoSAT(int _vars) :
        vars(_vars),
        n(2 * vars),
        adj(n),
        adj_t(n),
        used(n),
        order(),
        comp(n,-1),
        values(n){
            order.reserve(n);
        }

```

```
    void dfsA(int s){
        used[s] = true;
        for(int u : adj[s]){
            if(!used[u]) dfsA(u);
        }
        order.push_back(s);
    }

    void dfsB(int s, int sc){
        comp[s] = sc;
        for(int u : adj_t[s]){
            if(comp[u] == -1) dfsB(u,sc);
        }
    }

    bool twoSAT(){
        order.clear();
        used.assign(n, false);

        for(int i=0; i<n; i++){
            if(!used[i]) dfsA(i);
        }

        comp.assign(n, -1);

        for(int i=0, j=0; i<n; i++){
            int v = order[n-i-1];
            if(comp[v] == -1) dfsB(v, j++);
        }

        values.assign(vars, false);

        for(int i=0; i<n; i+=2){
            if(comp[i] == comp[i+1]) return false;
            values[i/2] = comp[i] > comp[i+1];
        }
        return true;
    }

    void add_disjunction(int a, bool na, int b, bool nb){
        a = 2* a ^ na;
        b = 2* b ^ nb;

        int not_a = a^1;
        int not_b = b^1;

        adj[not_a].push_back(b);
        adj[not_b].push_back(a);
        adj_t[b].push_back(not_a);
        adj_t[a].push_back(not_b);
    }
};
```

5.16 Undirected Cycles

```
bool dfs(int v, int par) { // passing vertex and its parent vertex
    visited[v] = true;
    for (int u : adj[v]) {
        if(u == par) continue; // skipping edge to parent vertex
        if (visited[u]) {
            cycle_end = v;
            cycle_start = u;
            return true;
        }
        parent[u] = v;
        if (dfs(u, parent[u]))
            return true;
    }
    return false;
}

void find_cycle() {
    visited.assign(n+1, false);
    parent.assign(n+1, -1);
    cycle_start = -1;

    for (int v = 0; v < n; v++) {
        if (!visited[v] && dfs(v, parent[v]))
            break;
    }

    if (cycle_start == -1) {
        cout << "IMPOSSIBLE" << endl;
    } else {
        vector<int> cycle;
        cycle.push_back(cycle_start);
        for (int v = cycle_end; v != cycle_start; v = parent[v])
            cycle.push_back(v);
        cycle.push_back(cycle_start);

        cout << cycle.size()<<endl;;
        for (int v : cycle)
            cout << v << " ";
        cout << endl;
    }
}
```

5.17 Articulation Points

```
// adj[u] = adjacent nodes of u
// ap = AP = articulation points
// p = parent
// disc[u] = discovery time of u
```

```cpp
// low[u] = 'low' node of u

int dfsAP(int u, int p) {
  int children = 0;
  low[u] = disc[u] = ++Time;
  for (int& v : adj[u]) {
    if (v == p) continue; // we don't want to go back through the same path.
                          // if we go back is because we found another way back
    if (!disc[v]) { // if V has not been discovered before
      children++;
      dfsAP(v, u); // recursive DFS call
      if (disc[u] <= low[v]) // condition #1
        ap[u] = 1;
      low[u] = min(low[u], low[v]); // low[v] might be an ancestor of u
    } else // if v was already discovered means that we found an ancestor
      low[u] = min(low[u], disc[v]); // finds the ancestor with the least discovery time
  }
  return children;
}

void AP() {
  ap = low = disc = vector<int>(adj.size());
  Time = 0;
  for (int u = 0; u < adj.size(); u++)
    if (!disc[u])
      ap[u] = dfsAP(u, u) > 1; // condition #2
}
```

5.18 Bridges

```cpp
// br = bridges, p = parent

vector<pair<int, int>> br;

int dfsBR(int u, int p) {
  low[u] = disc[u] = ++Time;
  for (int& v : adj[u]) {
    if (v == p) continue; // we don't want to go back through the same path.
                          // if we go back is because we found another way back
    if (!disc[v]) { // if V has not been discovered before
      dfsBR(v, u);  // recursive DFS call
      if (low[v] > disc[u]) // condition to find a bridge
        br.push_back({u, v});
      low[u] = min(low[u], low[v]); // low[v] might be an ancestor of u
    } else // if v was already discovered means that we found an ancestor
      low[u] = min(low[u], disc[v]); // finds the ancestor with the least discovery time
  }
}

void BR() {
  low = disc = vector<int>(adj.size());
  Time = 0;
  for (int u = 0; u < adj.size(); u++)
    if (!disc[u])
      dfsBR(u, u);
}
```

5.19 Bridges Online

```cpp
vector<int> par, dsu_2ecc, dsu_cc, dsu_cc_size;
int bridges;
int lca_iteration;
vector<int> last_visit;

void init(int n) {
    par.resize(n);
    dsu_2ecc.resize(n);
    dsu_cc.resize(n);
    dsu_cc_size.resize(n);
    lca_iteration = 0;
    last_visit.assign(n, 0);
    for (int i=0; i<n; ++i) {
        dsu_2ecc[i] = i;
        dsu_cc[i] = i;
        dsu_cc_size[i] = 1;
        par[i] = -1;
    }
    bridges = 0;
}

int find_2ecc(int v) {
    if (v == -1)
        return -1;
    return dsu_2ecc[v] == v ? v : dsu_2ecc[v] = find_2ecc(dsu_2ecc[v]);
}

int find_cc(int v) {
    v = find_2ecc(v);
    return dsu_cc[v] == v ? v : dsu_cc[v] = find_cc(dsu_cc[v]);
```

```
}

void make_root(int v) {
    int root = v;
    int child = -1;
    while (v != -1) {
        int p = find_2ecc(par[v]);
        par[v] = child;
        dsu_cc[v] = root;
        child = v;
        v = p;
    }
    dsu_cc_size[root] = dsu_cc_size[child];
}

void merge_path (int a, int b) {
    ++lca_iteration;
    vector<int> path_a, path_b;
    int lca = -1;
    while (lca == -1) {
        if (a != -1) {
            a = find_2ecc(a);
            path_a.push_back(a);
            if (last_visit[a] == lca_iteration){
                lca = a;
                break;
                }
            last_visit[a] = lca_iteration;
            a = par[a];
        }
        if (b != -1) {
            b = find_2ecc(b);
            path_b.push_back(b);
            if (last_visit[b] == lca_iteration){
                lca = b;
                break;
                }
            last_visit[b] = lca_iteration;
            b = par[b];
        }

    }

    for (int v : path_a) {
        dsu_2ecc[v] = lca;
        if (v == lca)
            break;
        --bridges;
    }
    for (int v : path_b) {
        dsu_2ecc[v] = lca;
        if (v == lca)
            break;
        --bridges;
    }
}

void add_edge(int a, int b) {
    a = find_2ecc(a);
    b = find_2ecc(b);
    if (a == b)
        return;

    int ca = find_cc(a);
    int cb = find_cc(b);

    if (ca != cb) {
        ++bridges;
        if (dsu_cc_size[ca] > dsu_cc_size[cb]) {
            swap(a, b);
            swap(ca, cb);
        }
        make_root(a);
        par[a] = dsu_cc[a] = b;
        dsu_cc_size[cb] += dsu_cc_size[a];
    } else {
        merge_path(a, b);
    }
}
```

5.20 Bridge Tree

```
vector<pair<int, int>> g[MAXN];
bool used[MAXN], isBridge[MAXM];
int comp[MAXN], tin[MAXN], minAncestor[MAXN];

vector<int> tree[MAXN]; // Store 2-edge-connected component tree.(
    Bridge tree).

void dfs(int v, int p) {
    tin[v] = minAncestor[v] = ++timer;
    used[v] = 1;
    for(auto &e: g[v]) {
        int to, id;
        tie(to, id) = e;
        if(to == p) continue;
        if(used[to]) {
            minAncestor[v] = min(minAncestor[v], tin[to]);
        } else {
            dfs(to, v);
            minAncestor[v] = min(minAncestor[v], minAncestor[to]);
            if(minAncestor[to] > tin[v]) {
                isBridge[id] = true;
```

```
            }
        }
    }
}

void dfs1(int v, int p) {
    used[v] = 1;
    comp[v] = compid;
    for(auto &e: g[v]) {
        int to, id;
        tie(to, id) = e;

        if(isBridge[id]) { // avoid traversing from this edge. so we get full component.
            continue;
        }
        if(used[to]) {
            continue;
        }
        dfs1(to, v);
    }
}

vector<pair<int, int>> edges;

void addEdge(int from, int to, int id) {
    g[from].push_back({to, id});
    g[to].push_back({from, id});
    edges[id] = {from, to};
}

void initB() {

    for(int i = 0; i <= compid; ++i)
        tree[i].clear();
    for(int i = 1; i <= N; ++i)
        used[i] = false;
    for(int i = 1; i <= M; ++i)
        isBridge[i] = false;

    timer = 0;
    compid = 0;
}

void bridge_tree() {

    initB();

    dfs(1, -1); //Assuming graph is connected.

    for(int i = 1; i <= N; ++i)
        used[i] = 0;

    for(int i = 1; i <= N; ++i) {
        if(!used[i]) {
            dfs1(i, -1);
            ++compid;
        }
    }

    for(int i = 1; i <= M; ++i) {
        if(isBridge[i]) {
            int u, v;
            tie(u, v) = edges[i];
            // connect two componets using edge.
            tree[comp[u]].push_back(comp[v]);
            tree[comp[v]].push_back(comp[u]);
        }
    }
}

void init() {
    edges.clear(); edges.resize(M + 1);
    for(int i = 1; i <= N; ++i)
        g[i].clear();
}
```

5.21 Dynamic Connectivity

```
const int BS=18;
//ascii https://elcodigoascii.com.ar/


struct query{
    int citya,cityb,l,r,id;
    query(int citya, int cityb, int l, int r,int id): citya(citya),
        cityb(cityb), l(l), r(r),
    id(id)
    {}
};

class DSU{
    private:
        vector<int> leader;
        vector<int> sz;
        int n;

        int getLeader(int a){
            if(leader[a]==a) return a;
            leader[a]=getLeader(leader[a]);
            return leader[a];
        }

```

```
    public:
        DSU(int n): n(n){
            leader=vector<int> (n+1);
            sz=vector<int> (n+1,1);
            FO(i,n+1){
                leader[i]=i;
            }
        }

        void clear(){
            leader=vector<int> (n+1);
            sz=vector<int> (n+1,1);
            FO(i,n+1){
                leader[i]=i;
            }
        }

        void unite(int a,int b){
            a=getLeader(a);
            b=getLeader(b);
            if(a==b) return;
            if(sz[a]<sz[b]){
                swap(a,b);
            }
            leader[b]=a;
            sz[a]+=sz[b];
        }

        bool isConnected(int a,int b){
            return getLeader(a)==getLeader(b);
        }

};


inline void solve()
{
    int n,m,q; cin>>n>>m>>q;
    vector<vector<query>> timeline(m);
    vector<vector<query>> nxt(m);
    vector<pair<int,int>> edges(m);
    vector<int> res(q,-2);
    DSU dj(n);
    FO(i,m){
        int a,b; cin>>a>>b;
        edges[i]={a,b};
    }

    FO(i,q){
        int a,b; cin>>a>>b;
        if(a==b) res[i]=-1;
        else timeline[(m-1)/2].push_back({a,b,0,m-1,i});
    }
    FO(i,BS){
        dj.clear();
        FO(i,m) nxt[i].clear();
        for(int j=0;j<m;j++)
        {
            dj.unite(edges[j].X,edges[j].Y);

            for(query &q:timeline[j])
            {
                if(q.l>q.r) continue;
                if(dj.isConnected(q.citya,q.cityb)){

                    res[q.id]=j;
                    int mid=(j-1+q.l)/2;
                    nxt[mid].push_back(query(q.citya,q.cityb,q.l,j
                        -1,q.id));
                }
                else{
                    int mid=(j+1+q.r)/2;
                    nxt[mid].push_back(query(q.citya,q.cityb,j+1,q.r
                        ,q.id));
                }
            }
        }
        timeline.swap(nxt);
    }
    for(int r:res){
        cout<<r+1<<endl;
    }
}
```

5.22 Eulerian Path

```
int n, m;
vector<vector<int>> g;
vector<int> in, out, path;

// Undirected

int n, m;
vector<vector<pair<int, int>>> g;
vector<int> path;
vector<bool> seen;

void dfs(int node) {
  while (!g[node].empty()) {
    auto [son, idx] = g[node].back();
    g[node].pop_back();
    if (seen[idx]) { continue; }
    seen[idx] = true;
```

```
    dfs(son);
  }
  path.push_back(node);
}

// Directed
void dfs(int node) {
  while (!g[node].empty()) {
    int son = g[node].back();
    g[node].pop_back();
    dfs(son);
  }
  path.push_back(node);
}
```

5.23 Gale-Shapley

```
// Checks if woman 'w' prefers 'm1' over 'm'
bool wPrefersM1OverM(vector<vector<int>> &prefer, int w, int m, int
    m1)
{
    int N = prefer[0].size();
    for (int i = 0; i < N; i++)
    {
        // If m1 comes before m, w prefers
        // her current engagement
        if (prefer[w][i] == m1)
            return true;

        // If m comes before m1, w prefers m
        if (prefer[w][i] == m)
            return false;
    }
}

// Implements the stable marriage algorithm
vector<int> stableMarriage(vector<vector<int>> &prefer)
{
    int N = prefer[0].size();

    // Stores women's partners
    vector<int> wPartner(N, -1);

    // Tracks free men
    vector<bool> mFree(N, false);
    int freeCount = N;

    while (freeCount > 0)
    {
        int m;
        for (m = 0; m < N; m++)
            if (!mFree[m])
                break;

        // Process each woman in m's preference list
        for (int i = 0; i < N && !mFree[m]; i++)
        {
            int w = prefer[m][i];
            if (wPartner[w - N] == -1)
            {
                // Engage m and w if w is free
                wPartner[w - N] = m;
                mFree[m] = true;
                freeCount--;
            }
            else
            {
                int m1 = wPartner[w - N];
                // If w prefers m over her current partner, reassign
                if (!wPrefersM1OverM(prefer, w, m, m1))
                {
                    wPartner[w - N] = m;
                    mFree[m] = true;
                    mFree[m1] = false;
                }
            }
        }
    }
    return wPartner;
}
```

5.24 Hungarian Algorithm

```
vector<int> u (n+1), v (m+1), p (m+1), way (m+1);
for (int i=1; i<=n; ++i) {
    p[0] = i;
    int j0 = 0;
    vector<int> minv (m+1, INF);
    vector<bool> used (m+1, false);
    do {
        used[j0] = true;
        int i0 = p[j0],  delta = INF,  j1;
        for (int j=1; j<=m; ++j)
            if (!used[j]) {
                int cur = A[i0][j]-u[i0]-v[j];
                if (cur < minv[j])
                    minv[j] = cur,  way[j] = j0;
                if (minv[j] < delta)
                    delta = minv[j],  j1 = j;
            }
```

```
        for (int j=0; j<=m; ++j)
            if (used[j])
                u[p[j]] += delta,  v[j] -= delta;
            else
                minv[j] -= delta;
        j0 = j1;
    } while (p[j0] != 0);
    do {
        int j1 = way[j0];
        p[j0] = p[j1];
        j0 = j1;
    } while (j0);
}

vector<int> ans (n+1);
for (int j=1; j<=m; ++j)
    ans[p[j]] = j;

int cost = -v[0];
```

5.25 SPFA

```
const int INF = 1000000000;
vector<vector<pair<int, int>>> adj;

bool spfa(int s, vector<int>& d) {
    int n = adj.size();
    d.assign(n, INF);
    vector<int> cnt(n, 0);
    vector<bool> inqueue(n, false);
    queue<int> q;

    d[s] = 0;
    q.push(s);
    inqueue[s] = true;
    while (!q.empty()) {
        int v = q.front();
        q.pop();
        inqueue[v] = false;

        for (auto edge : adj[v]) {
            int to = edge.first;
            int len = edge.second;

            if (d[v] + len < d[to]) {
                d[to] = d[v] + len;
                if (!inqueue[to]) {
                    q.push(to);
                    inqueue[to] = true;
                    cnt[to]++;
                    if (cnt[to] > n)
                        return false;  // negative cycle
                }
            }
        }
    }
    return true;
}
```

5.26 Tarjan

```
/** Takes in an adjacency list and calculates the SCCs of the graph.
    */
class TarjanSolver {
  private:
  vector<vector<int>> rev_adj;
  vector<int> post;
  vector<int> comp;

  vector<bool> visited;
  int timer = 0;
  int id = 0;

  void fill_post(int at) {
    visited[at] = true;
    for (int n : rev_adj[at]) {
      if (!visited[n]) { fill_post(n); }
    }
    post[at] = timer++;
  }

  void find_comp(int at) {
    visited[at] = true;
    comp[at] = id;
    for (int n : adj[at]) {
      if (!visited[n]) { find_comp(n); }
    }
  }

  public:
  const vector<vector<int>> &adj;

  TarjanSolver(const vector<vector<int>> &adj)
      : adj(adj), rev_adj(adj.size()), post(adj.size()), comp(adj.
          size()),
        visited(adj.size()) {
    vector<int> nodes(adj.size());
    for (int n = 0; n < adj.size(); n++) {
      nodes[n] = n;
      for (int next : adj[n]) { rev_adj[next].push_back(n); }
```

```
    }

    for (int n = 0; n < adj.size(); n++) {
      if (!visited[n]) { fill_post(n); }
    }
    std::sort(nodes.begin(), nodes.end(),
              [&](int n1, int n2) { return post[n1] > post[n2]; });

    visited.assign(adj.size(), false);
    for (int n : nodes) {
      if (!visited[n]) {
        find_comp(n);
        id++;
      }
    }
  }

  int comp_num() const { return id; }

  int get_comp(int n) const { return comp[n]; }
};
```

5.27 Sqrt Decomposition

```
// Sqrt Decomposition -- range sum queries & point updates in O((N+Q)*sqrt(N))
#include <vector>
#include <cmath>

using ll = long long;

template <typename T>
class SqrtDecomp {
    int n, sz;
    std::vector<T> a;
    std::vector<ll> block;

public:
    SqrtDecomp(const std::vector<T>& arr) {
        n = (int)arr.size();
        sz = (int)std::sqrt(n) + 1;
        a = arr;
        block.assign((n + sz - 1) / sz, 0);
        for (int i = 0; i < n; i++)
            block[i / sz] += arr[i];
    }

    // point update: set a[i] = val
    void update(int i, T val) {
        block[i / sz] += val - a[i];
        a[i] = val;
    }

    // inclusive range query [l, r]
    ll query(int l, int r) const {
        ll res = 0;
        int start = l / sz, end = r / sz;
        if (start == end) {
            for (int i = l; i <= r; i++)
                res += a[i];
            return res;
        }
        // left partial block
        for (int i = l; i < (start + 1) * sz; i++)
            res += a[i];
        // full blocks
        for (int b = start + 1; b < end; b++)
            res += block[b];
        // right partial block
        for (int i = end * sz; i <= r; i++)
            res += a[i];
        return res;
    }
};
```

6 Data Structures

6.1 2D Segment Tree

```
class SegmentTree2D{
private:
    vector<vector<int>> segtree;
    int n;
    int m;
    int d1;
    int d2;

public:

    void build(vector<vector<int>> &a){
        n = a.size();
        m = a[0].size();
        d1=1;
        while(d1<n) d1<<=1;

        d2=1;
        while(d2<m) d2<<=1;

        segtree.assign(2*d1, vector<int>(2*d2,0));
```

```
21
22          // Set leaves
23          for(int i=0; i<n; i++)
24              for(int j=0; j<m; j++)
25                  segtree[d1+i][d2+j] = a[i][j];
26
27          // Propagate on columns
28          for(int i=d1; i<2*d1; i++){
29              for(int j=d2-1; j>0; j--)
30                  segtree[i][j] = segtree[i][2*j] + segtree[i][2*j+1];
31          }
32
33          // Propagate on rows
34          for(int i=d1-1; i>0; i--){
35              for(int j=1; j<2*d2; j++)
36                  segtree[i][j] = segtree[2*i][j] + segtree[2*i+1][j];
37          }
38      }
39
40      void update(int x, int y, int k){
41          int r = x + d1;
42          int c = y + d2;
43
44          segtree[r][c] = k;
45
46          // Update columns for the row corresponding to x
47          for(int j=c>>1; j>0; j>>=1)
48              segtree[r][j] = segtree[r][2*j] + segtree[r][2*j+1];
49
50
51          for(int i=r>>1; i>0; i>>=1){
52              segtree[i][c] = segtree[2*i][c] + segtree[2*i+1][c];
53              for(int j=c>>1; j>0; j>>=1)
54                  segtree[i][j] = segtree[i][2*j] + segtree[i][2*j+1];
55          }
56      }
57
58
59
60      int query(int x1, int y1, int x2, int y2){
61          int lx = x1+d1, rx = x2+d1+1;
62
63          ll arns = 0;
64          while(lx<rx) {
65              if(lx & 1){
66                  arns += query_y(lx, y1, y2);
67                  ++lx;
68              }
69              if (rx & 1){
70                  --rx;
71                  arns += query_y(rx, y1, y2);
72              }
73              lx >>= 1; rx >>= 1;
74          }
75          return arns;
76      }
77
78      int query_y(int row, int y1, int y2) {
79          int ly = y1 + d2, ry = y2 + 1 + d2;
80          int s = 0;
81          while (ly < ry) {
82              if(ly & 1) s+=segtree[row][ly++];
83              if(ry & 1) s+=segtree[row][--ry];
84              ly >>= 1; ry >>= 1;
85          }
86          return s;
87      }
88
89  };
```

6.2 Bit Trie

```
1   class Trie{
2   private:
3       Trie *children[2];
4       bool term_st; // is_end
5
6       Trie* searchPrefix(int x){
7           Trie* node = this;
8
9           for(int i=29; i>=0; i--){
10              int bit = (x>>i)&1;
11              if(!node->children[bit]) return nullptr;
12              node = node->children[bit];
13          }
14
15          return node;
16      }
17
18  public:
19      Trie() : term_st(false) {
20          for(int i=0; i<2; i++)
21              children[i] = nullptr;
22      }
23
24      void insert(int x) {
25          Trie* node = this;
26          for(int i=29; i>=0; i--){
27              int bit = (x>>i)&1;
28              if(!node->children[bit]) node->children[bit] = new Trie
                    ();
29              node = node->children[bit];
30          }
```

```
31
32          node->term_st = true;
33      }
34
35      bool search(int x) {
36          Trie* node = searchPrefix(x);
37          return node != nullptr && node->term_st;
38      }
39
40      bool startsWith(int x) {
41          Trie* node = searchPrefix(x);
42          return node != nullptr;
43      }
44
45      int maxXor(int x){
46          Trie* node = this;
47          int arns = 0;
48          for(int i=29; i>=0; i--){
49              int bit = (x>>i)&1;
50              int not_bit = 1-bit;
51              if(node->children[not_bit]){
52                  arns|=(1<<i);
53                  node = node->children[not_bit];
54              }else{
55
56                  node = node->children[bit];
57              }
58          }
59
60          return arns;
61      }
62  };
```

6.3 Disjoint Set Union

```
1   class DisjointSets{
2       private:
3           vector<int> parents;
4           vector<int> sizes;
5           int components;
6       public:
7           DisjointSets(int size) : parents(size), sizes(size,1),
                components(size){
8               for(int i=0; i<size; i++){parents[i] = i;}
9           }
10
11          int find(int x) {return parents[x] == x ? x : (parents[x] =
                find(parents[x]));}
12
13          bool unite(int x, int y){
14              int x_root = find(x);
15              int y_root = find(y);
16
17              if(x_root == y_root) {return false;}
18
19              if(sizes[x_root] < sizes[y_root]) {swap(x_root,y_root);}
20              sizes[x_root] += sizes[y_root];
21              parents[y_root] = x_root;
22              components--;
23              return true;
24          }
25
26          vector<int> getAllComponentSizes(){
27              map<int, int> component_sizes;
28              for (int i = 0; i < parents.size(); ++i){
29                  int root = find(i);
30                  if (component_sizes.find(root) == component_sizes.
                        end()){
31                      component_sizes[root] = sizes[root];
32                  }
33              }
34
35              vector<int> result;
36              for (auto& [root, size] : component_sizes) {
37                  result.push_back(size);
38              }
39
40              return result;
41          }
42
43
44          bool connected(int x, int y) { return find(x) == find(y);}
45          int getSize(int x) {return sizes[find(x)];}
46          int getComponents() const {return components;}
47  };
```

6.4 Fenwick Tree

```
1   // Fenwick Tree (Binary Indexed Tree)
2   class FenwickTree {
3       vector<long long> tree;
4       int n;
5
6   public:
7       FenwickTree(int n) : n(n), tree(n + 1, 0) {}
8
9       void update(int idx, int delta) {
10          for (++idx; idx <= n; idx += idx & -idx) {
11              tree[idx] += delta;
12          }
```

```
    }

    long long query(int idx) {
        long long sum = 0;
        for (++idx; idx > 0; idx -= idx & -idx) {
            sum += tree[idx];
        }
        return sum;
    }

    long long query(int l, int r) {
        return query(r) - (l > 0 ? query(l - 1) : 0);
    }
};
```

6.5 GP Hash Table

```
using namespace __gnu_pbds;

#define ll long long
#define pll pair<ll,ll>
#define fi first
#define sc second
#define u_map gp_hash_table

struct custom_hash {
    static uint64_t splitmix64(uint64_t x) {
        // http://xorshift.di.unimi.it/splitmix64.c
        x += 0x9e3779b97f4a7c15;
        x = (x ^ (x >> 30)) * 0xbf58476d1ce4e5b9;
        x = (x ^ (x >> 27)) * 0x94d049bb133111eb;
        return x ^ (x >> 31);
    }

    size_t operator()(uint64_t x) const {
        static const uint64_t FIXED_RANDOM = chrono::steady_clock::
            now().time_since_epoch().count();
        return splitmix64(x + FIXED_RANDOM);
    }
};

inline void solve(){
    int n; ll t;
    cin>>n>>t;
    vector<ll> arr(n);
    for(ll &x: arr) cin>>x;
    u_map<ll,pll, custom_hash> h;

    for(int i=0; i<n; i++){
  for(int j=i+1; j<n; j++){
      ll key = t - arr[i] - arr[j];
      if(h.find(key) != h.end()){
    cout<<i+1<<" "<<j+1<<" "<<h[key].fi+1<<" "<<h[key].sc+1<<endl;
    return;
      }
  }

  for(int j=0; j<i; j++) h[arr[i]+arr[j]] = {j,i};
    }

    cout<<"IMPOSSIBLE"<<endl;
}
```

6.6 Implicit Treap

```
// Implicit Treap (sequence by position)
struct ImplicitTreap {
    static int nextPriority() {
        static mt19937 rng(chrono::steady_clock::now().
            time_since_epoch().count());
        static uniform_int_distribution<int> dist(INT_MIN, INT_MAX);
        return dist(rng);
    }

    struct Node {
        int val;
        int prior;
        int sz;
        long long sum;
        Node *l, *r;

        explicit Node(int v)
            : val(v), prior(nextPriority()), sz(1), sum(v), l(
                nullptr), r(nullptr) {}
    };

    using pNode = Node*;
    pNode root = nullptr;

    int sz(pNode t) { return t ? t->sz : 0; }
    long long sum(pNode t) { return t ? t->sum : 0; }

    void pull(pNode t) {
        if (!t) return;
        t->sz = 1 + sz(t->l) + sz(t->r);
        t->sum = t->val + sum(t->l) + sum(t->r);
    }

    void split(pNode t, int leftSize, pNode &l, pNode &r) {
        if (!t) {
```

```
            l = r = nullptr;
            return;
        }
        int curLeft = sz(t->l);
        if (leftSize <= curLeft) {
            split(t->l, leftSize, l, t->l);
            r = t;
        } else {
            split(t->r, leftSize - curLeft - 1, t->r, r);
            l = t;
        }
        pull(t);
    }

    pNode merge(pNode l, pNode r) {
        if (!l || !r) return l ? l : r;
        if (l->prior > r->prior) {
            l->r = merge(l->r, r);
            pull(l);
            return l;
        }
        r->l = merge(l, r->l);
        pull(r);
        return r;
    }

    void insert(int pos, int val) { // insert before pos
        pNode L, R;
        split(root, pos, L, R);
        root = merge(merge(L, new Node(val)), R);
    }

    void erase(int pos) {
        pNode A, B, C;
        split(root, pos, A, B);
        split(B, 1, B, C); // B is erased node
        root = merge(A, C);
    }

    long long rangeSum(int l, int r) {
        pNode A, B, C;
        split(root, r + 1, B, C);
        split(B, l, A, B);
        long long ans = sum(B);
        root = merge(merge(A, B), C);
        return ans;
    }
};

// Example of use:
// ImplicitTreap it;
// it.insert(0, 5); it.insert(1, 7); it.insert(1, 3); // [5,3,7]
// long long s = it.rangeSum(0, 2); // 15
```

6.7 Iterative Segment Tree

```
class SegmentTree{
private:
    vector<int> segtree;
    int n;
    int d;

public:

    void build(vector<int> &a){
        n = a.size();
        d=1;
        while(d<n) d<<=1;
        segtree.assign(2*d, 0);
        for(int i=0; i<n; i++)
            segtree[d+i] = a[i];

        for(int i=d-1; i>=1; i--)
            segtree[i] = segtree[2*i] + segtree[2*i+1];
    }

    int find_first(int x){
        if(segtree[1]<x) return -1;
        int i=1;

        while(i<d){
            if(segtree[2*i] >= x) i = 2*i;
            else i = 2*i+1;
        }


        return i-d;
    }

    int find_kth(int x){
        //if(segtree[1]<x) return -1;
        int i=1;

        while(i<d){
            if(segtree[2*i] >= x) i = 2*i;
            else{
                x -= segtree[2*i];
                i = 2*i+1;
            }
        }
        return i-d;
    }
```

```

    void update(int pos){
        pos+=d;
        segtree[pos] = 0;

        while(pos>1){
            pos>>=1;
            segtree[pos] = segtree[2*pos] + segtree[2*pos+1];
        }
    }


    int query(int l, int r){
        l+=d;
        r+=d;

        int mx = INT_MIN;
        while(l<=r){
            if(l%2 == 1) mx = max(mx, segtree[l++]);
            if(r%2 == 0) mx = max(mx, segtree[r--]);
            l>>=1;
            r>>=1;
        }
        return mx;
    }

};
```

6.8 Lazy Segment Tree (Max)

```
enum QueryType{ MAX, NONE };

struct Query{
    QueryType type = NONE;
    ll val = 0;
};

template <typename T> class LazySegTree {
    private:
        const int n;
        vector<T> tree;
        vector<Query> lazy;

        void build(int v, int l, int r, const vector<T> &a){
            if(l==r) tree[v] = a[l];
            else{
                int m = (l+r)/2;
                build(2*v, l, m, a);
                build(2*v+1, m+1, r, a);
                tree[v] = max(tree[2*v], tree[2*v+1]);
            }
        }

        void apply(int v, int len, const Query &x){
            tree[v] += x.val;
            lazy[v].val += x.val;
            lazy[v].type = MAX;
        }

        void push(int v, int l, int r){
            int m = (l+r)/2;
            apply(2*v, m-l+1, lazy[v]);
            apply(2*v+1, r-m, lazy[v]);
            lazy[v] = Query();
        }

        void update(int v, int l, int r, int ql, int qr, const Query
             &x){
            if(qr < l or ql > r) return;

            if(ql<=l and r <= qr) apply(v,r-l+1,x);
            else{
                push(v,l,r);
                int m = (l+r)/2;
                update(2*v,l,m,ql,qr,x);
                update(2*v+1,m+1,r,ql,qr,x);
                tree[v] = max(tree[2*v], tree[2*v+1]);
            }
        }

        T query(int v, int l, int r, int ql, int qr){
            if(qr < l || ql > r) return LLONG_MIN;
            if(l>=ql and r<=qr) return tree[v];
            push(v,l,r);
            int m = (l+r)/2;

            return max(query(2*v, l, m, ql, qr), query(2*v+1,m+1,r,
                ql,qr));
        }

    public:
        LazySegTree(const vector<T> &a) : n(a.size()-1), tree(8 * (a
            .size()-1)), lazy(8 * (a.size()-1)){
            build(1, 0, n, a);
        }

        void update(int ql, int qr, const Query &x){
            update(1,0, n,ql,qr,x);
        }

        T query(int ql, int qr){ return query(1,0,n,ql,qr);};

```

```
};
```

6.9 Lazy Segment Tree (Sum)

```
enum QueryType{ ADD, SET, NONE };

struct Query{
    QueryType type = NONE;
    ll val = 0;
};

template <typename T> class LazySegTree {
    private:
        const int n;
        vector<T> tree;
        vector<Query> lazy;

        void build(int v, int l, int r, const vector<T> &a){
            if(l==r) tree[v] = a[l];
            else{
                int m = (l+r)/2;
                build(2*v, l, m, a);
                build(2*v+1, m+1, r, a);
                tree[v] = tree[2*v] + tree[2*v+1];
            }
        }

        void apply(int v, int len, const Query &x){
            if(x.type == ADD){
                if(lazy[v].type != SET)
                    lazy[v] = Query{ADD, lazy[v].val + x.val};
                else
                    lazy[v] = Query{SET, lazy[v].val + x.val};

                tree[v] += x.val*len;
            }else if(x.type == SET){
                tree[v] = x.val*len;
                lazy[v] = x;
            }
        }

        void push(int v, int l, int r){
            int m = (l+r)/2;
            apply(2*v, m-l+1, lazy[v]);
            apply(2*v+1, r-m, lazy[v]);
            lazy[v] = Query();
        }

        void update(int v, int l, int r, int ql, int qr, const Query
             &x){
            if(qr < l or ql > r) return;

            if(ql<=l and r <= qr) apply(v,r-l+1,x);
            else{
                push(v,l,r);
                int m = (l+r)/2;
                update(2*v,l,m,ql,qr,x);
                update(2*v+1,m+1,r,ql,qr,x);
                tree[v] = tree[2*v] + tree[2*v+1];
            }
        }

        T query(int v, int l, int r, int ql, int qr){
            if(qr < l || ql > r) return 0;
            if(l>=ql and r<=qr) return tree[v];
            push(v,l,r);
            int m = (l+r)/2;

            return query(2*v, l, m, ql, qr) + query(2*v+1,m+1,r,ql,
                qr);
        }

    public:
        LazySegTree(const vector<T> &a) : n(a.size()), tree(4 * n),
            lazy(4 * n){
            build(1, 0, n - 1, a);
        }

        void update(int ql, int qr, const Query &x){
            update(1,0, n-1,ql,qr,x);
        }

        T query(int ql, int qr){ return query(1,0,n-1,ql,qr);};

};
```

6.10 Lazy Treap

```
// Lazy Implicit Treap (range add, range sum, range reverse)
struct LazyTreap {
    static int nextPriority() {
        static mt19937 rng(chrono::steady_clock::now().
            time_since_epoch().count());
        static uniform_int_distribution<int> dist(INT_MIN, INT_MAX);
        return dist(rng);
    }

    struct Node {
        int val;
        int prior;
```

```
        int sz;
        long long sum;
        long long lazyAdd;
        bool rev;
        Node *l, *r;

        explicit Node(int v)
            : val(v), prior(nextPriority()), sz(1), sum(v), lazyAdd
                (0), rev(false), l(nullptr), r(nullptr) {}
    };

    using pNode = Node*;
    pNode root = nullptr;

    int sz(pNode t) { return t ? t->sz : 0; }
    long long sum(pNode t) { return t ? t->sum : 0; }

    void applyAdd(pNode t, long long add) {
        if (!t) return;
        t->val += add;
        t->sum += add * sz(t);
        t->lazyAdd += add;
    }

    void applyRev(pNode t) {
        if (!t) return;
        t->rev ^= 1;
        swap(t->l, t->r);
    }

    void push(pNode t) {
        if (!t) return;
        if (t->rev) {
            applyRev(t->l);
            applyRev(t->r);
            t->rev = false;
        }
        if (t->lazyAdd != 0) {
            applyAdd(t->l, t->lazyAdd);
            applyAdd(t->r, t->lazyAdd);
            t->lazyAdd = 0;
        }
    }

    void pull(pNode t) {
        if (!t) return;
        t->sz = 1 + sz(t->l) + sz(t->r);
        t->sum = t->val + sum(t->l) + sum(t->r);
    }

    void split(pNode t, int leftSize, pNode &l, pNode &r) {
        if (!t) {
            l = r = nullptr;
            return;
        }
        push(t);
        int curLeft = sz(t->l);
        if (leftSize <= curLeft) {
            split(t->l, leftSize, l, t->l);
            r = t;
        } else {
            split(t->r, leftSize - curLeft - 1, t->r, r);
            l = t;
        }
        pull(t);
    }

    pNode merge(pNode l, pNode r) {
        if (!l || !r) return l ? l : r;
        if (l->prior > r->prior) {
            push(l);
            l->r = merge(l->r, r);
            pull(l);
            return l;
        }
        push(r);
        r->l = merge(l, r->l);
        pull(r);
        return r;
    }

    void insert(int pos, int val) {
        pNode L, R;
        split(root, pos, L, R);
        root = merge(merge(L, new Node(val)), R);
    }

    void rangeAdd(int l, int r, long long add) {
        pNode A, B, C;
        split(root, r + 1, B, C);
        split(B, l, A, B);
        applyAdd(B, add);
        root = merge(merge(A, B), C);
    }

    void rangeReverse(int l, int r) {
        pNode A, B, C;
        split(root, r + 1, B, C);
        split(B, l, A, B);
        applyRev(B);
        root = merge(merge(A, B), C);
    }

    long long rangeSum(int l, int r) {
        pNode A, B, C;
        split(root, r + 1, B, C);
```

```
        split(B, l, A, B);
        long long ans = sum(B);
        root = merge(merge(A, B), C);
        return ans;
    }
};

// Example of use:
// LazyTreap lt;
// lt.insert(0, 1); lt.insert(1, 2); lt.insert(2, 3); // [1,2,3]
// lt.rangeAdd(0, 1, 5); // [6,7,3]
// lt.rangeReverse(0, 2); // [3,7,6]
// long long s = lt.rangeSum(0, 2); // 16
```

6.11 Merge Sort Tree

```
class MergeSortTree{
private:
    vector<vector<int>> segtree;
    int n;
    int d;

public:
    void build(vector<int> &a){
        n = a.size();
        d=1;
        while(d<n) d<<=1;
        segtree.assign(2*d, {0});
        for(int i=0; i<n; i++)
            segtree[d+i] = {a[i]};

        for(int i=d-1; i>=1; i--){
            segtree[i].resize(segtree[2*i].size() + segtree[2*i].
                size());
            merge(segtree[2*i].begin(),segtree[2*i].end(),
                  segtree[2*i+1].begin(), segtree[2*i+1].end(),
                  segtree[i].begin());
        }

    }

    int query(int l, int r, int x){
        l+=d;
        r+=d;

        int arns = 0;
        while(l<=r){
            if(l%2 == 1){
                arns += lower_bound(segtree[l].begin(), segtree[l].
                    end(), x) - segtree[l].begin();
                l++;
            }
            if(r%2 == 0){
                arns += lower_bound(segtree[r].begin(), segtree[r].
                    end(), x) - segtree[r].begin();
                r--;
            }
            l>>=1;
            r>>=1;
        }
        return arns;
    }

};
```

6.12 Ordered Set

```
#include <ext/pb_ds/assoc_container.hpp>
#include <ext/pb_ds/tree_policy.hpp>

#define pii pair<int,int>

using namespace std;
using namespace __gnu_pbds;

#define int long long

const int MAX = 2e5+1;
const int INF = LLONG_MAX;

template<typename T> using Tree = tree<T, null_type, less<T>,
    rb_tree_tag, tree_order_statistics_node_update>;

void solve(){
    int n,q; cin>>n>>q;
    vector<int> v(n);
    Tree<pii> oset;

    for(int &x : v) cin>>x;

    for(int i=0; i<n; i++){
        oset.insert({v[i],i});
    }

    for(int i=0; i<q;i++){
        char ty; cin>>ty;
        if(ty == '!'){
            int k, s; cin>>k>>s;
            k--;
            oset.erase({v[k],k});
```

```
            v[k] = s;
            oset.insert({v[k],k});
        }else{
            int a, b; cin>>a>>b;
            cout<<oset.order_of_key({b,INF}) - oset.order_of_key({a
                -1,INF})<<endl;
        }
    }

}
```

6.13 Recursive Segment Tree

```
ll  t[4*MAX];

// Shout-out to CP algo for the SegTree implementation: https://cp-
    algorithms.com/data_structures/segment_tree.html#memory-
    efficient-implementation

void buildSegTree(vector<ll> &a, int v, int tl, int tr) {
    if (tl == tr) {
        t[v] = a[tl];
    } else {
        int tm = (tl + tr) / 2;
        buildSegTree(a, v*2, tl, tm);
        buildSegTree(a, v*2+1, tm+1, tr);
        t[v] = t[v*2] + t[v*2+1];
    }
}


ll sum(int v, int tl, int tr, int l, int r) {
    if (l > r)
        return 0;
    if (l == tl && r == tr) {
        return t[v];
    }
    int tm = (tl + tr) / 2;
    return sum(v*2, tl, tm, l, min(r, tm))
           + sum(v*2+1, tm+1, tr, max(l, tm+1), r);
}

void update(int v, int tl, int tr, int pos, ll new_val) {
    if (tl == tr) {
        t[v] = new_val;
    } else {
        int tm = (tl + tr) / 2;
        if (pos <= tm)
            update(v*2, tl, tm, pos, new_val);
        else
            update(v*2+1, tm+1, tr, pos, new_val);
        t[v] = t[v*2] + t[v*2+1];
    }
}
```

6.14 Treap

```
// Treap (BST by key + heap by priority)
struct Treap {
    static int nextPriority() {
        static mt19937 rng(chrono::steady_clock::now().
            time_since_epoch().count());
        static uniform_int_distribution<int> dist(INT_MIN, INT_MAX);
        return dist(rng);
    }

    struct Node {
        int key;
        int prior;
        int cnt;
        Node *l, *r;

        explicit Node(int k) : key(k), prior(nextPriority()), cnt(1)
            , l(nullptr), r(nullptr) {}
    };

    using pNode = Node*;
    pNode root = nullptr;

    int cnt(pNode t) { return t ? t->cnt : 0; }

    void pull(pNode t) {
        if (!t) return;
        t->cnt = 1 + cnt(t->l) + cnt(t->r);
    }

    void split(pNode t, int key, pNode &l, pNode &r) {
        if (!t) {
            l = r = nullptr;
            return;
        }
        if (t->key <= key) {
            split(t->r, key, t->r, r);
            l = t;
        } else {
            split(t->l, key, l, t->l);
            r = t;
        }
        pull(t);
    }
```

```

    pNode merge(pNode l, pNode r) {
        if (!l || !r) return l ? l : r;
        if (l->prior > r->prior) {
            l->r = merge(l->r, r);
            pull(l);
            return l;
        }
        r->l = merge(l, r->l);
        pull(r);
        return r;
    }

    bool contains(pNode t, int key) {
        while (t) {
            if (t->key == key) return true;
            if (key < t->key) t = t->l;
            else t = t->r;
        }
        return false;
    }

    void insert(int key) {
        if (contains(root, key)) return; // set behavior (no
            duplicates)
        pNode L, R;
        split(root, key, L, R);
        root = merge(merge(L, new Node(key)), R);
    }

    void erase(int key) {
        pNode A, B, C;
        split(root, key, A, C);
        split(A, key - 1, A, B); // B contains only key (if exists)
        // delete subtree B if needed for memory cleanup
        root = merge(A, C);
    }

    int order_of_key(int key) { // count of elements < key
        pNode t = root;
        int ans = 0;
        while (t) {
            if (key <= t->key) {
                t = t->l;
            } else {
                ans += 1 + cnt(t->l);
                t = t->r;
            }
        }
        return ans;
    }

    int find_by_order(int k) { // 0-indexed kth smallest
        pNode t = root;
        while (t) {
            int left = cnt(t->l);
            if (k < left) t = t->l;
            else if (k == left) return t->key;
            else {
                k -= left + 1;
                t = t->r;
            }
        }
        return -1; // invalid k
    }
};

// Example of use:
// Treap tr;
// tr.insert(10); tr.insert(4); tr.insert(20);
// int x = tr.find_by_order(1); // 10
// int y = tr.order_of_key(15); // 2
```

6.15 Cycle Detection Online

```
class CycleDetectionDSU {
    vector<int> parent;
    vector<int> size;

public:
    CycleDetectionDSU(int n) : parent(n), size(n, 1) {
        iota(parent.begin(), parent.end(), 0);
    }

    int find(int x) {
        return parent[x] == x ? x : parent[x] = find(parent[x]);
    }

    // Returns true if adding edge u-v creates a cycle
    bool add_edge(int u, int v) {
        int u_root = find(u);
        int v_root = find(v);
        if (u_root == v_root) return true;

        if (size[u_root] < size[v_root]) swap(u_root, v_root);
        parent[v_root] = u_root;
        size[u_root] += size[v_root];
        return false;
    }
};
```

6.16 DSU with Rollbacks

```cpp
class DSU {
  private:
  vector<int> p, sz;
  // stores previous unites
  vector<pair<int &, int>> history;

  public:
  DSU(int n) : p(n), sz(n, 1) { iota(p.begin(), p.end(), 0); }

  int get(int x) { return x == p[x] ? x : get(p[x]); }

  void unite(int a, int b) {
    a = get(a);
    b = get(b);
    if (a == b) { return; }
    if (sz[a] < sz[b]) { swap(a, b); }

    // save this unite operation
    history.push_back({sz[a], sz[a]});
    history.push_back({p[b], p[b]});

    p[b] = a;
    sz[a] += sz[b];
  }

  int snapshot() { return history.size(); }

  void rollback(int until) {
    while (snapshot() > until) {
      history.back().first = history.back().second;
      history.pop_back();
    }
  }
};
```

6.17 Fenwick Lazy

```cpp
// Range Update - Range Query (1-based indexing)
struct FenwickRURQ {
    int n;
    std::vector<int> B1, B2;

    FenwickRURQ(int size) : n(size + 1), B1(size + 2), B2(size + 2)
        {}

    // Add val to range [l, r] (1-based)
    void range_add(int l, int r, int val) {
        add(B1, l, val);
        add(B1, r + 1, -val);
        add(B2, l, val * (l - 1));
        add(B2, r + 1, -val * r);
    }

    // Get sum of range [l, r] (1-based)
    int range_sum(int l, int r) {
        return prefix_sum(r) - prefix_sum(l - 1);
    }

private:
    void add(std::vector<int>& b, int idx, int val) {
        for(; idx < n; idx += idx & -idx)
            b[idx] += val;
    }

    int sum(const std::vector<int>& b, int idx) {
        int total = 0;
        for(; idx > 0; idx -= idx & -idx)
            total += b[idx];
        return total;
    }

    int prefix_sum(int idx) {
        return sum(B1, idx) * idx - sum(B2, idx);
    }
};
```

6.18 Implicit Treap 2

```cpp
mt19937 rng(chrono::steady_clock::now().time_since_epoch().count());

#define ll long long

struct TreapNode{
    ll key, pr, sz;
    TreapNode *l, *r;
};

typedef TreapNode* Treap;

int getSize(Treap &t){
    return t ? t->sz : 0;
}

void updateSize(Treap &t){
    if (t) t->sz = 1 + getSize(t->l) + getSize(t->r);
}

void split(Treap& t, ll k, Treap &l, Treap  &r){
```

```
    if(not t) l = r = nullptr;

    else if(k < t->key){
        split(t->l,k,l, t->l);
        r = t;
        updateSize(r);
    }else{
        split(t->r,k,t->r,r);
        l = t;
        updateSize(l);
    }
}

void insert(Treap& t, Treap a){
    if(not t) t=a;
    else if(a->pr > t->pr){
        split(t, a->key, a->l, a->r);
        t = a;
    }else{
        if(a->key < t-> key) insert(t->l,a);
        else insert(t->r,a);
    }
    updateSize(t);
}

void merge(Treap &t, Treap l, Treap r){
    if(not l) t = r;
    else if(not r) t = l;

    else if(l->pr > r->pr){
        merge(l->r, l->r,r);
        t=l;
        updateSize(t);
    }else{
        merge(r->l,l,r->l);
        t=r;
        updateSize(t);
    }
}

void erase(Treap &t, ll k){
    if(not t) return;
    if(t->key == k) merge(t,t->l, t->r);

    else{
        if(k<t->key) erase(t->l,k);
        else erase(t->r, k);
    }
    updateSize(t);
}

bool find(Treap& t, ll k){
    if (not t) return false;
    if(t->key == k) return true;
    if(k<t->key) return find(t->l,k);
    return find(t->r,k);
}

void insertValue(Treap &t, ll k){
    if(not find(t,k)){
        Treap new_node = new TreapNode {k,rng(), 0,nullptr, nullptr
            };
        insert(t, new_node);
    }
}

ll getKth(Treap &t, int k){
    if(!t || k<=0 || k>getSize(t)) return 0;
    int leftSize = getSize(t->l);
    if(k == leftSize+1) return t->key;
    if(k <= leftSize) return getKth(t->l,k);
    return getKth(t->r, k-leftSize-1);
}
```

6.19 Online Bipartiteness

```
void make_set(int v) {
    parent[v] = make_pair(v, 0);
    rank[v] = 0;
    bipartite[v] = true;
}

pair<int, int> find_set(int v) {
    if (v != parent[v].first) {
        int parity = parent[v].second;
        parent[v] = find_set(parent[v].first);
        parent[v].second ^= parity;
    }
    return parent[v];
}

void add_edge(int a, int b) {
    pair<int, int> pa = find_set(a);
    a = pa.first;
    int x = pa.second;

    pair<int, int> pb = find_set(b);
    b = pb.first;
    int y = pb.second;

    if (a == b) {
        if (x == y)
            bipartite[a] = false;
```

```
    } else {
        if (rank[a] < rank[b])
            swap (a, b);
        parent[b] = make_pair(a, x^y^1);
        bipartite[a] &= bipartite[b];
        if (rank[a] == rank[b])
            ++rank[a];
    }
}

bool is_bipartite(int v) {
    return bipartite[find_set(v).first];
}
```

6.20 Path Copy Persistent Array

```
struct Node {
  int val;
  Node *l, *r;

  Node(ll x) : val(x), l(nullptr), r(nullptr) {}
  Node(Node *ll, Node *rr) : val(0), l(ll), r(rr) {}
};

int n, a[100001];      // The initial array and its size
Node *roots[100001];   // The persistent array's roots

Node *build(int l = 0, int r = n - 1) {
  if (l == r) return new Node(a[l]);
  int mid = (l + r) / 2;
  return new Node(build(l, mid), build(mid + 1, r));
}

Node *update(Node *node, int val, int pos, int l = 0, int r = n - 1)
    {
  if (l == r) return new Node(val);
  int mid = (l + r) / 2;
  if (pos > mid) return new Node(node->l, update(node->r, val, pos,
      mid + 1, r));
  else return new Node(update(node->l, val, pos, l, mid), node->r);
}

int query(Node *node, int pos, int l = 0, int r = n - 1) {
  if (l == r) return node->val;
  int mid = (l + r) / 2;
  if (pos > mid) return query(node->r, pos, mid + 1, r);
  return query(node->l, pos, l, mid);
}

int get_item(int index, int time) {
  // Gets the array item at a given index and time
  return query(roots[time], index);
}

void update_item(int index, int value, int prev_time, int curr_time)
    {
  // Updates the array item at a given index and time
  roots[curr_time] = update(roots[prev_time], index, value);
}

void init_arr(int nn, int *init) {
  // Initializes the persistent array, given an input array
  n = nn;
  for (int i = 0; i < n; i++) a[i] = init[i];
  roots[0] = build();
}
```

6.21 Persistent Array

```
vector<pair<int, int>> arr[100001];  // The persistent array

int get_item(int index, int time) {
  // Gets the array item at a given index and time
  auto ub =
      upper_bound(arr[index].begin(), arr[index].end(), make_pair(
          time, INT_MAX));
  return prev(ub)->second;
}

void update_item(int index, int value, int time) {
  // Updates the array item at a given index and time
  // Note that this only works if the time is later than all
      previous
  // update times
  assert(arr[index].back().first < time);
  arr[index].push_back({time, value});
}

void init_arr(int n, int *init) {
  // Initializes the persistent array, given an input array
  for (int i = 0; i < n; i++) arr[i].push_back({0, init[i]});
}
```

6.22 Persistent Segment Tree

```
using ll = long long;

```

```
class PersistentSegtree {
  private:
  struct Node {
    ll sum = 0;
    int l = 0, r = 0;
  };

  const int n;
  vector<Node> tree;
  int timer = 1;

  Node join(int l, int r) { return Node{tree[l].sum + tree[r].sum, l, r}; }

  int build(int tl, int tr, const vector<int> &arr) {
    if (tl == tr) {
      tree[timer] = {arr[tl], 0, 0};
      return timer++;
    }

    int mid = (tl + tr) / 2;
    tree[timer] = join(build(tl, mid, arr), build(mid + 1, tr, arr));

    return timer++;
  }

  int set(int v, int pos, int val, int tl, int tr) {
    if (tl == tr) {
      tree[timer] = {val, 0, 0};
      return timer++;
    }

    int mid = (tl + tr) / 2;
    if (pos <= mid) {
      tree[timer] = join(set(tree[v].l, pos, val, tl, mid), tree[v].r);
    } else {
      tree[timer] = join(tree[v].l, set(tree[v].r, pos, val, mid + 1, tr));
    }

    return timer++;
  }

  ll range_sum(int v, int ql, int qr, int tl, int tr) {
    if (qr < tl || tr < ql) { return 0ll; }
    if (ql <= tl && tr <= qr) { return tree[v].sum; }

    int mid = (tl + tr) / 2;
    return range_sum(tree[v].l, ql, qr, tl, mid) +
           range_sum(tree[v].r, ql, qr, mid + 1, tr);
  }

  public:
  PersistentSegtree(int n, int MX_NODES) : n(n), tree(MX_NODES) {}

  int build(const vector<int> &arr) { return build(0, n - 1, arr); }

  int set(int root, int pos, int val) { return set(root, pos, val, 0, n - 1); }

  ll range_sum(int root, int l, int r) { return range_sum(root, l, r, 0, n - 1); }

  int add_copy(int root) {
    tree[timer] = tree[root];
    return timer++;
  }
};
```

6.23 Prefix Suffix Segment Tree

```
typedef long long ll;
typedef vector<int> vec;
typedef vector<pair<int,int>> vpii;
const ll mod=1e9+7;
const int MAX=1e5+3;
const int limit=2e5+3;
const int TAM=2e5+1;
ll t[4*TAM];
ll prefix[4*TAM],suffix[4*TAM],biggest[4*TAM];
//ascii https://elcodigoascii.com.ar/
ll cero=0;
void build(int root,int l,int r,vector<ll> &arr)
{
    if(l==r)
    {
        t[root]=arr[l];
        suffix[root]=max(t[root],cero);
        prefix[root]=max(t[root],cero);
        biggest[root]=max(t[root],cero);
        return;
    }
    int mid=(l+r)/2;
    build(2*root,l,mid,arr);
    build(2*root+1,mid+1,r,arr);
    t[root]=t[2*root]+t[2*root+1];
    biggest[root]=max(biggest[2*root],
    max(biggest[2*root+1],suffix[2*root]+prefix[2*root+1]));
    prefix[root]=max(prefix[2*root],t[2*root]+prefix[2*root+1]);
    suffix[root]=max(suffix[2*root+1],t[2*root+1]+suffix[2*root]);

```

```
}

void update(int root,int l,int r,int pos,ll val)
{
    if(l==r)
    {
        t[root]=val;
        suffix[root]=max(cero,t[root]);
        prefix[root]=max(cero,t[root]);
        biggest[root]=max(t[root],cero);
        return;
    }
    int mid=(l+r)/2;
    if(pos>mid)
    {
        update(2*root+1,mid+1,r,pos,val);
    }
    else{
        update(2*root,l,mid,pos,val);
    }
    t[root]=t[2*root]+t[2*root+1];
    biggest[root]=max(biggest[2*root],
    max(biggest[2*root+1],suffix[2*root]+prefix[2*root+1]));
    prefix[root]=max(prefix[2*root],t[2*root]+prefix[2*root+1]);
    suffix[root]=max(suffix[2*root+1],t[2*root+1]+suffix[2*root]);
}

ll consult(int root,int l,int r, int a,int b)
{
    if(a>b) return 0;
    if(l==a && r==b) return t[root];
    int mid=(l+r)/2;
    return consult(2*root,l,mid,a,min(b,mid))+
    consult(2*root+1,mid+1,r,max(mid+1,a),b);
}
```

6.24 Wicho Lazy Segment Tree

```
typedef long long ll;
typedef vector<int> vec;
typedef vector<pair<int,int>> vpii;
const ll mod=1e9+7;
const int MAX=1e5+3;
const int limit=2e5+3;
const int TAM=2e5+1;
ll t[4*TAM];
ll op[4*TAM];
int type[4*TAM];
//ascii https://elcodigoascii.com.ar/

void propagate(int root,int l,int r)
{
    if(type[root]==1)
    {
        t[root]+=op[root]*(r+1-l);
        if(l!=r){
            op[2*root]+=op[root];
            op[2*root+1]+=op[root];
            type[2*root+1]=max(1,type[2*root+1]);
            type[2*root]=max(1,type[2*root]);
        }
    }
    else
    {
        if(type[root]==2){
            t[root]=op[root]*(r+1-l);
            if(l!=r){
                op[2*root]=op[root];
                op[2*root+1]=op[root];
                type[2*root+1]=2;
                type[2*root]=2;
            }
        }
    }
    op[root]=0;
    type[root]=0;
}

void build(int root,int l,int r,vector<ll> &arr)
{
    if(l==r)
    {
        t[root]=arr[l];
        op[root]=0;
        type[root]=0;
        return;
    }
    int mid=(l+r)/2;
    build(2*root,l,mid,arr);
    build(2*root+1,mid+1,r,arr);
    t[root]=t[2*root]+t[2*root+1];
    op[root]=0;
    type[root]=0;
}

void sum(int root,int l,int r,int a,int b,ll val)
{
    propagate(root,l,r);
    if(a>b) return;
    if(l==a && r==b)
    {
        op[root]=val;
        type[root]=1;
```

```
        propagate(root,l,r);
        return;
    }
    int mid=(l+r)/2;
    sum(2*root,l,mid,a,min(b,mid),val);
    sum(2*root+1,mid+1,r,max(mid+1,a),b,val);
    t[root]=t[2*root]+t[2*root+1];
}

void setR(int root,int l,int r,int a,int b,ll val)
{
    propagate(root,l,r);
    if(a>b) return;
    if(l==a && r==b)
    {
        op[root]=val;
        type[root]=2;
        propagate(root,l,r);
        return;
    }
    int mid=(l+r)/2;
    setR(2*root,l,mid,a,min(b,mid),val);
    setR(2*root+1,mid+1,r,max(mid+1,a),b,val);
    t[root]=t[2*root]+t[2*root+1];
}

ll consult(int root,int l,int r, int a,int b)
{
    propagate(root,l,r);
    if(a>b) return 0;
    if(l==a && r==b){
        return t[root];
    }
    int mid=(l+r)/2;
    return consult(2*root,l,mid,a,min(b,mid))+
    consult(2*root+1,mid+1,r,max(mid+1,a),b);
}
```

6.25 Li Chao Tree

```
typedef long long ll;
typedef complex<ll> pt;
typedef vector<int> vec;

const ll mod=1e9+7;
const ll MAX=1e5+3;
const ll INF=1e18;

pt lines[4*MAX];

//Li Chao Tree

ll dot(pt a,pt b){
    return a.X*b.X+a.Y*b.Y;
}

ll f(pt line,ll val){
    return dot(line,pt{val,1});
}

void addLine(pt cur,int root=1,int l=0,int r=MAX)
{

    int m=(l+r)/2;
    bool left_border=f(cur,l)>f(lines[root],l);
    bool mid=f(cur,m)>f(lines[root],m);

    if(mid){
        swap(lines[root],cur);
    }
    if(r-l==1) return;
    else if(left_border!=mid){
        addLine(cur,2*root,l,m);
    }
    else{
        addLine(cur,2*root+1,m,r);
    }
}

ll maximum(int x,int root=1,int l=0,int r=MAX)
{
    int mid=(l+r)/2;

    if(r-l==1){
        return f(lines[root],x);
    }
    if(x<mid){
        return max(f(lines[root],x),maximum(x,2*root,l,mid));
    }
    return max(f(lines[root],x),maximum(x,2*root+1,mid,r));
}

void addSegment(pt cur,int a,int b,int root=1,int l=0,int r=MAX)
{
    if(a>b) return;

    if(a==l && b==r){
        addLine(cur,root,l,r);
        return;
    }
    if(r-l==1) return;
    int mid=(l+r)/2;
    addSegment(cur,a,min(mid,b),2*root,l,mid);
```

```
    addSegment(cur,max(mid,a),b,2*root+1,mid,r);
}
```

6.26 Mo's Algorithm

```
// Mo's Algorithm for offline range query processing
// Time: O((N + Q) * sqrt(N))
// Based on KACTL: https://github.com/kth-competitive-programming/kactl

#pragma once

#include <bits/stdc++.h>
using namespace std;

// Mo's Algorithm for offline range query processing.
// T: answer/value type
// Add/Remove: functors or lambdas with operator(T) or add(T)/remove(T) methods
template <typename T>
struct MoQueries {
    struct Query {
        int l, r, idx;
    };

    vector<Query> queries;
    int n;

    explicit MoQueries(int n = 0) : n(n) {}

    void add_query(int l, int r) {
        queries.push_back({l, r, (int)queries.size()});
    }

    // Solve using provided add/remove/get operations
    // Operations can be lambdas or functor objects with operator()
    // add(x): add element x to current range
    // remove(x): remove element x from current range
    // get(): return current answer of type T
    template <typename Array, typename Add, typename Remove, typename Get>
    vector<T> solve(const Array& a, Add add, Remove remove, Get get) {
        int q = queries.size();
        int blk = static_cast<int>(sqrt(n));

        sort(queries.begin(), queries.end(), [&](const Query& i, const Query& j) {
            int bi = i.l / blk;
            int bj = j.l / blk;
            if (bi != bj) return bi < bj;
            // Alternate sorting direction per block to reduce pointer movement
            return (bi & 1) ? i.r > j.r : i.r < j.r;
        });

        vector<T> res(q);
        int L = 0, R = 0;

        for (const auto& qu : queries) {
            while (L > qu.l) add(a[--L]);
            while (R < qu.r) add(a[R++]);
            while (L < qu.l) remove(a[L++]);
            while (R > qu.r) remove(a[--R]);
            res[qu.idx] = get();
        }
        return res;
    }
};
```

7 Range Queries

7.1 Sparse Table

```
// Sparse Table for Range Minimum Query
class SparseTable {
    vector<vector<int>> table;
    vector<int> log;
    int n;

public:
    SparseTable(vector<int>& arr) {
        n = arr.size();
        int maxLog = log2(n) + 1;
        table.assign(n, vector<int>(maxLog));
        log.assign(n + 1, 0);

        // Precompute logarithms
        for (int i = 2; i <= n; i++) {
            log[i] = log[i/2] + 1;
        }

        // Build sparse table
        for (int i = 0; i < n; i++) {
            table[i][0] = arr[i];
        }

        for (int j = 1; j < maxLog; j++) {
            for (int i = 0; i + (1 << j) <= n; i++) {
```

```
                table[i][j] = min(table[i][j-1],
                                  table[i + (1 << (j-1))][j-1]);
            }
        }
    }

    int query(int l, int r) {
        int j = log[r - l + 1];
        return min(table[l][j], table[r - (1 << j) + 1][j]);
    }
};
```

8 Mathematics

8.1 Formula Sheet (Contest)

8.1.1 Combinatorics

$$\binom{n}{k} = \frac{n!}{k!(n-k)!}$$

$$P(n,k) = \frac{n!}{(n-k)!} \quad \text{(ordered, no repetition)}$$

$$\text{Combinations with repetition} = \binom{n+k-1}{k}$$

$$\text{Permutations with repetition} = \frac{n!}{a_1!a_2!\cdots a_m!}$$

$$\binom{n}{k} = \binom{n-1}{k-1} + \binom{n-1}{k} \quad \text{(Pascal identity)}$$

Multinomial coefficient (split n items into groups $k_1, \ldots, k_m$):

$$\binom{n}{k_1, k_2, \ldots, k_m} = \frac{n!}{k_1!k_2!\cdots k_m!}, \quad \sum_i k_i = n$$

Catalan numbers (balanced parentheses, rooted full binary trees, non-crossing matchings):

$$C_n = \frac{1}{n+1}\binom{2n}{n} = \frac{(2n)!}{(n+1)!n!}$$

$$C_0 = 1, \quad C_n = \sum_{i=0}^{n-1} C_i C_{n-1-i}$$

- **Valid Parentheses**: Count of balanced parentheses expressions with n pairs.
- **Full Binary Trees**: Structurally unique full binary trees with $n+1$ leaves.
- **Polygon Triangulation**: Ways to triangulate a convex $(n+2)$-gon.
- **Dyck Paths**: Paths from $(0,0)$ to $(2n,0)$ that never dip below the x-axis.
- **Non-Crossing Partitions**: Ways to connect $2n$ points on a circle without crossing chords.
- **Stack Permutations**: Valid stack-sortable permutations of length n.
- **Mountain Ranges**: Sequences of $2n$ up/down steps forming valid mountain ranges.
- **Unique BSTs**: Number of distinct binary search trees with n keys.
- **Diagonal-Avoiding Paths**: Paths in a grid from $(0,0)$ to (n,n) without crossing the diagonal.

8.2 Cayley's Formula

Number of labeled trees of n vertices: n^{n-2}.

Number of rooted forest of n vertices is: $(n+1)^{n-1}$

8.3 Geometric series

Finite:

$$\sum_{k=0}^{n} ar^k = \begin{cases} a\dfrac{1-r^{n+1}}{1-r} & \text{if } r \neq 1, \\ a(n+1) & \text{if } r = 1. \end{cases}$$

Infinite:

$$\sum_{k=0}^{\infty} ar^k = \frac{a}{1-r} \quad \text{(converges iff } |r| < 1)$$

8.4 Divisors

The number of divisors of any number n is:

$$\begin{cases} \approx 100 & n < 5 \times 10^4 \\ \approx 500 & n < 1 \times 10^7 \\ \approx 2000 & n < 1 \times 10^10 \\ \approx 200000 & n < 1 \times 10^19 \end{cases}$$

8.5 Number of primes between 1 and n

$$\frac{n}{ln(n)}$$

8.6 Pythagorean triplets

$$a = k \cdot (m^2 - n^2), \quad b = k \cdot (2mn), \quad c = k \cdot (m^2 + n^2)$$

With $m > n > 0$, $k \geq 1$, $m \perp n$, and either m or n even.

8.7 Derangements

Permutations of a set such that none of the elements appear in their original position.

$$D(n) = (n-1)(D(n-1) + D(n-2)) = nD(n-1) + (-1)^n = \lfloor \frac{n!}{e} + \frac{1}{2} \rfloor$$

Pascal triangle facts:

$$\sum_{k=0}^{n} \binom{n}{k} = 2^n, \qquad \sum_{k=0}^{n} (-1)^k \binom{n}{k} = 0 \quad (n \geq 1)$$

8.7.1 Sums

Gaussian sums (first powers to fourth powers):

$$\sum_{i=1}^{n} i = \frac{n(n+1)}{2}$$
$$\sum_{i=1}^{n} i^2 = \frac{n(n+1)(2n+1)}{6}$$
$$\sum_{i=1}^{n} i^3 = \left(\frac{n(n+1)}{2}\right)^2$$
$$\sum_{i=1}^{n} i^4 = \frac{n(n+1)(2n+1)(3n^2+3n-1)}{30}$$

Useful finite progressions:

$$\sum_{i=0}^{n-1} ar^i = a\frac{1-r^n}{1-r} \ (r \neq 1), \qquad \sum_{i=0}^{n-1} (a+id) = \frac{n}{2}(2a + (n-1)d)$$

Rare progressions:

$$\sum_{i=1}^{n-1} i \cdot (n-i) = \frac{n(n^2-1)}{6}$$

8.7.2 Geometry

For points $A(x_1, y_1), B(x_2, y_2)$:

$$\text{Euclidean}(A,B) = \sqrt{(x_1-x_2)^2 + (y_1-y_2)^2}$$
$$\text{Manhattan}(A,B) = |x_1-x_2| + |y_1-y_2|$$
$$\text{Chebyshev}(A,B) = \max\{|x_1-x_2|, |y_1-y_2|\}$$

For vectors $\vec{a} = (a_x, a_y)$, $\vec{b} = (b_x, b_y)$:

$$\vec{a} \cdot \vec{b} = a_x b_x + a_y b_y$$
$$\vec{a} \times \vec{b} = a_x b_y - a_y b_x$$

Note: In 2D, $\text{sign}(\vec{a} \times \vec{b})$ gives orientation (left/right turn), and $|\vec{a} \times \vec{b}|/2$ is the triangle area.

8.7.3 Probability

$$\mathbb{E}[X] = \sum_x x\, P(X = x)$$
$$\mathbb{E}[aX + b] = a\,\mathbb{E}[X] + b$$
$$\mathbb{E}\left[\sum_{i=1}^{m} X_i\right] = \sum_{i=1}^{m} \mathbb{E}[X_i] \quad \text{(linearity, no independence needed)}$$

Indicator trick (very useful): If I_A is 1 when event A happens and 0 otherwise, then $\mathbb{E}[I_A] = P(A)$. Count expected objects by summing indicators.

8.7.4 Number Theory and Counting Essentials

$$\gcd(a,b) \cdot \operatorname{lcm}(a,b) = ab$$
$$\varphi(n) = n \prod_{p \mid n} \left(1 - \frac{1}{p}\right)$$
$$a^{\varphi(m)} \equiv 1 \pmod{m} \ (\gcd(a,m) = 1)$$

Inclusion-exclusion for sets $A_1, \ldots, A_m$:

$$\left|\bigcup_i A_i\right| = \sum_i |A_i| - \sum_{i<j} |A_i \cap A_j| + \sum_{i<j<k} |A_i \cap A_j \cap A_k| - \cdots$$

Stars and bars (nonnegative solutions of $x_1 + \cdots + x_k = n$):

$$\#\text{solutions} = \binom{n+k-1}{k-1}$$

8.8 Manhattan Distance on a Grid

For the sum of Manhattan distances between all pairs of points in an $n \times m$ grid:

Compress the $n \times m$ grid along the x axis, leaving a row of m points where each column has n points. The horizontal contribution is:

$$D_x = n^2 \frac{m(m^2-1)}{6}$$

Do the same along the y axis:

$$D_y = m^2 \frac{n(n^2-1)}{6}$$

Total pairwise Manhattan distance in the grid:

$$D = n^2 \frac{m(m^2-1)}{6} + m^2 \frac{n(n^2-1)}{6}$$

For the external Manhattan distances between K disjoint rectangular blocks:

NOTE: Each block must have a constant height (same number of rows).

$$D_{\text{external}} = \sum_{i=1}^{K-1} \sum_{j=i+1}^{K} A_i A_j C_{ij}$$

Where A_i is the area of block i, and C_{ij} is the Manhattan distance between the centers of blocks i and j projected onto one axis. Repeat the same formula for the y axis and add the two contributions. To get a block's center, average the x and y coordinates of its corners; it doesn't have to be an integer.

8.9 Binary Exponentiation

```cpp
const int MAX = 2e5+1;
const int MOD = 1e9+7;
int binpow(int a, int b, int m){
    int arns = 1;
    a%=m;
    while(b>0){
        if(b&1) arns = (arns*a)%m;
        a = (a*a)%m;
        b>>=1;
    }
    return arns%m;
}
```

8.10 Binomial Coefficients

```
vector<ll> fact(MAX);
vector<ll> i_fact(MAX);

ll binpow(ll a, ll b){
    ll arns = 1;
    a%=MOD;
    while(b>0){
        if(b&1)
           arns = (arns*a)%MOD;
        a = (a*a)%MOD;
        b>>=1;
    }
    return arns;
}

ll modInv(ll a, ll m=MOD){
    return binpow(a,MOD-2);
}

void factorial(){
    fact[0] = fact[1] = 1;
    for(int i=2; i<MAX; i++){
        fact[i] = (fact[i-1] * i)%MOD;
    }

    i_fact[MAX-1] = modInv(fact[MAX-1]);
    for(int i=MAX-2; i>=0; i--){
        i_fact[i] = (i_fact[i+1]*(i+1))%MOD;
    }

}

ll bn_coef(ll n, ll k){
    return fact[n] * i_fact[n-k]%MOD *i_fact[k]%MOD;
}
```

8.11 Extended Euclidean

```
// Extended Euclidean Algorithm
// Returns gcd(a, b) and finds x, y such that ax + by = gcd(a, b)
int extgcd(int a, int b, int &x, int &y) {
    if (b == 0) {
        x = 1;
        y = 0;
        return a;
    }
    int x1, y1;
    int g = extgcd(b, a % b, x1, y1);
    x = y1;
    y = x1 - (a / b) * y1;
    return g;
}
```

8.12 FFT

```
// FFT works with sizes that are powers of 2.
// The size should be >= n+m-1 where n,m are input sizes.

typedef complex<double> cd;
double PI = acos(-1);

// My favorite black box <3
vector<cd> fft(vector<cd> a, int d = 1) {
    int n = a.size();
    vector<cd> r(n);

    for (int k = 0; k < n; k++) {
        int b = 0;
        for (int z = 1; z < n; z *= 2) {
            b *= 2;
            if (k & z)
                b++;
        }
        r[b] = a[k];
    }

    for (int m = 2; m <= n; m *= 2) {
        cd wm = exp(cd{0, d * 2 * PI / m});

        for (int k = 0; k < n; k += m) {
            cd w = 1;
            for (int j = 0; j < m / 2; j++) {
                cd u = r[k + j];
                cd t = w * r[k + j + m / 2];
                r[k + j] = u + t;
                r[k + j + m / 2] = u - t;
                w = w * wm;
            }
        }
    }

    if (d == -1) {
        for (int i = 0; i < n; i++)
            r[i] /= n;
    }

    return r;
}

```

```cpp
void solve() {
    int n, m;
    cin>>n>>m;

    vector<int> s(n), msk(m);

    for(int i = 0; i < n; i++){
        cin >> s[i];
    }
    for(int i = 0; i < m; i++){
        cin >> msk[i];
    }

    reverse(msk.begin(), msk.end());

    int sz = 1;
    while (sz < n + m - 1) sz <<= 1;
    vector<cd> a(sz, cd(0,0)), b(sz, cd(0,0));

    for(int i=0; i<n; i++){
        a[i] += cd(s[i],0);
    }


    for(int i=0; i<m; i++){
        b[i] += cd(msk[i],0);
    }

    auto A = fft(a);
    auto B = fft(b);
    vector<cd> C(sz);

    for(int i = 0; i < sz; i++){
        C[i] = A[i] * B[i];
    }

    auto conv = fft(C, -1);

    for(int i = 0; i < n+m-1; i++){
        cout << (ll)round(conv[i].real()) << " ";
    }
    cout << endl;
}
```

8.13 Big Integer

```cpp
static const __int128 MOD = (__int128)1000000000000000031ULL;
static const __int128 g   = (__int128)42ULL;

ostream &operator<<(ostream &os, const __int128 & value){
    char buffer[64];
    char *pos = end(buffer)-1;
    *pos = '\0';
    __int128 tmp = value < 0 ? -value : value;

    do{
        --pos;
        *pos = tmp%10 + '0';
        tmp/=10;
    }while(tmp!=0);
    if(value < 0){
        --pos;
        *pos = '-';
    }

    return os << pos;
}

inline __int128 mulm(__int128 a, __int128 b){
    __uint128_t z = ( (__uint128_t)a * (__uint128_t)b );
    return (__int128)( z % MOD );
}
__int128 binpow(__int128 a, __int128 e){
    __int128 r = 1;
    while (e > 0){
        if (e & 1) r = mulm(r, a);
        a = mulm(a, a);
        e >>= 1;
    }
    return r;
}

void solve(){
  int n; cin>>n;

    __int128 p = 300;
  __int128 cnt = 1;
  for(ll i = 1000000LL;i>n;i--){
    p = binpow(g,p);
  }
  cout << p << '\n';
}
```

8.14 NTT

```cpp
// Number Theoretic Transform (NTT)
// Usage: exact polynomial convolution under modulo arithmetic (no floating-point precision issues).
// Good for multiplying polynomials / counting ways where answers are required modulo MOD.
```

```cpp
// Constraints: n must be a power of 2 and n <= ROOT_PW for this modulus/root setup.

const int MOD = 7340033;            // 7 * 2^20 + 1
const int ROOT = 5;                 // primitive root for MOD
const int ROOT_INV = 4404020;       // inverse(ROOT) mod MOD
const int ROOT_PW = 1 << 20;        // max power-of-two supported

int binpow(int a, int b, int m = MOD) {
    int arns = 1;
    while (b > 0) {
        if (b & 1) arns = (int)(1LL * arns * a % m);
        a = (int)(1LL * a * a % m);
        b >>= 1;
    }
    return arns;
}

void ntt(vector<int> &a, bool invert) {
    int n = (int)a.size();

    // Bit-reversal permutation.
    for (int i = 1, j = 0; i < n; i++) {
        int bit = n >> 1;
        while (j & bit) {
            j ^= bit;
            bit >>= 1;
        }
        j ^= bit;
        if (i < j) swap(a[i], a[j]);
    }

    for (int len = 2; len <= n; len <<= 1) {
        int wlen = invert ? ROOT_INV : ROOT;
        for (int i = len; i < ROOT_PW; i <<= 1) {
            wlen = (int)(1LL * wlen * wlen % MOD);
        }

        for (int i = 0; i < n; i += len) {
            int w = 1;
            for (int j = 0; j < len / 2; j++) {
                int u = a[i + j];
                int v = (int)(1LL * a[i + j + len / 2] * w % MOD);

                a[i + j] = u + v;
                if (a[i + j] >= MOD) a[i + j] -= MOD;

                a[i + j + len / 2] = u - v;
                if (a[i + j + len / 2] < 0) a[i + j + len / 2] += MOD;

                w = (int)(1LL * w * wlen % MOD);
            }
        }
    }

    if (invert) {
        int n_inv = binpow(n, MOD - 2, MOD);
        for (int &x : a) {
            x = (int)(1LL * x * n_inv % MOD);
        }
    }
}

// Polynomial multiplication modulo MOD with NTT.
vector<int> multiply_ntt(vector<int> a, vector<int> b) {
    int n = 1;
    while (n < (int)a.size() + (int)b.size() - 1) n <<= 1;

    // Must fit this NTT configuration. Use another modulus/CRT if larger needed.
    if (n > ROOT_PW) return {};

    a.resize(n);
    b.resize(n);

    ntt(a, false);
    ntt(b, false);

    for (int i = 0; i < n; i++) {
        a[i] = (int)(1LL * a[i] * b[i] % MOD);
    }

    ntt(a, true);
    return a;
}

void solve() {
    // Example usage:
    // Input two polynomials A (size n) and B (size m), output coefficients of A*B mod MOD.
    int n, m;
    cin >> n >> m;
    vector<int> a(n), b(m);
    for (int &x : a) cin >> x;
    for (int &x : b) cin >> x;

    vector<int> c = multiply_ntt(a, b);
    if (c.empty()) {
        // This means size exceeded ROOT_PW with current modulus configuration.
        return;
    }

    for (int i = 0; i < n + m - 1; i++) {
        cout << c[i] << " ";
```

```
    }
    cout << endl;
}
```

8.15 Graph Paths

```
vector<vector<int>> mat_mult(vector<vector<int>> &A, vector<vector<
    int>> &B, int n, int m, int p){
    vector<vector<int>> arns(n,vector<int> (p));
    for(int i=0; i<n; i++){
        for(int j=0; j<m; j++){
            for(int k=0; k<p; k++){
                arns[i][k] += A[i][j] * B[j][k];
                arns[i][k]%=MOD;
            }
        }
    }

    return arns;
}

vector<vector<int>> mat_pow(vector<vector<int>> &A, int n){
    int sz = A.size();
    vector<vector<int>> arns(sz,vector<int> (sz,0));
    for(int i=0; i<sz; i++)
        arns[i][i] = 1;
    while(n>0){
        if(n&1) arns = mat_mult(arns,A,sz,sz,sz);
        A = mat_mult(A,A,sz,sz,sz);
        n>>=1;
    }
    return arns;
}

void solve(){
    int n, m, k; cin>>n>>m>>k;
    vector<vector<int>> adj(n,vector<int>(n,0));
    for(int i=0; i<m; i++){
        int a,b; cin>>a>>b;
        a--; b--;
        adj[a][b]++;
    }
    vector<vector<int>> arns = mat_pow(adj,k);
    cout<<arns[0][n-1]<<endl;
}
```

8.16 Graph Paths Min-Plus

```
const int MAX = 2e6+1;
const int MOD = 1e9+7;
const int INF = 4e18;

int modInv(int num){
    if(num == 1) return 1;
    return (MOD - ((MOD/num)*modInv(MOD%num))%MOD+MOD)%MOD;
}

vector<vector<int>> mat_zum(vector<vector<int>> &A, vector<vector<
    int>> &B, int n, int m, int p){
    vector<vector<int>> arns(n,vector<int> (p, INF));
    for(int i=0; i<n; i++){
        for(int j=0; j<m; j++){
            for(int k=0; k<p; k++){
                if (A[i][j] < INF && B[j][k] < INF) {
                    arns[i][k] = min(arns[i][k], A[i][j] + B[j][k]);
                }
            }
        }
    }

    return arns;
}

vector<vector<int>> mat_pow(vector<vector<int>> &A, int n){
    int sz = A.size();
    vector<vector<int>> arns(sz,vector<int> (sz,INF));
    for(int i = 0; i < sz; i++) arns[i][i] = 0;
    while(n>0){
        if(n&1) arns = mat_zum(arns,A,sz,sz,sz);
        A = mat_zum(A,A,sz,sz,sz);
        n>>=1;
    }
    return arns;
}

void solve(){
    int n, m, k; cin>>n>>m>>k;
    vector<vector<int>> adj(n,vector<int>(n,INF));
    for(int i=0; i<m; i++){
        int a,b,c; cin>>a>>b>>c;
        a--; b--;
        adj[a][b]=min(adj[a][b], c);
    }
    vector<vector<int>> arns = mat_pow(adj,k);
    if(arns[0][n-1] == INF) cout<<"-1"<<endl;
    else cout<<arns[0][n-1]<<endl;
}
```

8.17 Log Fibonacci

```cpp
const int MAX = 2e6+1;
const int MOD = 1e9+7;

vector<vector<int>> mat_mult(vector<vector<int>> &A, vector<vector<
    int>> &B, int n, int m, int p){
    vector<vector<int>> arns(n,vector<int> (p));
    for(int i=0; i<n; i++){
        for(int j=0; j<m; j++){
            for(int k=0; k<p; k++){
                arns[i][k] += A[i][j] * B[j][k];
                arns[i][k]%=MOD;
            }
        }
    }

    return arns;
}

vector<vector<int>> mat_pow(vector<vector<int>> &A, int n){
    vector<vector<int>> arns = {{1,0},{0,1}};
    while(n>0){
        if(n&1) arns = mat_mult(arns,A,2,2,2);
        A = mat_mult(A,A,2,2,2);
        n>>=1;
    }
    return arns;
}

void solve(){
    int n; cin>>n;
    vector<vector<int>> a = {{1,1},{1,0}};
    vector<vector<int>> arns = mat_pow(a,n);

    cout<<arns[0][1]<<endl;
}
```

8.18 Matrix Determinant

```cpp
const double EPS = 1E-9;

double matrix_determinant(vector<vector<double>> a) {
    int n = a.size();
    double det = 1;
    for (int i=0; i<n; ++i) {
        int k = i;
        for (int j=i+1; j<n; ++j)
            if (abs (a[j][i]) > abs (a[k][i]))
                k = j;
        if (abs (a[k][i]) < EPS) {
            det = 0;
            break;
        }
        swap (a[i], a[k]);
        if (i != k)
            det = -det;
        det *= a[i][i];
        for (int j=i+1; j<n; ++j)
            a[i][j] /= a[i][i];
        for (int j=0; j<n; ++j)
            if (j != i && abs (a[j][i]) > EPS)
                for (int k=i+1; k<n; ++k)
                    a[j][k] -= a[i][k] * a[j][i];
    }
    return det;
}
```

8.19 Matrix Rank

```cpp
const double EPS = 1E-9;

int compute_rank(vector<vector<double>> A) {
    int n = A.size();
    int m = A[0].size();

    int rank = 0;
    vector<bool> row_selected(n, false);
    for (int i = 0; i < m; ++i) {
        int j;
        for (j = 0; j < n; ++j) {
            if (!row_selected[j] && abs(A[j][i]) > EPS)
                break;
        }

        if (j != n) {
            ++rank;
            row_selected[j] = true;
            for (int p = i + 1; p < m; ++p)
                A[j][p] /= A[j][i];
            for (int k = 0; k < n; ++k) {
                if (k != j && abs(A[k][i]) > EPS) {
                    for (int p = i + 1; p < m; ++p)
                        A[k][p] -= A[j][p] * A[k][i];
                }
            }
        }
    }
    return rank;
}
```

8.20 Gauss-Jordan

```cpp
const double EPS = 1e-9;
const int INF = 2; // it doesn't actually have to be infinity or a big number

int gauss (vector < vector<double> > a, vector<double> & ans) {
    int n = (int) a.size();
    int m = (int) a[0].size() - 1;

    vector<int> where (m, -1);
    for (int col=0, row=0; col<m && row<n; ++col) {
        int sel = row;
        for (int i=row; i<n; ++i)
            if (abs (a[i][col]) > abs (a[sel][col]))
                sel = i;
        if (abs (a[sel][col]) < EPS)
            continue;
        for (int i=col; i<=m; ++i)
            swap (a[sel][i], a[row][i]);
        where[col] = row;

        for (int i=0; i<n; ++i)
            if (i != row) {
                double c = a[i][col] / a[row][col];
                for (int j=col; j<=m; ++j)
                    a[i][j] -= a[row][j] * c;
            }
        ++row;
    }

    ans.assign (m, 0);
    for (int i=0; i<m; ++i)
        if (where[i] != -1)
            ans[i] = a[where[i]][m] / a[where[i]][i];
    for (int i=0; i<n; ++i) {
        double sum = 0;
        for (int j=0; j<m; ++j)
            sum += ans[j] * a[i][j];
        if (abs (sum - a[i][m]) > EPS)
            return 0;
    }

    for (int i=0; i<m; ++i)
        if (where[i] == -1)
            return INF;
    return 1;
}
```

8.21 Miller-Rabin (Primality Test)

```cpp
// Deterministic Miller-Rabin primality test for unsigned 64-bit integers.
// Purpose: fast primality checks for big n (up to 1e18 / uint64 range).
// Complexity per query: O(k * log n), where k is the number of bases (here constant 7).
// Typical CP usage: primality checks in factorization, number theory tasks, random tests.

using u64 = uint64_t;
using u128 = __uint128_t;

u64 mod_mul(u64 a, u64 b, u64 mod) {
  return (u128)a * b % mod;
}

u64 mod_pow(u64 a, u64 e, u64 mod) {
  u64 r = 1;
  while (e > 0) {
    if (e & 1) r = mod_mul(r, a, mod);
    a = mod_mul(a, a, mod);
    e >>= 1;
  }
  return r;
}

bool isPrime(u64 n) {
  if (n < 2) return false;

  // Fast small-prime filter.
  for (u64 p : {2ULL, 3ULL, 5ULL, 7ULL, 11ULL, 13ULL, 17ULL, 19ULL, 23ULL, 29ULL, 31ULL, 37ULL}) {
    if (n % p == 0) return n == p;
  }

  // Write n - 1 = d * 2^s with d odd.
  u64 d = n - 1;
  int s = 0;
  while ((d & 1) == 0) {
    d >>= 1;
    s++;
  }

  // Deterministic bases for testing all uint64 values.
  // Valid for n < 2^64.
  for (u64 a : {2ULL, 325ULL, 9375ULL, 28178ULL, 450775ULL, 9780504ULL, 1795265022ULL}) {
    if (a % n == 0) continue;

    u64 x = mod_pow(a % n, d, n);
```

```
    if (x == 1 || x == n - 1) continue;

    bool witness = true;
    for (int r = 1; r < s; r++) {
      x = mod_mul(x, x, n);
      if (x == n - 1) {
        witness = false;
        break;
      }
    }

    if (witness) return false;
  }

  return true;
}

// Usage example:
// u64 n; cin >> n;
// cout << (isPrime(n) ? "YES" : "NO") << '\n';
```

8.22 Linear Sieve

```
const int N = 10000000;
vector<int> lp(N+1);
vector<int> pr;

for (int i=2; i <= N; ++i) {
    if (lp[i] == 0) {
        lp[i] = i;
        pr.push_back(i);
    }
    for (int j = 0; i * pr[j] <= N; ++j) {
        lp[i * pr[j]] = pr[j];
        if (pr[j] == lp[i]) {
            break;
        }
    }
}
```

8.23 Euler Totient Function

```
void phi_1_to_n(int n) {
    vector<int> phi(n + 1);
    for (int i = 0; i <= n; i++)
        phi[i] = i;

    for (int i = 2; i <= n; i++) {
        if (phi[i] == i) {
            for (int j = i; j <= n; j += i)
                phi[j] -= phi[j] / i;
        }
    }
}
```

8.24 Fast Totient Function

```
int prime[MAX_PRIMES];
int SPF[MAX_SIZE];
int et[MAX_SIZE];
int idx=0;


ll bin_pow(ll x,ll k){
    ll res=1;
    while(k){
        if(k&1){
            res=(res*x)%MOD;
        }
        x=(x*x%MOD);
        k/=2;
    }
    return res;
}

ll inv(ll x){ return bin_pow(x,MOD-2);}

void manipulated_seive(int N)
{

    for (int i=2; i<N ; i++)
    {

        if (SPF[i]==0)
        {
            prime[idx++]=i;
            SPF[i] = i;
            et[i]=i-1;
        }
        else{
            //et[i]=(SPF[i]-1)*et[i/SPF[i]];
        }
        for (int j=0;
             j < idx &&
             i*prime[j] < N && prime[j] <= SPF[i];
             j++)
        {
```

```
41
42
43              if(prime[j]==SPF[i]){
44                  et[i*prime[j]]=prime[j]*et[i];
45              }
46              else{
47                  et[i*prime[j]]=(prime[j]-1)*et[i];
48              }
49              SPF[i*prime[j]] = prime[j] ;
50          }
51      }
52  }
53
54  // driver  program to test above function
55  int main()
56  {
57      ios_base::sync_with_stdio(false);
58      cin.tie(NULL);
59      int N,q; cin>>N>>q;
60      //memset(isprime,1,sizeof(isprime));
61      manipulated_seive(N+1);
62
63      ll Q=inv(N-1);
64      for(int i=0;i<q;i++){
65          int x; cin>>x;
66          ll P=(N-x)+et[x]-1;
67          //cout<<et[x]<<endl;
68          cout<<(P*Q)%MOD<<"\n";
69      }
70      return 0;
71  }
```

8.25 Diophantine Equations

```
1   void shift_solution(int & x, int & y, int a, int b, int cnt) {
2       x += cnt * b;
3       y -= cnt * a;
4   }
5
6   int find_all_solutions(int a, int b, int c, int minx, int maxx, int
        miny, int maxy) {
7       int x, y, g;
8       if (!find_any_solution(a, b, c, x, y, g))
9           return 0;
10      a /= g;
11      b /= g;
12
13      int sign_a = a > 0 ? +1 : -1;
14      int sign_b = b > 0 ? +1 : -1;
15
16      shift_solution(x, y, a, b, (minx - x) / b);
17      if (x < minx)
18          shift_solution(x, y, a, b, sign_b);
19      if (x > maxx)
20          return 0;
21      int lx1 = x;
22
23      shift_solution(x, y, a, b, (maxx - x) / b);
24      if (x > maxx)
25          shift_solution(x, y, a, b, -sign_b);
26      int rx1 = x;
27
28      shift_solution(x, y, a, b, -(miny - y) / a);
29      if (y < miny)
30          shift_solution(x, y, a, b, -sign_a);
31      if (y > maxy)
32          return 0;
33      int lx2 = x;
34
35      shift_solution(x, y, a, b, -(maxy - y) / a);
36      if (y > maxy)
37          shift_solution(x, y, a, b, sign_a);
38      int rx2 = x;
39
40      if (lx2 > rx2)
41          swap(lx2, rx2);
42      int lx = max(lx1, lx2);
43      int rx = min(rx1, rx2);
44
45      if (lx > rx)
46          return 0;
47      return (rx - lx) / abs(b) + 1;
48  }
```

8.26 Mod Struct

```
1   typedef long long ll;
2
3   struct llm {
4       static const ll MOD=1e9+7;
5       ll v;
6       llm& operator+=(const llm& o){v=(v+o.v)%MOD;return *this;}
7       llm& operator-=(const llm& o){v=(v-o.v+MOD)%MOD;return *this;}
8       llm& operator*=(const llm& o){v=v*o.v%MOD;return *this;}
9       static ll pw(ll x,ll k){ll r=1;while(k){if(k&1)r=r*x%MOD;x=x*x%
            MOD;k/=2;}return r;}
10      llm& operator/=(const llm& o){v=v*pw(o.v,MOD-2)%MOD;return *this
            ;}
11      friend llm operator+(llm l,const llm r){return l+=r;}
12      friend llm operator-(llm l,const llm r){return l-=r;}
```

```
    friend llm operator*(llm l,const llm r){return l*=r;}
    friend llm operator/(llm l,const llm r){return l/=r;}

};
```

8.27 Factorials

```
void precompute(ll n = MAXN - 1) {
    factorial[0] = factorial[1] = 1;

    // Compute factorials
    for (ll i = 2; i <= n; i++) {
        factorial[i] = (factorial[i - 1] * i) % MOD;
    }

    // Compute inverse factorials efficiently
    inv_factorial[n] = modInv(factorial[n]);
    for (ll i = n - 1; i >= 0; i--) {
        inv_factorial[i] =
            (inv_factorial[i + 1] * (i + 1)) % MOD;
    }
}
```

8.28 Number of Divisors

8.29 Modular Inverse

```
ll modInv(ll a, ll mod = MOD) {
    return power(a, mod - 2, mod);
}
```

8.30 Discrete Logarithm

```
// Returns minimum x for which a ^ x % m = b % m.
int solve(int a, int b, int m) {
    a %= m, b %= m;
    int k = 1, add = 0, g;
    while ((g = gcd(a, m)) > 1) {
        if (b == k)
            return add;
        if (b % g)
            return -1;
        b /= g, m /= g, ++add;
        k = (k * 1ll * a / g) % m;
    }

    int n = sqrt(m) + 1;
    int an = 1;
    for (int i = 0; i < n; ++i)
        an = (an * 1ll * a) % m;

    unordered_map<int, int> vals;
    for (int q = 0, cur = b; q <= n; ++q) {
        vals[cur] = q;
        cur = (cur * 1ll * a) % m;
    }

    for (int p = 1, cur = k; p <= n; ++p) {
        cur = (cur * 1ll * an) % m;
        if (vals.count(cur)) {
            int ans = n * p - vals[cur] + add;
            return ans;
        }
    }
    return -1;
}
```

8.31 Segmented Sieve

```
vector<char> segmentedSieve(long long L, long long R) {
    // generate all primes up to sqrt(R)
    long long lim = sqrt(R);
    vector<char> mark(lim + 1, false);
    vector<long long> primes;
    for (long long i = 2; i <= lim; ++i) {
        if (!mark[i]) {
            primes.emplace_back(i);
            for (long long j = i * i; j <= lim; j += i)
                mark[j] = true;
        }
    }

    vector<char> isPrime(R - L + 1, true);
    for (long long i : primes)
        for (long long j = max(i * i, (L + i - 1) / i * i); j <= R;
            j += i)
            isPrime[j - L] = false;
    if (L == 1)
        isPrime[0] = false;
    return isPrime;
}
```

8.32 Sieve of Eratosthenes

```cpp
vector<bool> sieve_eratosthenes(int n) {
    vector<bool> is_prime(n+1, true);
    is_prime[0] = is_prime[1] = false;
    for (int i = 2; i * i <= n; i++) {
        if (is_prime[i]) {
            for (int j = i * i; j <= n; j += i)
                is_prime[j] = false;
        }
    }
    return is_prime;
}

int count_primes(int n) {
    const int S = 10000;

    vector<int> primes;
    int nsqrt = sqrt(n);
    vector<char> is_prime(nsqrt + 2, true);
    for (int i = 2; i <= nsqrt; i++) {
        if (is_prime[i]) {
            primes.push_back(i);
            for (int j = i * i; j <= nsqrt; j += i)
                is_prime[j] = false;
        }
    }

    int result = 0;
    vector<char> block(S);
    for (int k = 0; k * S <= n; k++) {
        fill(block.begin(), block.end(), true);
        int start = k * S;
        for (int p : primes) {
            int start_idx = (start + p - 1) / p;
            int j = max(start_idx, p) * p - start;
            for (; j < S; j += p)
                block[j] = false;
        }
        if (k == 0)
            block[0] = block[1] = false;
        for (int i = 0; i < S && start + i <= n; i++) {
            if (block[i])
                result++;
        }
    }
    return result;
}
```

8.33 Simpson Method

```cpp
const int N = 1000 * 1000; // number of steps (already multiplied by 2)

double simpson_integration(double a, double b){
    double h = (b - a) / N;
    double s = f(a) + f(b); // a = x_0 and b = x_2n
    for (int i = 1; i <= N - 1; ++i) { // Refer to final Simpson's formula
        double x = a + h * i;
        s += f(x) * ((i & 1) ? 4 : 2);
    }
    s *= h / 3;
    return s;
}
```

8.34 Precompute Pascal Triangle

```cpp
int binomialCoeff(int n, int k) {
    vector<int> dp(k + 1);

      // nC0 is 1
    dp[0] = 1;

    for (int i = 1; i <= n; i++) {

        // Compute next row of pascal triangle using
        // the previous row
        for (int j = min(i, k); j > 0; j--)
            dp[j] = dp[j] + dp[j - 1];
    }
    return dp[k];
}
```

8.35 Sqrt and Inverse Sqrt

```cpp
double sqrt_newton(double n) {
    const double eps = 1E-15;
    double x = 1;
    for (;;) {
        double nx = (x + n / x) / 2;
        if (abs(x - nx) < eps)
            break;
        x = nx;
    }
    return x;
```

```
}

int isqrt_newton(int n) {
    int x = 1;
    bool decreased = false;
    for (;;) {
        int nx = (x + n / x) >> 1;
        if (x == nx || nx > x && decreased)
            break;
        decreased = nx < x;
        x = nx;
    }
    return x;
}
```

8.36 Sum of Divisors

```
long long SumOfDivisors(long long num) {
    long long total = 1;

    for (int i = 2; (long long)i * i <= num; i++) {
        if (num % i == 0) {
            int e = 0;
            do {
                e++;
                num /= i;
            } while (num % i == 0);

            long long sum = 0, pow = 1;
            do {
                sum += pow;
                pow *= i;
            } while (e-- > 0);
            total *= sum;
        }
    }
    if (num > 1) {
        total *= (1 + num);
    }
    return total;
}
```

8.37 Ternary Search

```
double ternary_search(double l, double r) {
    double eps = 1e-9;              //set the error limit here
    while (r - l > eps) {
        double m1 = l + (r - l) / 3;
        double m2 = r - (r - l) / 3;
        double f1 = f(m1);      //evaluates the function at m1
        double f2 = f(m2);      //evaluates the function at m2
        if (f1 < f2)
            l = m1;
        else
            r = m2;
    }
    return f(l);                    //return the maximum of f(x) in
        [l, r]
}
```

8.38 Trivial Factorization

```
vector<long long> trial_division2(long long n) {
    vector<long long> factorization;
    while (n % 2 == 0) {
        factorization.push_back(2);
        n /= 2;
    }
    for (long long d = 3; d * d <= n; d += 2) {
        while (n % d == 0) {
            factorization.push_back(d);
            n /= d;
        }
    }
    if (n > 1)
        factorization.push_back(n);
    return factorization;
}
```

8.39 Chinese Remainder Theorem

```
#include <bits/stdc++.h>
using namespace std;
typedef long long ll;

// Extended GCD returning gcd and setting x, y such that ax + by =
    gcd(a, b)
ll extgcd(ll a, ll b, ll &x, ll &y) {
    if (b == 0) {
        x = 1;
        y = 0;
        return a;
    }
    ll x1, y1;
```

```
    ll g = extgcd(b, a % b, x1, y1);
    x = y1;
    y = x1 - (a / b) * y1;
    return g;
}

// Chinese Remainder Theorem
// Given x == a[i] (mod m[i]) for pairwise coprime m[i], returns (x_mod, lcm)
// where x is the unique solution modulo lcm = product of m[i]
pair<ll, ll> crt(const vector<ll>& a, const vector<ll>& m) {
    ll x = 0;
    ll M = 1;

    for (size_t i = 0; i < a.size(); i++) {
        ll ai = ((a[i] % m[i]) + m[i]) % m[i];
        ll mi = m[i];

        // Solve: M * t == (ai - x) (mod mi)
        ll diff = (ai - x) % mi;
        if (diff < 0) diff += mi;

        ll t, y;
        ll g = extgcd(M, mi, t, y);
        // M and mi are coprime, so g == 1
        t = (t % mi + mi) % mi;
        t = (__int128)t * diff % mi;

        x = x + (__int128)M * t;
        M = (__int128)M * mi / g;

        x %= M;
        if (x < 0) x += M;
    }

    return {x, M};
}
```

8.40 Garner's Algorithm

```
// Garner's Algorithm for Chinese Remainder Theorem with non-coprime moduli
// Solves x == a_i (mod m_i) for arbitrary moduli
// Returns x mod lcm(m_i), or -1 if no solution

#include <bits/stdc++.h>
using namespace std;
using ll = long long;

ll egcd(ll a, ll b, ll &x, ll &y) {
    if (b == 0) { x = 1; y = 0; return a; }
    ll x1, y1;
    ll g = egcd(b, a % b, x1, y1);
    x = y1;
    y = x1 - (a / b) * y1;
    return g;
}

// Extended modular inverse that works for non-coprime case
// Returns inverse of a modulo mod, or -1 if doesn't exist
// When gcd(a, mod) = d > 1, the inverse of (a/d) modulo (mod/d) is returned
// and the gcd is returned in g
ll modInvExt(ll a, ll mod, ll &g) {
    ll x, y;
    g = egcd(a, mod, x, y);
    if (g != 1) return -1;
    return (x % mod + mod) % mod;
}

// Check if system x == a[i] (mod m[i]) is consistent
bool checkConsistency(const vector<ll> &a, const vector<ll> &m) {
    int n = a.size();
    for (int i = 0; i < n; i++) {
        for (int j = i + 1; j < n; j++) {
            ll g = std::gcd(m[i], m[j]);
            if ((a[j] - a[i]) % g != 0) return false;
        }
    }
    return true;
}

// Compute lcm of all moduli
ll lcmAll(const vector<ll> &m) {
    ll res = 1;
    for (ll mod : m) {
        res = std::lcm(res, mod);
    }
    return res;
}

// Garner's algorithm for arbitrary (not necessarily coprime) moduli
// Returns solution x mod lcm(m_i), or -1 if no solution exists
// Uses Garner's recurrence with precomputed inverses for coprime components
ll garner(const vector<ll> &a, const vector<ll> &m) {
    int n = a.size();
    if (n == 0) return 0;

    // Check consistency of the system
    if (!checkConsistency(a, m)) return -1;

    // Build solution iteratively using Garner's recurrence
```

```
    // At step i: we have cur satisfying congruences 0..i-1
    // We want to find t such that: cur + Q * t == a[i] (mod m[i])
    // where Q = lcm(m[0], ..., m[i-1])
    //
    // This is equivalent to Q * t == (a[i] - cur) (mod m[i])
    // Let d = gcd(Q, m[i]). For solution to exist, d | (a[i] - cur)
    // Then: (Q/d) * t == (a[i] - cur)/d (mod m[i]/d)
    // Since gcd(Q/d, m[i]/d) = 1, we can find inverse

    ll cur = a[0] % m[0];
    if (cur < 0) cur += m[0];
    ll Q = m[0];  // lcm of processed moduli

    for (int i = 1; i < n; i++) {
        ll diff = a[i] - cur;
        ll mi = m[i];

        // d = gcd(Q, mi)
        ll d = std::gcd(Q, mi);

        // Check divisibility: solution exists iff d | diff
        if (diff % d != 0) return -1;  // Should not happen if
            consistency check passed

        // Reduce: (Q/d) * t == (diff/d) (mod mi/d)
        ll Qd = Q / d;
        ll mid = mi / d;
        ll diffd = diff / d;

        // Find inverse of Qd modulo mid
        ll inv, g;
        inv = modInvExt(Qd % mid, mid, g);
        if (inv == -1) return -1;  // Should not happen

        // t = (diffd * inv) mod mid
        ll t = (diffd % mid * inv) % mid;
        if (t < 0) t += mid;

        // Update cur and Q
        cur = cur + Q * t;
        Q = std::lcm(Q, mi);

        // Normalize cur to be within [0, Q)
        cur %= Q;
        if (cur < 0) cur += Q;
    }

    return cur % lcmAll(m);
}
```

8.41 Polynomial Operations

```
// Polynomial Operations class
// Supports +, -, *, /, %, eval, and interpolate over arbitrary type
    T
// Uses __int128 for safe multiplication and reduces mod 1e9+7 when
    needed

#include <bits/stdc++.h>
using namespace std;

typedef long long ll;
const ll MOD = 1e9 + 7;

template<typename T>
struct Poly {
    vector<T> a;

    // Constructors
    Poly() {}
    Poly(const vector<T>& v) : a(v) {}
    Poly(const T& scalar) : a(1, scalar) {}

    // Degree, -1 for zero polynomial
    int deg() const {
        return (int)a.size() - 1;
    }

    // Ensure no trailing zeros
    void normalize() {
        while (!a.empty() && a.back() == T(0)) a.pop_back();
    }

    // Coefficient access
    T& operator[](int i) { return a[i]; }
    const T& operator[](int i) const { return a[i]; }

    // Addition
    Poly operator+(const Poly& o) const {
        Poly res;
        res.a.resize(max(size(), o.size()));
        for (size_t i = 0; i < res.a.size(); i++) {
            T x = i < size() ? a[i] : T(0);
            T y = i < o.size() ? o.a[i] : T(0);
            res.a[i] = x + y;
        }
        res.normalize();
        return res;
    }

    // Subtraction
    Poly operator-(const Poly& o) const {
```

```
        Poly res;
        res.a.resize(max(size(), o.size()));
        for (size_t i = 0; i < res.a.size(); i++) {
            T x = i < size() ? a[i] : T(0);
            T y = i < o.size() ? o.a[i] : T(0);
            res.a[i] = x - y;
        }
        res.normalize();
        return res;
    }

    // Scalar multiplication
    Poly operator*(const T& scalar) const {
        Poly res;
        res.a.resize(size());
        for (size_t i = 0; i < size(); i++) {
            res.a[i] = a[i] * scalar;
        }
        res.normalize();
        return res;
    }

    // Polynomial multiplication using __int128 for intermediate results
    Poly operator*(const Poly& o) const {
        if (deg() < 0 || o.deg() < 0) return Poly();

        Poly res;
        res.a.resize(size() + o.size() - 1);
        for (size_t i = 0; i < size(); i++) {
            for (size_t j = 0; j < o.size(); j++) {
                __int128 prod = (__int128)a[i] * (__int128)o.a[j];
                ll prod_ll = (ll)(prod % MOD);
                res.a[i + j] = (T)((ll)res.a[i + j] + prod_ll);
                if (sizeof(T) != sizeof(ll)) {
                    // Keep values bounded for non-ll types
                    ll val = (ll)res.a[i + j] % MOD;
                    if (val < 0) val += MOD;
                    res.a[i + j] = (T)val;
                }
            }
        }
        res.normalize();
        return res;
    }

    // Division with remainder ( Euclidean division )
    // Returns quotient, remainder is stored in this object
    Poly divmod(const Poly& o, Poly& remainder) const {
        if (deg() < 0) {
            remainder = Poly();
            return Poly();
        }

        if (o.deg() < 0) {
            // Division by zero - should not happen in well-formed code
            remainder = *this;
            return Poly();
        }

        int n = deg();
        int m = o.deg();
        if (n < m) {
            remainder = *this;
            return Poly();
        }

        Poly q;
        q.a.resize(n - m + 1);
        remainder = *this;

        while (remainder.deg() >= m && remainder.deg() >= 0) {
            int d = remainder.deg() - m;
            T coef = remainder.a.back() / o.a.back();

            for (int i = 0; i <= m; i++) {
                remainder.a[d + i] = remainder.a[d + i] - coef * o.a[i];
            }
            q.a[d] = coef;
            remainder.normalize();
        }

        q.normalize();
        remainder.normalize();
        return q;
    }

    // Division operator (quotient only)
    Poly operator/(const Poly& o) const {
        Poly rem;
        return divmod(o, rem);
    }

    // Modulo operator (remainder only)
    Poly operator%(const Poly& o) const {
        Poly rem;
        divmod(o, rem);
        return rem;
    }

    // Evaluate polynomial at point x using Horner's method
    T eval(T x) const {
        if (deg() < 0) return T(0);
        T result = a.back();
```

```
        for (int i = deg() - 1; i >= 0; i--) {
            result = result * x + a[i];
        }
        return result;
    }

    // Interpolate: given xs and ys, find polynomial P such that P[xs[i]] = ys[i]
    // Uses Lagrange interpolation formula
    static Poly interpolate(const vector<T>& xs, const vector<T>& ys) {
        int n = xs.size();
        if (n == 0) return Poly();
        if (n == 1) return Poly(ys[0]);

        // Build Lagrange basis polynomials efficiently
        // Precompute prefix and suffix products
        vector<Poly> pref(n), suff(n);
        for (int i = 0; i < n; i++) {
            pref[i] = Poly({-xs[i], T(1)});
            suff[i] = Poly({-xs[i], T(1)});
        }
        for (int i = 1; i < n; i++) {
            pref[i] = pref[i] * pref[i - 1];
        }
        for (int i = n - 2; i >= 0; i--) {
            suff[i] = suff[i] * suff[i + 1];
        }

        Poly result;
        for (int i = 0; i < n; i++) {
            Poly num = (i == 0 ? Poly(T(1)) : pref[i - 1]) *
                       (i == n - 1 ? Poly(T(1)) : suff[i + 1]);

            // Compute denominator: prod_{j != i} (xs[i] - xs[j])
            T den = T(1);
            for (int j = 0; j < n; j++) {
                if (i != j) {
                    den = den * (xs[i] - xs[j]);
                }
            }

            // L_i(x) = num(x) / den
            // We need to handle division - for integers mod MOD, use modular inverse
            // For general T, we assume den divides num evenly or T supports division
            Poly term = num * (ys[i] / den);
            result = result + term;
        }

        result.normalize();
        return result;
    }

    // Size helper
    size_t size() const { return a.size(); }
};

// Explicit template instantiations for common types
template struct Poly<ll>;
template struct Poly<int>;
template struct Poly<long double>;
template struct Poly<double>;

void solve() {
    // Example usage:
    // Polynomial addition, subtraction, multiplication
    Poly<ll> p1({1, 2, 3});        // 1 + 2x + 3x^2
    Poly<ll> p2({1, 1});           // 1 + x

    auto p3 = p1 + p2;             // 2 + 3x + 3x^2
    auto p4 = p1 - p2;             // 0 + 1x + 3x^2
    auto p5 = p1 * p2;             // (1 + 2x + 3x^2)(1 + x)

    // Division: p1 / p2 = quotient and remainder
    Poly<ll> rem;
    Poly<ll> q = p1.divmod(p2, rem);

    // Evaluation
    (void)p1.eval(ll(2));      // p1(2) = 1 + 4 + 12 = 17

    // Interpolation
    vector<ll> xs = {0, 1, 2};
    vector<ll> ys = {1, 3, 7};     // P(0)=1, P(1)=3, P(2)=7 -> P(x)=1+2x+x^2
    Poly<ll> interp = Poly<ll>::interpolate(xs, ys);
}
```

8.42 Linear Recurrence

```
// Linear Recurrence solver using Berlekamp-Massey + Kitamasa
// O(k^3 log n) where k is the recurrence order
// Mod 1e9+7

#include <bits/stdc++.h>
using namespace std;

using ll = long long;
const ll MOD = 1e9 + 7;

ll modpow(ll a, ll e) {
    ll r = 1;
```

```
    while (e) {
        if (e & 1) r = r * a % MOD;
        a = a * a % MOD;
        e >>= 1;
    }
    return r;
}

// Berlekamp-Massey: find minimal linear recurrence relation
// Returns coefficients c where sequence s satisfies:
// s[n] = c[0]*s[n-1] + c[1]*s[n-2] + ... + c[k-1]*s[n-k]
vector<ll> berlekampMassey(const vector<ll> &s) {
    vector<ll> c, oldC;
    int f = -1;
    for (int i = 0; i < (int)s.size(); i++) {
        ll delta = s[i] % MOD;
        for (int j = 1; j <= (int)c.size(); j++) {
            delta = (delta - c[j - 1] * s[i - j]) % MOD;
        }
        if (delta == 0) continue;
        if (f == -1) {
            c.assign(i + 1, 0);
            c[i] = 1;
            f = i;
        } else {
            vector<ll> d = oldC;
            for (ll &x : d) x = (MOD - x) % MOD;
            d.insert(d.begin(), 1);
            ll df1 = 0;
            for (int j = 1; j <= (int)d.size(); j++) {
                df1 = (df1 + d[j - 1] * s[f + 1 - j]) % MOD;
            }
            ll coef = delta * modpow(df1, MOD - 2) % MOD;
            for (ll &x : d) x = x * coef % MOD;
            d.insert(d.end(), i - f - 1, 0);
            vector<ll> temp = c;
            c.resize(max(c.size(), d.size()));
            for (int j = 0; j < (int)d.size(); j++)
                c[j] = (c[j] + d[j]) % MOD;
            if (i - (int)temp.size() > f - (int)oldC.size()) {
                oldC = temp;
                f = i;
            }
        }
    }
    // Normalize: leading coefficient should be 1
    if (!c.empty() && c.back() != 1) {
        ll inv = modpow(c.back(), MOD - 2);
        for (ll &x : c) x = x * inv % MOD;
    }
    return c;
}

// Kitamasa: combine two polynomials modulo the recurrence relation
vector<ll> combine(const vector<ll> &a, const vector<ll> &b, const
    vector<ll> &trans) {
    int k = trans.size();
    vector<ll> res(2 * k);
    for (int i = 0; i < k; i++) {
        for (int j = 0; j < k; j++) {
            res[i + j] = (res[i + j] + a[i] * b[j]) % MOD;
        }
    }
    // Reduce degree >= k terms using the recurrence
    for (int i = 2 * k - 1; i >= k; i--) {
        for (int j = 1; j <= k; j++) {
            res[i - j] = (res[i - j] + res[i] * trans[j - 1]) % MOD;
        }
    }
    res.resize(k);
    return res;
}

// Kitamasa: compute coefficients for a^n
vector<ll> kitamasa(ll n, const vector<ll> &trans) {
    int k = trans.size();
    vector<ll> res(k), coef(k);
    res[0] = 1;
    coef[1 % k] = 1;  // Kitamasa(1) in the semiring: [0, 1, 0, ...]
    while (n > 0) {
        if (n & 1) res = combine(res, coef, trans);
        coef = combine(coef, coef, trans);
        n >>= 1;
    }
    return res;
}

// Compute the n-th term of a linear recurrence
// init: first k terms of the sequence
// trans: recurrence coefficients [c0, c1, ..., c_{k-1}]
//        meaning s[n] = c0*s[n-1] + c1*s[n-2] + ... + c_{k-1}*s[n-k
    ]
ll linear_recurrence(vector<ll> init, vector<ll> trans, ll n) {
    int k = trans.size();
    if (k == 0) return 0;
    if (n < (ll)init.size()) return init[n] % MOD;
    vector<ll> coef = kitamasa(n, trans);
    ll res = 0;
    for (int i = 0; i < k; i++) {
        res = (res + coef[i] * init[i]) % MOD;
    }
    return res;
}
```

8.43 Matrix Exponentiation

```cpp
// Matrix exponentiation: A^n in O(k^3 log n) for kxk matrix.
// Templated on element type (ll, modint, etc.).

#include <vector>

template<class T>
struct Matrix {
    int n;
    std::vector<std::vector<T>> a;

    Matrix() {}
    Matrix(int _n, int _m, T val = T{}) : n(_n), a(_n, std::vector<T
        >(_m, val)) {}

    static Matrix identity(int n) {
        Matrix res(n, n);
        for (int i = 0; i < n; i++)
            res.a[i][i] = T{1};
        return res;
    }

    Matrix operator+(const Matrix& o) const {
        Matrix res(n, o.n);
        for (int i = 0; i < n; i++)
            for (int j = 0; j < n; j++)
                res.a[i][j] = a[i][j] + o.a[i][j];
        return res;
    }

    Matrix operator*(const Matrix& o) const {
        int m = o.n;
        Matrix res(n, m);
        for (int i = 0; i < n; i++)
            for (int k = 0; k < n; k++) if (a[i][k] != T{})
                for (int j = 0; j < m; j++)
                    res.a[i][j] += a[i][k] * o.a[k][j];
        return res;
    }

    Matrix pow(long long e) const {
        Matrix res = identity(n);
        Matrix base = *this;
        while (e > 0) {
            if (e & 1) res = res * base;
            base = base * base;
            e >>= 1;
        }
        return res;
    }
};
```

8.44 Modular Square Root

```cpp
// Returns one x such that x^2 == a (mod p), where p is prime.
// The other root is p - x. Returns -1 if no square root exists.

typedef long long ll;

static ll mod_pow(ll a, ll e, ll mod) {
    ll r = 1;
    a %= mod;
    while (e > 0) {
        if (e & 1) r = r * a % mod;
        a = a * a % mod;
        e >>= 1;
    }
    return r;
}

ll mod_sqrt(ll a, ll p) {
    a %= p;
    if (a < 0) a += p;

    if (a == 0) return 0;
    if (p == 2) return a;

    // Legendre symbol: a^((p-1)/2) mod p == 1  -> quadratic residue
    //                                    == p-1 -> non-residue
    if (mod_pow(a, (p - 1) / 2, p) != 1) return -1;

    // p == 3 (mod 4): simple closed form
    if (p % 4 == 3) return mod_pow(a, (p + 1) / 4, p);

    // p == 1 (mod 4): Tonelli-Shanks
    // Factor p-1 = q * 2^s with q odd
    ll q = p - 1;
    int s = 0;
    while ((q & 1) == 0) {
        q >>= 1;
        ++s;
    }

    // Find a quadratic non-residue z
    ll z = 2;
    while (mod_pow(z, (p - 1) / 2, p) != p - 1) ++z;

    ll c = mod_pow(z, q, p);          // c = z^q mod p
    ll x = mod_pow(a, (q + 1) / 2, p); // x = a^{(q+1)/2} mod p
    ll t = mod_pow(a, q, p);          // t = a^q mod p
    int m = s;

    for (int i = 1; i < m; ++i) {
        ll tp = t;
```

```
        int d = 1;
        while (d < m - i) {
            tp = tp * tp % p;
            d <<= 1;
        }
        if (tp != 1) {
            x = x * mod_pow(c, 1 << (m - i - 1), p) % p;
            c = c * c % p;
            t = t * c % p;
        }
    }

    return x;
}
```

8.45 Simplex

```
/**
 * Simplex algorithm for linear programming.
 * Solves max c*x subject to Ax <= b, x >= 0.
 * Uses two-phase simplex with Bland's rule for guaranteed termination.
 *
 * Usage:
 *   Simplex simplex(n);           // n = number of variables
 *   simplex.set_objective(c);     // c = objective coefficients
 *   simplex.add_constraint(a, b); // Ax <= b
 *   auto [max_val, solution] = simplex.solve();
 */

#include <bits/stdc++.h>
using namespace std;

const double INF = 1e100;
const double EPS = 1e-9;

struct Simplex {
    int n;                          // number of variables
    int m;                          // number of constraints
    vector<vector<double>> A;       // constraint matrix (m constraints, n variables)
    vector<double> b;               // RHS of constraints
    vector<double> c;               // objective coefficients

    Simplex(int n_) : n(n_), m(0), A(), b(), c(n_, 0.0) {}

    void set_objective(const vector<double>& c_) {
        c = c_;
    }

    void add_constraint(const vector<double>& a, double b_) {
        A.push_back(a);
        b.push_back(b_);
        m++;
    }

    pair<double, vector<double>> solve() {
        // Normalize: Ax <= b with b >= 0. If b < 0, multiply constraint by -1.
        for (int i = 0; i < m; i++) {
            if (b[i] < -EPS) {
                for (int j = 0; j < n; j++) A[i][j] = -A[i][j];
                b[i] = -b[i];
            }
        }

        // Two-phase simplex.
        // Phase 1: introduce artificial variables and find initial BFS.
        // Phase 2: solve original problem.

        // Variables: x[0..n-1] original, s[0..m-1] slack, a[0..m-1] artificial
        int N = n + m + m;
        int M = m;

        vector<vector<double>> T(M, vector<double>(N + 1, 0.0));

        for (int i = 0; i < M; i++) {
            for (int j = 0; j < n; j++) T[i][j] = A[i][j];
            T[i][n + i] = 1.0;                                  // slack
            T[i][n + m + i] = 1.0;                              // artificial
            T[i][N] = b[i];
        }

        // Phase 1 objective: minimize sum of artificial variables
        vector<double> obj(N + 1, 0.0);
        for (int i = 0; i < m; i++) obj[n + m + i] = 1.0;

        // Basic variables are artificial vars
        vector<int> basic(M);
        for (int i = 0; i < M; i++) basic[i] = n + m + i;

        // Eliminate artificial vars from objective row
        for (int i = 0; i < M; i++) {
            if (basic[i] >= n + m) {
                for (int j = 0; j <= N; j++) obj[j] -= T[i][j];
            }
        }

        // Run simplex on phase 1
        bool infeasible = !simplex_iterate(T, obj, basic, N, M);
```

```
        // Check feasibility: artificial vars must be zero
        double phase1_val = -obj[N];
        for (int i = 0; i < M; i++) phase1_val -= obj[basic[i]] * T[i][N];

        if (phase1_val > EPS || infeasible) {
            return {-INF, {}};
        }

        // Phase 2: build new tableau without artificial variables
        // Keep current BFS by carrying over basic variables
        int N2 = n + m;
        vector<vector<double>> T2(M, vector<double>(N2 + 1, 0.0));

        for (int i = 0; i < M; i++) {
            for (int j = 0; j < n; j++) T2[i][j] = A[i][j];
            T2[i][n + i] = 1.0;
            T2[i][N2] = b[i];
        }

        // Determine new basic variables
        vector<int> basic2(M, -1);
        for (int i = 0; i < M; i++) {
            if (basic[i] < n + m) {
                basic2[i] = basic[i];
            }
        }

        // Pivot out artificial variables that are still basic
        for (int i = 0; i < M; i++) {
            if (basic2[i] == -1) {
                // Find a non-artificial variable to pivot in
                for (int j = 0; j < N2; j++) {
                    if (abs(T2[i][j]) > EPS) {
                        pivot(T2, basic2, i, j, M, N2);
                        break;
                    }
                }
            }
        }

        // Set up phase 2 objective
        vector<double> obj2(N2 + 1, 0.0);
        for (int j = 0; j < n; j++) obj2[j] = c[j];

        // Eliminate basic variables from objective
        for (int i = 0; i < M; i++) {
            if (basic2[i] != -1 && basic2[i] < N2) {
                double coeff = obj2[basic2[i]];
                if (abs(coeff) > EPS) {
                    for (int j = 0; j <= N2; j++) obj2[j] -= coeff *
                        T2[i][j];
                }
            }
        }

        // Run simplex on phase 2
        bool unbounded = !simplex_iterate(T2, obj2, basic2, N2, M);

        if (unbounded) {
            return {INF, {}};
        }

        // Extract solution
        vector<double> x(n, 0.0);
        for (int i = 0; i < M; i++) {
            if (basic2[i] >= 0 && basic2[i] < n) {
                x[basic2[i]] = T2[i][N2];
            }
        }

        double max_val = -obj2[N2];
        for (int i = 0; i < M; i++) {
            if (basic2[i] >= 0) max_val -= obj2[basic2[i]] * T2[i][N2];
        }

        return {max_val, x};
    }

private:
    void pivot(vector<vector<double>>& T, vector<int>& basic, int row, int col, int m, int n) {
        double piv = T[row][col];
        if (abs(piv) < EPS) return;

        for (int j = 0; j <= n; j++) T[row][j] /= piv;
        for (int i = 0; i < m; i++) {
            if (i != row && abs(T[i][col]) > EPS) {
                double factor = T[i][col];
                for (int j = 0; j <= n; j++) T[i][j] -= factor * T[row][j];
            }
        }
        basic[row] = col;
    }

    bool simplex_iterate(vector<vector<double>>& T, vector<double>& obj, vector<int>& basic, int n, int m) {
        while (true) {
            // Bland's rule: entering variable is smallest index with negative reduced cost
            int entering = -1;
            for (int j = 0; j < n; j++) {
                if (obj[j] < -EPS) {
```

```
                    entering = j;
                    break;
                }
            }
            if (entering == -1) break;

            // Minimum ratio test with Bland's rule for leaving variable
            int leaving = -1;
            double min_ratio = INF;
            for (int i = 0; i < m; i++) {
                if (T[i][entering] > EPS) {
                    double ratio = T[i][n] / T[i][entering];
                    if (ratio < min_ratio - EPS || (abs(ratio - min_ratio) < EPS && basic[i] < basic[leaving])) {
                        min_ratio = ratio;
                        leaving = i;
                    }
                }
            }

            if (leaving == -1) return false; // unbounded

            pivot(T, basic, leaving, entering, m, n);

            double coeff = obj[entering];
            if (abs(coeff) > EPS) {
                for (int j = 0; j <= n; j++) obj[j] -= coeff * T[leaving][j];
            }
        }
        return true;
    }
};
```

9 String Algorithms

9.1 Aho-Corasick

```
// Aho-Corasick Automaton (lowercase a-z)
struct AhoCorasick {
    static const int ALPH = 26;

    struct Node {
        int next[ALPH];
        int link;
        vector<int> out; // pattern ids ending here

        Node() : link(0) {
            fill(next, next + ALPH, -1);
        }
    };

    vector<Node> t;
    vector<int> patLen;

    AhoCorasick() { t.push_back(Node()); }

    int addString(const string& s) {
        int v = 0;
        for (char ch : s) {
            int c = ch - 'a';
            if (t[v].next[c] == -1) {
                t[v].next[c] = (int)t.size();
                t.push_back(Node());
            }
            v = t[v].next[c];
        }
        int id = (int)patLen.size();
        patLen.push_back((int)s.size());
        t[v].out.push_back(id);
        return id;
    }

    void build() {
        queue<int> q;

        for (int c = 0; c < ALPH; c++) {
            if (t[0].next[c] == -1) {
                t[0].next[c] = 0;
            } else {
                int u = t[0].next[c];
                t[u].link = 0;
                q.push(u);
            }
        }

        while (!q.empty()) {
            int v = q.front();
            q.pop();

            for (int c = 0; c < ALPH; c++) {
                int u = t[v].next[c];
                if (u == -1) {
                    t[v].next[c] = t[t[v].link].next[c];
                    continue;
                }

                t[u].link = t[t[v].link].next[c];
                for (int id : t[t[u].link].out) t[u].out.push_back(id);
```

```
                q.push(u);
            }
        }
    }

    // Returns all matches as {ending_index_in_text, pattern_id}
    vector<pair<int, int>> findAll(const string& text) const {
        vector<pair<int, int>> ans;
        int v = 0;
        for (int i = 0; i < (int)text.size(); i++) {
            int c = text[i] - 'a';
            v = t[v].next[c];
            for (int id : t[v].out) {
                ans.push_back({i, id});
            }
        }
        return ans;
    }
};

// Example of use:
// Count occurrences of each pattern in text.
vector<int> countPatternOccurrences(const string& text, const vector
    <string>& patterns) {
    AhoCorasick ac;
    for (const string& p : patterns) ac.addString(p);
    ac.build();

    vector<int> cnt(patterns.size(), 0);
    auto matches = ac.findAll(text);
    for (auto [endIdx, id] : matches) {
        (void)endIdx;
        cnt[id]++;
    }
    return cnt;
}
```

9.2 DP on Trie

```
#ifndef MOD
#define MOD 1000000007
#endif

class Trie{
private:
    Trie *children[26];
    bool term_st; // is_end

    Trie* searchPrefix(const string &p){
        Trie* node = this;

        for(char c:p){
            int pos = c - 'a';
            if(!node->children[pos]) return nullptr;
            node = node->children[pos];
        }
        return node;
    }

public:
    Trie() : term_st(false) {
        for(int i=0; i<26; i++)
            children[i] = nullptr;
    }

    void insert(string word) {
        Trie* node = this;
        for(char c:word){
            int pos = c - 'a';
            if(!node->children[pos]) node->children[pos] = new Trie
                ();
            node = node->children[pos];
        }

        node->term_st = true;
    }

    bool search(string word) {
        Trie* node = searchPrefix(word);
        return node != nullptr && node->term_st;
    }

    bool startsWith(string prefix) {
        Trie* node = searchPrefix(prefix);
        return node != nullptr;
    }

    int arnsDP(string s, int n){
        vector<int> dp(n+1, 0);
        // dp[i] -> ways to form the prefix s[0..i-1]
        dp[0] = 1;

        for(int i = 0; i < n; i++){
            Trie *it = this; // Iterate the Trie from the root
            for(int j = i; j < n; j++){
                int pos = s[j] - 'a';
                if(!it->children[pos]) break;
                it = it->children[pos];

                if(it->term_st)
                    dp[j+1] = (dp[j+1] + dp[i]) % MOD;

            }
```

```
        }
        return dp[n];
    }
};
```

9.3 KMP

```
// Knuth-Morris-Pratt Algorithm
vector<int> computeLPS(string pattern) {
    int m = pattern.length();
    vector<int> lps(m);
    int len = 0;
    lps[0] = 0;

    for (int i = 1; i < m; ) {
        if (pattern[i] == pattern[len]) {
            len++;
            lps[i] = len;
            i++;
        } else {
            if (len != 0) {
                len = lps[len - 1];
            } else {
                lps[i] = 0;
                i++;
            }
        }
    }
    return lps;
}

vector<int> KMP(string text, string pattern) {
    vector<int> lps = computeLPS(pattern);
    vector<int> matches;
    int n = text.length();
    int m = pattern.length();
    int i = 0, j = 0;

    while (i < n) {
        if (text[i] == pattern[j]) {
            i++;
            j++;
        }

        if (j == m) {
            matches.push_back(i - j);
            j = lps[j - 1];
        } else if (i < n && text[i] != pattern[j]) {
            if (j != 0) {
                j = lps[j - 1];
            } else {
                i++;
            }
        }
    }
    return matches;
}
```

9.4 Manacher Algorithm

```
// Odd-length manacher: d1[i] = radius of palindrome centered at i (
    including center)
vector<int> manacher_odd(const string& s) {
    int n = (int)s.size();
    vector<int> d(n);
    int l = 0, r = -1;
    for (int i = 0; i < n; i++) {
        int k = (i > r) ? 1 : min(d[l + r - i], r - i + 1);
        while (i - k >= 0 && i + k < n && s[i - k] == s[i + k]) k++;
        d[i] = k;
        if (i + k - 1 > r) {
            l = i - k + 1;
            r = i + k - 1;
        }
    }
    return d;
}

// Even-length manacher: d2[i] = radius of palindrome centered
    between i-1 and i
vector<int> manacher_even(const string& s) {
    int n = (int)s.size();
    vector<int> d(n);
    int l = 0, r = -1;
    for (int i = 0; i < n; i++) {
        int k = (i > r) ? 0 : min(d[l + r - i + 1], r - i + 1);
        while (i - k - 1 >= 0 && i + k < n && s[i - k - 1] == s[i +
            k]) k++;
        d[i] = k;
        if (i + k - 1 > r) {
            l = i - k;
            r = i + k - 1;
        }
    }
    return d;
}

// Returns array of max palindrome radius at each position.
// Result[i] = max palindrome half-length at position i (odd-length
    uses d1, even-length uses d2).
```

```cpp
vector<int> manacher(string s) {
    string t;
    for (char c : s) {
        t += string("#") + c;
    }
    t += "#";
    vector<int> d1 = manacher_odd(t);
    vector<int> d2 = manacher_even(t);
    int n = (int)s.size();
    vector<int> res(n);
    for (int i = 0; i < n; i++) {
        int pos = 2 * i + 1; // position in transformed string
        int odd = d1[pos] - 1; // radius - 1 = original odd palindrome half-length contribution
        int even = 0;
        if (pos - 1 >= 0) even = d2[pos - 1]; // even palindrome radius at same center
        res[i] = max(odd, even);
    }
    return res;
}
```

9.5 Palindromic Tree

```cpp
// Palindromic Tree (Eertree)
struct PalindromicTree {
    struct Node {
        int len;
        int link;
        int next[26];
        int occ; // occurrences as palindrome suffix while adding characters

        Node(int l = 0) : len(l), link(0), occ(0) {
            fill(next, next + 26, -1);
        }
    };

    vector<Node> tree;
    string s;
    int suff; // node of the longest suffix-palindrome of current string

    PalindromicTree() { init(); }

    void init() {
        tree.clear();
        tree.push_back(Node(-1)); // root 0: odd length root
        tree.push_back(Node(0));  // root 1: even length root
        tree[0].link = 0;
        tree[1].link = 0;
        s.clear();
        suff = 1;
    }

    int getLink(int v, int pos) const {
        while (true) {
            int curLen = tree[v].len;
            if (pos - 1 - curLen >= 0 && s[pos - 1 - curLen] == s[pos]) {
                return v;
            }
            v = tree[v].link;
        }
    }

    void addChar(char ch) {
        int c = ch - 'a';
        s.push_back(ch);
        int pos = (int)s.size() - 1;

        int cur = getLink(suff, pos);

        if (tree[cur].next[c] != -1) {
            suff = tree[cur].next[c];
            tree[suff].occ++;
            return;
        }

        int newNode = (int)tree.size();
        tree.push_back(Node(tree[cur].len + 2));
        tree[cur].next[c] = newNode;

        if (tree[newNode].len == 1) {
            tree[newNode].link = 1;
        } else {
            int linkCand = getLink(tree[cur].link, pos);
            tree[newNode].link = tree[linkCand].next[c];
        }

        suff = newNode;
        tree[suff].occ++;
    }

    void build(const string& str) {
        init();
        for (char ch : str) addChar(ch);
    }

    // Number of distinct palindromic substrings.
    int distinctPalindromes() const {
        return max(0, (int)tree.size() - 2);
    }
```

```
};

// Example of use:
int countDistinctPalindromes(const string& s) {
    PalindromicTree pt;
    pt.build(s);
    return pt.distinctPalindromes();
}
```

9.6 Polynomial Hash

```
#define ll long long
#define all(x) x.begin(), x.end()

mt19937 rng((uint32_t) chrono::steady_clock::now().time_since_epoch
    ().count());
const int MOD = 1e9 + 9;
const ll BASE = uniform_int_distribution<ll>(0, MOD-1)(rng);

vector<ll> pows = {1};
vector<ll> p_hash;

void hash_str(const string &s){
    p_hash.assign(s.size()+1, 0);

    while(pows.size() <= s.size()){
  pows.push_back((pows.back()* BASE)%MOD);
    }

    p_hash[0] = 0;
    for(int i=0; i<s.size(); i++){
  p_hash[i+1] = ((p_hash[i]* BASE)% MOD + s[i])%MOD;
    }
}

ll get_hash(int l, int r){
    ll raw = (p_hash[r+1] - (p_hash[l] * pows[r-l+1]));
    return (raw % MOD  + MOD) % MOD;
}

inline void solve(){
    string s; cin>>s;
    int n = s.size();

    hash_str(s);

    for(int i=0; i<n-1; i++){
  if(get_hash(0,i) == get_hash(n-i-1, n-1)){
      cout<<i+1<<" ";
  }
    }
    cout<<endl;
}
```

9.7 Rabin-Karp

```
// Rabin-Karp (single pattern) with double mod hash.
// Finds all occurrences of pattern in text in O(n + m).

using ll = long long;

const ll MOD1 = 1000000007LL;
const ll MOD2 = 1000000009LL;
const ll BASE = 911382323LL;

vector<int> rabinKarp(const string &text, const string &pattern) {
    int n = (int)text.size();
    int m = (int)pattern.size();
    vector<int> arns;

    if (m == 0 || m > n) return arns;

    vector<ll> p1(n + 1, 1), p2(n + 1, 1);
    vector<ll> h1(n + 1, 0), h2(n + 1, 0);

    for (int i = 1; i <= n; i++) {
        p1[i] = (p1[i - 1] * BASE) % MOD1;
        p2[i] = (p2[i - 1] * BASE) % MOD2;

        h1[i] = (h1[i - 1] * BASE + (unsigned char)text[i - 1] + 1)
            % MOD1;
        h2[i] = (h2[i - 1] * BASE + (unsigned char)text[i - 1] + 1)
            % MOD2;
    }

    ll ph1 = 0, ph2 = 0;
    for (char c : pattern) {
        ph1 = (ph1 * BASE + (unsigned char)c + 1) % MOD1;
        ph2 = (ph2 * BASE + (unsigned char)c + 1) % MOD2;
    }

    auto getHash = [&](int l, int r) {
        ll x1 = (h1[r + 1] - (h1[l] * p1[r - l + 1]) % MOD1 + MOD1)
            % MOD1;
        ll x2 = (h2[r + 1] - (h2[l] * p2[r - l + 1]) % MOD2 + MOD2)
            % MOD2;
        return pair<ll, ll>{x1, x2};
    };

    for (int i = 0; i + m - 1 < n; i++) {
```

```
        if (getHash(i, i + m - 1) == pair<ll, ll>{ph1, ph2}) {
            arns.push_back(i);
        }
    }

    return arns;
}

inline void solve() {
    string text, pattern;
    cin >> text >> pattern;

    vector<int> pos = rabinKarp(text, pattern);
    cout << pos.size() << endl;
    for (int i : pos) cout << i << " ";
    cout << endl;
}
```

9.8 Suffix Array + LCP

```
// Suffix Array + LCP (Kasai)
struct SuffixArray {
    string s;
    vector<int> sa;    // Suffix array: indices of suffixes in
        sorted order
    vector<int> rank;  // rank[i] = position of suffix i in sa
    vector<int> lcp;   // lcp[i] = LCP(sa[i], sa[i-1]), lcp[0] = 0

    SuffixArray() {}
    explicit SuffixArray(const string& str) { build(str); }

    void build(const string& str) {
        s = str;
        int n = (int)s.size();
        sa.resize(n);
        rank.resize(n);

        for (int i = 0; i < n; i++) {
            sa[i] = i;
            rank[i] = s[i];
        }

        vector<int> tmp(n);
        for (int k = 1;; k <<= 1) {
            auto cmp = [&](int i, int j) {
                if (rank[i] != rank[j]) return rank[i] < rank[j];
                int ri = (i + k < n) ? rank[i + k] : -1;
                int rj = (j + k < n) ? rank[j + k] : -1;
                return ri < rj;
            };

            sort(sa.begin(), sa.end(), cmp);
            tmp[sa[0]] = 0;

            for (int i = 1; i < n; i++) {
                tmp[sa[i]] = tmp[sa[i - 1]] + (cmp(sa[i - 1], sa[i])
                    ? 1 : 0);
            }

            rank = tmp;
            if (rank[sa[n - 1]] == n - 1) break;
        }

        buildLCP();
    }

    void buildLCP() {
        int n = (int)s.size();
        lcp.assign(n, 0);
        for (int i = 0; i < n; i++) rank[sa[i]] = i;

        int k = 0;
        for (int i = 0; i < n; i++) {
            if (rank[i] == 0) {
                k = 0;
                continue;
            }
            int j = sa[rank[i] - 1];
            while (i + k < n && j + k < n && s[i + k] == s[j + k]) k
                ++;
            lcp[rank[i]] = k;
            if (k) k--;
        }
    }
};

// Example of use:
// Find the longest repeated substring in s.
string longestRepeatedSubstring(const string& s) {
    if (s.empty()) return "";
    SuffixArray sa(s);

    int bestLen = 0;
    int bestPos = 0;
    for (int i = 1; i < (int)sa.lcp.size(); i++) {
        if (sa.lcp[i] > bestLen) {
            bestLen = sa.lcp[i];
            bestPos = sa.sa[i];
        }
    }
    return s.substr(bestPos, bestLen);
}
```

9.9 Suffix Automaton

```cpp
// Suffix Automaton
struct SuffixAutomaton {
    struct State {
        int link;
        int len;
        int next[26];

        State() : link(-1), len(0) {
            fill(next, next + 26, -1);
        }
    };

    vector<State> st;
    int last;

    SuffixAutomaton() {
        st.reserve(1);
        st.push_back(State());
        last = 0;
    }

    void extend(char ch) {
        int c = ch - 'a';

        int cur = (int)st.size();
        st.push_back(State());
        st[cur].len = st[last].len + 1;

        int p = last;
        while (p != -1 && st[p].next[c] == -1) {
            st[p].next[c] = cur;
            p = st[p].link;
        }

        if (p == -1) {
            st[cur].link = 0;
        } else {
            int q = st[p].next[c];
            if (st[p].len + 1 == st[q].len) {
                st[cur].link = q;
            } else {
                int clone = (int)st.size();
                st.push_back(st[q]);
                st[clone].len = st[p].len + 1;

                while (p != -1 && st[p].next[c] == q) {
                    st[p].next[c] = clone;
                    p = st[p].link;
                }

                st[q].link = st[cur].link = clone;
            }
        }

        last = cur;
    }

    void build(const string& s) {
        st.clear();
        st.push_back(State());
        last = 0;
        for (char ch : s) extend(ch);
    }

    // Number of distinct substrings of the built string.
    long long countDistinctSubstrings() const {
        long long ans = 0;
        for (int v = 1; v < (int)st.size(); v++) {
            ans += st[v].len - st[st[v].link].len;
        }
        return ans;
    }

    bool contains(const string& t) const {
        int v = 0;
        for (char ch : t) {
            int c = ch - 'a';
            if (st[v].next[c] == -1) return false;
            v = st[v].next[c];
        }
        return true;
    }
};

// Example of use:
long long distinctSubstringsIn(const string& s) {
    SuffixAutomaton sam;
    sam.build(s);
    return sam.countDistinctSubstrings();
}
```

9.10 Suffix Automata 2

```cpp
struct state {
    int len, link;
    map<char, int> next;
};

const int MAXLEN = 100000;
state st[MAXLEN * 2];
```

```
int sz, last;

void sa_init() {
    st[0].len = 0;
    st[0].link = -1;
    sz++;
    last = 0;
}

void sa_extend(char c) {
    int cur = sz++;
    st[cur].len = st[last].len + 1;
    int p = last;
    while (p != -1 && !st[p].next.count(c)) {
        st[p].next[c] = cur;
        p = st[p].link;
    }
    if (p == -1) {
        st[cur].link = 0;
    } else {
        int q = st[p].next[c];
        if (st[p].len + 1 == st[q].len) {
            st[cur].link = q;
        } else {
            int clone = sz++;
            st[clone].len = st[p].len + 1;
            st[clone].next = st[q].next;
            st[clone].link = st[q].link;
            while (p != -1 && st[p].next[c] == q) {
                st[p].next[c] = clone;
                p = st[p].link;
            }
            st[q].link = st[cur].link = clone;
        }
    }
    last = cur;
}

long long get_diff_strings(){
    long long tot = 0;
    for(int i = 1; i < sz; i++) {
        tot += st[i].len - st[st[i].link].len;
    }
    return tot;
}

long long get_tot_len_diff_substings() {
    long long tot = 0;
    for(int i = 1; i < sz; i++) {
        long long shortest = st[st[i].link].len + 1;
        long long longest = st[i].len;

        long long num_strings = longest - shortest + 1;
        long long cur = num_strings * (longest + shortest) / 2;
        tot += cur;
    }
    return tot;
}

string lcs (string S, string T) {
    sa_init();
    for (int i = 0; i < S.size(); i++)
        sa_extend(S[i]);

    int v = 0, l = 0, best = 0, bestpos = 0;
    for (int i = 0; i < T.size(); i++) {
        while (v && !st[v].next.count(T[i])) {
            v = st[v].link ;
            l = st[v].len;
        }
        if (st[v].next.count(T[i])) {
            v = st [v].next[T[i]];
            l++;
        }
        if (l > best) {
            best = l;
            bestpos = i;
        }
    }
    return T.substr(bestpos - best + 1, best);
}
```

9.11 String Stream

```
// Createa a stringstream object
stringstream ss(fullString);
// Extract the strings
ss >> a >> colon >> b >> comma >> c;
```

9.12 XOR Hash

```
// XOR Hash for strings (very fast, collision-prone compared to
    polynomial hash).
// Good for randomized checks, grouping, and quick fingerprints.

using ull = unsigned long long;

mt19937_64 rng_xor((uint64_t)chrono::steady_clock::now().
    time_since_epoch().count());
array<ull, 256> H;
```

```

void initXorHash() {
    for (int i = 0; i < 256; i++) H[i] = rng_xor();
}

// Order-sensitive XOR hash with position mixing.
ull xorHash(const string &s) {
    ull arns = 0;
    for (int i = 0; i < (int)s.size(); i++) {
        ull x = H[(unsigned char)s[i]];
        arns ^= (x + 0x9e3779b97f4a7c15ULL + (arns << 6) + (arns >>
            2) + (ull)(i + 1));
    }
    return arns;
}

inline void solve() {
    initXorHash();

    int n; cin >> n;
    unordered_map<ull, int> freq;

    for (int i = 0; i < n; i++) {
        string s; cin >> s;
        freq[xorHash(s)]++;
    }

    // Number of distinct hashes (not guaranteed distinct strings
        due to collisions).
    cout << (int)freq.size() << endl;
}
```

9.13 Z Algorithm

```
// Z-Algorithm
vector<int> z_function(string s) {
    int n = s.length();
    vector<int> z(n);
    int l = 0, r = 0;

    for (int i = 1; i < n; i++) {
        if (i <= r) {
            z[i] = min(r - i + 1, z[i - l]);
        }
        while (i + z[i] < n && s[z[i]] == s[i + z[i]]) {
            z[i]++;
        }
        if (i + z[i] - 1 > r) {
            l = i;
            r = i + z[i] - 1;
        }
    }
    return z;
}

// Pattern matching using Z-algorithm
vector<int> patternMatch(string text, string pattern) {
    string combined = pattern + "$" + text;
    vector<int> z = z_function(combined);
    vector<int> matches;
    int m = pattern.length();

    for (int i = m + 1; i < combined.length(); i++) {
        if (z[i] == m) {
            matches.push_back(i - m - 1);
        }
    }
    return matches;
}
```

10 Computational Geometry

10.1 Convex Hull

```
const ll mod=1e9+7;
const ll limit=4e9;
//ascii https://elcodigoascii.com.ar/

int orientation(point a,point b,point c){
    ll ori=(b.y-c.y)*(b.x-a.x)-(b.y-a.y)*(b.x-c.x);
    if(ori==0) return 0;
    if(ori>0) return 1;
    return 2;
}

void getLastTwo(point &a,point &b,stack<point> &s)
{
    a=s.top();
    s.pop();
    b=s.top();
    s.pop();
}

void show(point a){
    cout<<a.x<<" "<<a.y<<endl;
}

//Graham scan

```

```
void solve(){
    int n; cin>>n;
    vector<point> puntos(n);
    FO(i,n){
        ll a,b; cin>>a>>b;
        puntos[i]=make_pair(a,b);
    }
    sort(all(puntos));
    //Lower Part
    stack<point> lower;
    FO(i,n)
    {
        if(lower.size()<2){
            lower.push(puntos[i]);
            continue;
        }
        point a,b;
        getLastTwo(a,b,lower);
        if(orientation(a,b,puntos[i])<2)
        {
            lower.push(b);
            lower.push(a);
            lower.push(puntos[i]);
        }
        else{
            lower.push(b);
            i--;
        }
    }
    stack<point> upper;
    for(int i=n-1;i>=0;i--)
    {
        if(upper.size()<2){
            upper.push(puntos[i]);
            continue;
        }
        point a,b;
        getLastTwo(a,b,upper);
        if(orientation(a,b,puntos[i])<2)
        {
            upper.push(b);
            upper.push(a);
            upper.push(puntos[i]);
        }
        else{
            upper.push(b);
            i++;
        }
    }

    set<point> res;

    while(!lower.empty()){
        res.insert(lower.top());
        lower.pop();
    }
    while(!upper.empty()){
        res.insert(upper.top());
        upper.pop();
    }
    cout<<res.size()<<endl;
    for(auto c:res) show(c);
}
```

10.2 Complex Point

```
typedef long long ll;
typedef vector<int> vec;
const ll mod=1e9+7;
const int MAX=2e5+3;

//ascii https://elcodigoascii.com.ar/

T norma(pt a){return a.x*a.x+a.y*a.y;}


int sgn(T X){
    return (T(0)<X)-(T(0)>X);
}


pt translate(pt a,pt v){return a+v;}
pt scale(pt p,pt c,T factor){return c+(p-c)*factor;}
pt rot(pt p,T a){return p*polar(1.0,a);}
pt perp(pt p){return pt({-p.y,p.x});}
pt linearFunc(pt p,pt q,pt r,pt fp,pt fq){
    return fp+(r-p)*(fq-fp)/(q-p);
}
T dot(pt v,pt w){ return v.x*w.x+v.y*w.y;}
T cross(pt v,pt w){ return v.x*w.y-v.y*w.x;}

bool isperp(pt a,pt b){return dot(a,b)==0;}

double angle(pt v,pt w){
    return acos(clamp(dot(v,w)/abs(v)/abs(w),-1.0,-1.0));
}

T orientation(pt a,pt b,pt c){return cross(b-a,c-a);}

bool inAngle(pt a,pt b,pt c,pt p){
    if(orientation(a,b,c)<0) swap(b,c);
    return sgn(orientation(a,b,p))*sgn(orientation(a,c,p))<=0;
}
```

```

bool isconvex(vector<pt> p){
    bool hasPos=false,hasNeg=false;
    for(int i=0,n=p.size();i<n;i++){
        int o=orientation(p[i],p[(i+1)%n],p[(i+2)%n]);
        if(o>0) hasPos=true;
        if(o<0) hasNeg=true;
    }
    return !(hasPos && hasNeg);
}

inline void solve()
{
    pt p{3,-4};
    p+=pt({1,2});
    cout<<p<<endl;
    cout<<norma(p)<<endl;
}
```

10.3 Line Segment Intersection

```
\begin{lstlisting}[style=compactcpp]
// BeginCodeSnip{Point Class}
struct Point {
  int x, y;
  Point(int a = 0, int b = 0) : x(a), y(b) {}

  friend istream &operator>>(istream &in, Point &p) {
    int x, y;
    in >> p.x >> p.y;
    return in;
  }
};
// EndCodeSnip

int sign(long long num) {
  if (num < 0) {
    return -1;
  } else if (num == 0) {
    return 0;
  } else {
    return 1;
  }
}

long long trigonometric_sense(Point p, Point p1, Point p2) {
  return sign(1LL * (p1.x - p.x) * (p2.y - p.y) -
              1LL * (p2.x - p.x) * (p1.y - p.y));
}

```

```
// Check if the rectangles with [P1, P2] and [P3, P4] as diagonals
    intersect
bool quick_check(Point p1, Point p2, Point p3, Point p4) {
  int x1, x2, x3, x4, y1, y2, y3, y4;
  x1 = min(p1.x, p2.x), x2 = max(p1.x, p2.x);
  y1 = min(p1.y, p2.y), y2 = max(p1.y, p2.y);
  x3 = min(p3.x, p4.x), x4 = max(p3.x, p4.x);
  y3 = min(p3.y, p4.y), y4 = max(p3.y, p4.y);
  return x2 < x3 || x4 < x1 || y2 < y3 || y4 < y1;
}

bool check(Point p1, Point p2, Point p3, Point p4) {
  if (trigonometric_sense(p1, p2, p3) * trigonometric_sense(p1, p2,
      p4) > 0) {
    return false;
  }
  if (trigonometric_sense(p3, p4, p1) * trigonometric_sense(p3, p4,
      p2) > 0) {
    return false;
  }
  // Collinear case: check if projections on x-axis overlap properly
  // (not just touch at a single endpoint)
  // First check if all 4 points are collinear by verifying p3 and
      p4 lie on line p1-p2
  if (trigonometric_sense(p1, p2, p3) == 0 && trigonometric_sense(p1
      , p2, p4) == 0) {
    if (max(p1.x, p2.x) < min(p3.x, p4.x) ||
        max(p3.x, p4.x) < min(p1.x, p2.x))
      return false;
  }
  return true;
}

int main() {
  int test_num;
  cin >> test_num;
  for (int t = 0; t < test_num; t++) {
    Point p1, p2, p3, p4;
    cin >> p1 >> p2 >> p3 >> p4;

    if (quick_check(p1, p2, p3, p4)) {
      cout<<"NO"<<endl;
    } else if (check(p1, p2, p3, p4)) {
      cout<<"YES"<<endl;
    } else {
      cout<<"NO"<<endl;
    }
  }
}
```

10.4 Minimum Euclidean Distance

```
const ll mod=1e9+7;
const ll MAX=8e18;
const ll limit=1e9+1;
//ascii https://elcodigoascii.com.ar/

ll distance(point a,point b){
    return (a.X-b.X)*(a.X-b.X)+(a.Y-b.Y)*(a.Y-b.Y);
}

inline void solve()
{
    int n; cin>>n;
    vector<point> sortedX(n);
    set<point> sortedY;
    FO(i,n)
    {
        ll x,y; cin>>x>>y;
        sortedX[i]=make_pair(x,y);
    }
    sort(all(sortedX));
    sortedY.insert(make_pair(sortedX[0].Y,sortedX[0].X));
    ll d,minSquare=MAX;
    int j=0;
    FOR(i,1,n)
    {
        d=ceil(sqrt(minSquare));
        while(sortedX[i].X-sortedX[j].X>d)
        {
            sortedY.erase(make_pair(sortedX[j].Y,sortedX[j].X));
            j++;
        }
        auto lower=sortedY.lower_bound(make_pair(sortedX[i].Y-d,0));
        auto upper=sortedY.upper_bound(make_pair(sortedX[i].Y+d,0));
        for(auto pointer=lower;pointer!=upper;pointer++)
        {
            minSquare=min(minSquare,distance(*pointer,make_pair(
                sortedX[i].Y,sortedX[i].X)));
        }
        sortedY.insert(make_pair(sortedX[i].Y,sortedX[i].X));
    }
    cout<<minSquare<<endl;
}
```

10.5 Minimum Manhattan Distance

```
pt rot(pt a)
{
    return {a.x+a.y,a.y-a.x};
}

ll max_coord(ll a,set<ll>& v)
{
    ll min_val=*v.begin();
    ll max_val=*std::prev(v.end());
    return max(abs(a-min_val),abs(a-max_val));
}



inline void solve()
{
    int n; cin>>n;
    vector<pt> points(n);
    set<ll> x_;
    set<ll> y_;
    ll res=0;
    cout<<res<<endl;
    ll a,b; cin>>a>>b;
    pt aux=rot({a,b});
    x_.insert(aux.x);
    y_.insert(aux.y);
    FO(i,n-1){
        cin>>a>>b;
        aux=rot({a,b});
        res=max({res,max_coord(aux.x,x_),max_coord(aux.y,y_)});
        cout<<res<<endl;
        x_.insert(aux.x);
        y_.insert(aux.y);

    }
}
```

10.6 Max Nearest Distance

```
void update(int pos,ll u)
{
    pos+=TOP;
    t[pos]=min(u,t[pos]);
    for(int i=pos/2;i;i/=2){
        t[i]=min(t[2*i],t[2*i+1]);
    }

}

ll query(int a,int b){
    ll res=INF;
    for(a+=TOP,b+=TOP;a<=b;a/=2,b/=2){
        if(a&1){
```

```
            res=min(res,t[a++]);
        }
        if(!(b&1)){
            res=min(res,t[b--]);
        }
    }
    return res;
}

ll nearest(vector<pt> &reserved,vector<pt> &freecamp,vector<ll> &
    dist)
{
    vector<tuple<ll,int,ll,int>> events;
    for(int i=0;i<reserved.size();i++){
        pt e=reserved[i];
        events.push_back({e.first,1,e.second,i});
    }
    for(int i=0;i<freecamp.size();i++){
        pt e=freecamp[i];
        events.push_back({e.first,0,e.second,i});
    }
    sort(all(events));
    reverse(all(events));
    fill(t,t+2*MAX,INF);
    ll res=INF;
    for(auto [x,cs,y,id]:events){
        //cout<<x<<" "<<cs<<" "<<y<<endl;
        if(cs==1){
            update(y,x+y);
        }
        else{
            // res=min(res,query(y,TOP-1)-(x+y));
            dist[id]=min(dist[id],query(y,TOP-1)-(x+y));
            //cout<<res<<endl;
        }
    }
    if(res>TOP) return 0;
    //cout<<res<<endl;
    return res;
}

vector<ll> dist(m,INF);
for(int i=0;i<(1<<2);i++)
{
    vector<pt> rot_reserved(n),rot_freecamp(m);
    rot_reserved=reserved;
    rot_freecamp=freecamp;
    FO(j,n){
        if(i&1){
            rot_reserved[j].first=TOP-reserved[j].first;
        }
        if(i&2){
            rot_reserved[j].second=TOP-reserved[j].second;
        }
    }
    FO(j,m){
        if(i&1){
            rot_freecamp[j].first=TOP-freecamp[j].first;
        }
        if(i&2){
            rot_freecamp[j].second=TOP-freecamp[j].second;
        }
    }
    nearest(rot_reserved,rot_freecamp,dist);
}
```

10.7 Point In Polygon

```
\begin{lstlisting}[style=compactcpp]
struct point{
    ll x,y;
    void show(){
        cout<<x<<" "<<y<<endl;
    }
};

int sign(ll a){
    if(a<0) return -1;
    if(a==0) return 0;
    if(a>0) return 1;
}

int signCP(point p,point p1,point p2)
{
    return sign(1LL*((p1.x-p.x)*(p2.y-p.y)-(p1.y-p.y)*(p2.x-p.x)));
}

bool intersect(point n, point m,point a,point b)
{
    if(signCP(n,a,b)*signCP(m,a,b)>0) return false;
    if(signCP(a,n,m)*signCP(b,n,m)>0) return false;
    return true;
}

bool inside(point a,point b,point c){
    return a.x>=min(b.x,c.x) && a.x<=max(b.x,c.x) && a.y>=min(b.y,c.
        y)
    && a.y<=max(b.y,c.y);
}

inline void solve()
{
    int n,m; cin>>n>>m;
```

```
    vector<point> vertices(n);
    FO(i,n)
    {
        cin>>vertices[i].x>>vertices[i].y;
    }
    point query,par,init,first,second;
    int counter;
    int resta=0;
    FO(i,m)
    {
        resta=0;
        counter=0;
        cin>>query.x>>query.y;
        par.x=query.x;
        par.y=-MAX-1;
        init.x=vertices[0].x;
        init.y=vertices[0].y;
        first.x=init.x;
        first.y=init.y;
        bool ver=false;
        for(int j=1;j<=n;j++)
        {
            second.x=vertices[j%n].x;
            second.y=vertices[j%n].y;
            point AB,u;
            AB.x=second.x-first.x;
            AB.y=second.y-first.y;
            u.x=second.x-query.x;
            u.y=second.y-query.y;
            if((AB.x*u.y-AB.y*u.x)==0 && inside(query,first,second))
                {
                cout<<"BOUNDARY"<<endl;
                ver=true;
                break;
            }
            if(intersect(query,par,first,second) && first.x<=query.x
                 && query.x<second.x)
            {
                counter++;
            }
            if(intersect(query,par,first,second) && second.x<=query.
                x && query.x<first.x){
                counter++;
            }
            first.x=second.x;
            first.y=second.y;
        }
        point AB,u;
        AB.x=init.x-first.x;
        AB.y=init.y-first.y;
        u.x=init.x-query.x;
        u.y=init.y-query.y;
        if(!ver){
            //if(intersect(query,par,first,init)) counter++;
            if((counter)&1) cout<<"INSIDE";
            else cout<<"OUTSIDE";
            cout<<endl;
        }
    }
}
```

10.8 Point Location Test

```
struct point{
    double x,y;
};

struct Vector{
    double a=0,b=0;
    void getVector(point p1,point p2){
        a=p2.x-p1.x;
        b=p2.y-p1.y;
    }

    double getModulo(){
        return pow(a*a+b*b,0.5);
    }

    Vector getUnitarian(){
        Vector x;
        x.a=a/getModulo();
        x.b=b/getModulo();
        //cout<<x.a<<" "<<x.b<<endl;
        return x;
    }

};

double dotProduct(Vector x,Vector y)
{
    return x.a*y.a+x.b*y.b;
}

double CrossProduct(Vector x,Vector y)
{
    return x.a*y.b-x.b*y.a;
}

inline void solve()
{

    point p1,p2,p3,p4;
    cin>>p1.x>>p1.y>>p2.x>>p2.y>>p3.x>>p3.y;
```

```
    Vector u,v,t;
    u.getVector(p1,p3);
    //cout<<u.a<<" "<<u.b<<endl;
    v.getVector(p2,p3);
    if(CrossProduct(u,v)>0) cout<<"LEFT"<<endl;
    else if(CrossProduct(u,v)<0) cout<<"RIGHT"<<endl;
    else cout<<"TOUCH"<<endl;

}
```

10.9 Polar Sort

```
#define x real()
#define y imag()

typedef long long ll;
typedef double T;
typedef complex<T> pt;
typedef vector<int> vec;
const ll mod=1e9+7;
const int MAX=2e5+3;

T cross(pt v,pt w){ return v.x*w.y-v.y*w.x;}
T norma(pt a){return a.x*a.x+a.y*a.y;}
//ascii https://elcodigoascii.com.ar/

bool half(pt p){
    assert(p.x!=0 || p.y!=0);
    return p.y>0 || (p.y==0 && p.x<0);
}

void polarSort(vector<pt> &v){
    sort(all(v),[](pt v,pt w){
        return make_tuple(half(v),0)<make_tuple(half(w),cross(v,w));
    });
}

void polarSortNorm(vector<pt> &v){
    sort(all(v),[](pt v,pt w){
        return make_tuple(half(v),0,norma(v))<make_tuple(half(w),
            cross(v,w),norma(w));
    });
}
```

10.10 Polygon Area

```
struct point{
    ll x,y;
};

ll CrossP(point a,point b){
    return a.x*b.y-a.y*b.x;
}

inline void solve()
{
    int n; cin>>n;
    ll res=0;
    point p1,p2,p3;
    cin>>p3.x>>p3.y;
    p1.x=p3.x;
    p1.y=p3.y;
    FO(i,n-1)
    {
        cin>>p2.x>>p2.y;
        res+=CrossP(p1,p2);
        p1.x=p2.x;
        p1.y=p2.y;
    }
    res+=CrossP(p1,p3);
    cout<<abs(res)<<endl;
}
```

10.11 Closest Pair of Points

```
#include <utility>
#include <vector>
#include <algorithm>
using namespace std;

typedef long long ll;
const ll MAX = 8e18;

inline ll dist(const pair<ll,ll>& a, const pair<ll,ll>& b){
    ll dx = a.first - b.first;
    ll dy = a.second - b.second;
    return dx*dx + dy*dy;
}

pair<pair<ll,ll>, pair<ll,ll>> bruteForce(const vector<pair<ll,ll>>&
     pts, int left, int right){
    ll bestd = MAX;
    pair<pair<ll,ll>, pair<ll,ll>> best = {pts[left], pts[left+1]};
    for(int i = left; i < right; i++){
        for(int j = i + 1; j < right; j++){
            ll d = dist(pts[i], pts[j]);
            if(d < bestd){
```

```
                bestd = d;
                best = {pts[i], pts[j]};
            }
        }
    }
    return best;
}

struct cmpY {
    bool operator()(const pair<ll,ll>& a, const pair<ll,ll>& b)
        const {
        return a.second < b.second;
    }
};

pair<pair<ll,ll>, pair<ll,ll>> closestPairRecursive(const vector<
    pair<ll,ll>>& pts, int left, int right){
    if(right - left <= 3){
        return bruteForce(pts, left, right);
    }
    int mid = left + (right - left) / 2;
    ll midX = pts[mid].first;

    auto leftPair = closestPairRecursive(pts, left, mid);
    auto rightPair = closestPairRecursive(pts, mid, right);
    ll d = min(dist(leftPair.first, leftPair.second), dist(rightPair
        .first, rightPair.second));
    pair<pair<ll,ll>, pair<ll,ll>> best = leftPair;
    if(dist(rightPair.first, rightPair.second) < dist(best.first,
        best.second)){
        best = rightPair;
    }

    vector<pair<ll,ll>> strip;
    strip.reserve(right - left);
    for(int i = left; i < right; i++){
        if(abs(pts[i].first - midX) < d){
            strip.push_back(pts[i]);
        }
    }
    sort(strip.begin(), strip.end(), cmpY());

    for(size_t i = 0; i < strip.size(); i++){
        for(size_t j = i + 1; j < strip.size() && (strip[j].second -
             strip[i].second) < d; j++){
            ll d2 = dist(strip[i], strip[j]);
            if(d2 < d){
                d = d2;
                best = {strip[i], strip[j]};
            }
        }
    }
    return best;
}

pair<pair<ll,ll>, pair<ll,ll>> closest_pair(vector<pair<ll,ll>>& pts
    ){
    sort(pts.begin(), pts.end());
    return closestPairRecursive(pts, 0, (int)pts.size());
}
```

10.12 Half-Plane Intersection

```
#include <bits/stdc++.h>
using namespace std;

typedef long long ll;
const long double EPS = 1e-12;

struct Point {
    ll x, y;
    Point() {}
    Point(ll _x, ll _y) : x(_x), y(_y) {}
    bool operator==(const Point& o) const { return x == o.x && y ==
        o.y; }
    bool operator!=(const Point& o) const { return !(*this == o); }
};

Point operator+(const Point& a, const Point& b) { return Point(a.x +
     b.x, a.y + b.y); }
Point operator-(const Point& a, const Point& b) { return Point(a.x -
     b.x, a.y - b.y); }
Point operator*(const Point& a, ll k) { return Point(a.x * k, a.y *
    k); }

ll cross(const Point& a, const Point& b) { return a.x * b.y - a.y *
    b.x; }
ll dot(const Point& a, const Point& b) { return a.x * b.x + a.y * b.
    y; }

// Half-plane: the left side of directed line (p, p+v) is the valid
    region
struct HalfPlane {
    Point p;      // point on the line
    Point v;      // direction vector
    long double a; // angle for sorting

    HalfPlane() {}
    HalfPlane(const Point& _p, const Point& _v) : p(_p), v(_v) {
        a = atan2((long double)v.y, (long double)v.x);
    }

    // Positive if q is in the half-plane (left of line)
```

```
    ll eval(const Point& q) const {
        return cross(v, q - p);
    }

    bool contains(const Point& q) const {
        return eval(q) >= 0;
    }
};

// Angle comparison for sorting
bool hpAngleCmp(const HalfPlane& hp1, const HalfPlane& hp2) {
    return hp1.a < hp2.a;
}

// Check if two half-planes are parallel
bool parallel(const HalfPlane& hp1, const HalfPlane& hp2) {
    return cross(hp1.v, hp2.v) == 0;
}

// Line intersection: p1 + t*v1 = p2 + s*v2
Point lineIntersect(const Point& p1, const Point& v1, const Point&
    p2, const Point& v2) {
    Point u = p2 - p1;
    ll d = cross(v1, v2);
    if (d == 0) return Point(0, 0);  // parallel - caller should
        handle
    long double t = (long double)cross(u, v2) / (long double)d;
    return Point(p1.x + (ll)(v1.x * t), p1.y + (ll)(v1.y * t));
}

// Intersection of two half-plane boundary lines
Point hpIntersect(const HalfPlane& hp1, const HalfPlane& hp2) {
    return lineIntersect(hp1.p, hp1.v, hp2.p, hp2.v);
}

// S&I Algorithm: O(N log N) half-plane intersection
vector<Point> half_plane_intersection(vector<HalfPlane>& hps) {
    if (hps.empty()) return {};

    // Sort by angle
    sort(hps.begin(), hps.end(), hpAngleCmp);

    // Remove half-planes with duplicate angles (keep the more
        restrictive one)
    vector<HalfPlane> filtered;
    for (size_t i = 0; i < hps.size(); i++) {
        if (i + 1 < hps.size() && fabs(hps[i].a - hps[i + 1].a) <
            EPS) {
            // Same angle: keep the one that contains the other's
                point
            if (hps[i].contains(hps[i + 1].p)) {
                continue;  // hps[i+1] is redundant
            }
        }
        filtered.push_back(hps[i]);
    }
    hps.swap(filtered);

    if (hps.empty()) return {};

    // Deque for the algorithm
    deque<HalfPlane> dq;

    for (const auto& hp : hps) {
        // Skip if parallel and on same side as last
        if (!dq.empty() && parallel(dq.back(), hp)) {
            if (dq.back().contains(hp.p)) {
                continue;  // hp is redundant
            } else {
                dq.pop_back();  // hp is more restrictive
            }
        }

        // Remove from back while intersection with new hp is
            outside
        while (dq.size() >= 2) {
            Point inter = hpIntersect(dq[dq.size() - 2], dq.back());
            if (hp.contains(inter)) break;
            dq.pop_back();
        }

        // Remove from front while intersection with new hp is
            outside
        while (dq.size() >= 2) {
            Point inter = hpIntersect(dq[0], dq[1]);
            if (hp.contains(inter)) break;
            dq.pop_front();
        }

        dq.push_back(hp);
    }

    // Final cleanup: remove last half-plane if it becomes redundant
    while (dq.size() > 2) {
        Point inter = hpIntersect(dq[0], dq[dq.size() - 2]);
        if (dq.back().contains(inter)) {
            dq.pop_back();
        } else {
            break;
        }
    }

    // Remove first if redundant
    while (dq.size() > 2) {
        Point inter = hpIntersect(dq[dq.size() - 1], dq[1]);
        if (dq[0].contains(inter)) {
```

```
            dq.pop_front();
        } else {
            break;
        }
    }

    if (dq.size() < 3) return {};  // Empty or unbounded

    // Extract polygon vertices
    vector<Point> result;
    for (size_t i = 0; i < dq.size(); i++) {
        size_t j = (i + 1) % dq.size();
        result.push_back(hpIntersect(dq[i], dq[j]));
    }

    return result;
}

// Legacy compatibility
struct point {
    ll x, y;
    point() {}
    point(ll _x, ll _y) : x(_x), y(_y) {}
    explicit operator Point() const { return Point(x, y); }
};
```

10.13 Smallest Enclosing Circle

```
#include <bits/stdc++.h>
using namespace std;

struct Point {
    double x, y;
    Point() {}
    Point(double _x, double _y) : x(_x), y(_y) {}
};

double dist(Point a, Point b) {
    double dx = a.x - b.x, dy = a.y - b.y;
    return sqrt(dx * dx + dy * dy);
}

pair<Point, double> trivial(const vector<Point>& R) {
    if (R.empty()) return {Point(0, 0), 0};
    if (R.size() == 1) return {R[0], 0};
    if (R.size() == 2) {
        Point c{(R[0].x + R[1].x) / 2, (R[0].y + R[1].y) / 2};
        return {c, dist(R[0], R[1]) / 2};
    }
    // 3 points - circumcircle
    double d = 2 * (R[0].x * (R[1].y - R[2].y) +
                    R[1].x * (R[2].y - R[0].y) +
                    R[2].x * (R[0].y - R[1].y));
    if (fabs(d) < 1e-12) return {Point(0, 0), 0}; // degenerate,
        shouldn't happen
    double ux = ((R[0].x * R[0].x + R[0].y * R[0].y) * (R[1].y - R
        [2].y) +
                 (R[1].x * R[1].x + R[1].y * R[1].y) * (R[2].y - R
                     [0].y) +
                 (R[2].x * R[2].x + R[2].y * R[2].y) * (R[0].y - R
                     [1].y)) / d;
    double uy = ((R[0].x * R[0].x + R[0].y * R[0].y) * (R[2].x - R
        [1].x) +
                 (R[1].x * R[1].x + R[1].y * R[1].y) * (R[0].x - R
                     [2].x) +
                 (R[2].x * R[2].x + R[2].y * R[2].y) * (R[1].x - R
                     [0].x)) / d;
    return {Point(ux, uy), dist(R[0], Point(ux, uy))};
}

pair<Point, double> welzl(const vector<Point>& pts_input) {
    vector<Point> pts = pts_input;
    shuffle(pts.begin(), pts.end(), mt19937{});
    vector<Point> R;

    function<pair<Point, double>(int)> rec = [&](int n) -> pair<
        Point, double> {
        if (n == 0 || R.size() == 3) {
            return trivial(R);
        }
        auto d = rec(n - 1);
        if (dist(pts[n - 1], d.first) <= d.second + 1e-9) {
            return d;
        }
        R.push_back(pts[n - 1]);
        auto ans = rec(n - 1);
        R.pop_back();
        return ans;
    };

    return rec(pts.size());
}
```

10.14 Rotating Calipers

```
#include <bits/stdc++.h>
using namespace std;

typedef long long ll;

```

```cpp
struct point{
    ll x,y;
};

ll dist2(point a, point b){
    ll dx = a.x - b.x;
    ll dy = a.y - b.y;
    return dx*dx + dy*dy;
}

ll cross(point a, point b){
    return a.x*b.y - a.y*b.x;
}

point operator-(point a, point b){
    return {a.x - b.x, a.y - b.y};
}

pair<point, point> hull_diameter(const vector<point>& hull){
    int n = hull.size();
    if (n == 1) return {hull[0], hull[0]};

    ll bestd = 0;
    pair<point, point> best = {hull[0], hull[0]};

    int j = 1;
    for (int i = 0; i < n; i++){
        // edge i -> (i+1)%n
        point edge_i = hull[(i+1)%n] - hull[i];
        // advance j to antipodal point (maximizes area with edge i)
        while (cross(edge_i, hull[(j+1)%n] - hull[i]) >
               cross(edge_i, hull[j] - hull[i]))
            j = (j + 1) % n;

        ll cur = dist2(hull[i], hull[j]);
        if (cur > bestd){
            bestd = cur;
            best = {hull[i], hull[j]};
        }
    }
    return best;
}
```

10.15 Pick's Theorem

```cpp
#include <vector>
#include <utility>
#include <cmath>
using namespace std;
#define ll long long

struct Point {
    ll x, y;
};

ll CrossP(Point a, Point b) {
    return a.x * b.y - a.y * b.x;
}

ll gcd(ll a, ll b) {
    if (a < 0) a = -a;
    if (b < 0) b = -b;
    while (b) {
        ll t = a % b;
        a = b;
        b = t;
    }
    return a;
}

// Pick's Theorem: A = I + B/2 - 1
// Returns (interior, boundary)
pair<long long, long long> lattice_points(const vector<Point>& poly) {
    int n = poly.size();
    if (n < 3) return {0, 0};

    // Compute signed area via shoelace
    ll area2 = 0;
    for (int i = 0; i < n; i++) {
        Point p1 = poly[i];
        Point p2 = poly[(i + 1) % n];
        area2 += CrossP(p1, p2);
    }
    ll area = llabs(area2) / 2;

    // Compute boundary points B = sum of gcd(|dx|, |dy|) for each edge
    ll B = 0;
    for (int i = 0; i < n; i++) {
        Point p1 = poly[i];
        Point p2 = poly[(i + 1) % n];
        ll dx = p2.x - p1.x;
        ll dy = p2.y - p1.y;
        B += gcd(dx, dy);
    }

    // I = area - B/2 + 1
    ll I = area - B / 2 + 1;

    return {I, B};
}
```

11 Dynamic Programming

11.1 Coin Problem

```
void solve(){
    int n, x; cin>>n>>x;
    vector<ll> coins(n);
    for(int i=0; i<n; i++){
        cin>>coins[i];
    }

    vector<ll> dp(x+1,LLONG_MAX);
    dp[0] = 0;
    for(int i=0; i<=x; i++){
        for(int j=0; j<n; j++){
            if(i-coins[j]>=0){
                dp[i] = min(dp[i], dp[i-coins[j]]+1);
            }

        }
    }

    if(dp[x] != LLONG_MAX){
        cout<<dp[x]<<endl;
    }else{
        cout<<"-1"<<endl;
    }


}
```

11.2 Coin Problem (Min Coins)

```
void solve(){
    int n, x; cin>>n>>x;
    vector<ll> coins(n);
    for(int i=0; i<n; i++){
        cin>>coins[i];
    }

    vector<ll> dp(x+1,0);
    dp[0] = 1;
    for(int i=0; i<=x; i++){
        for(int j=0; j<n; j++){
            if(i-coins[j]>=0){
                dp[i] = (dp[i] + dp[i-coins[j]]);
                dp[i]%=MOD;
            }
        }
    }


    cout<<dp[x]<<endl;
}
```

11.3 Coin Problem (Sorted)

```
void solve(){
    int n, x; cin>>n>>x;
    vector<ll> coins(n);
    for(int i=0; i<n; i++){
        cin>>coins[i];
    }

    int dp[102][1000005];
    dp[0][0] = 1;
    for(int i=1; i<=n; i++){
        for(int j=0; j<=x; j++){
            dp[i][j] = dp[i-1][j];
            int l = j-coins[i-1];
            if(l>=0){
                dp[i][j] += (dp[i][l])%MOD;
                dp[i][j]%=MOD;
            }
        }
    }


    cout<<dp[n][x]%MOD<<endl;
}
```

11.4 Knapsack

```
void solve(){
    int n,x; cin>>n>>x;
    vector<int> prices(n+1);
    vector<int> pages(n+1);
    for(int i=1; i<=n; i++){cin>>prices[i];}
    for(int i=1; i<=n; i++){cin>>pages[i];}

    vector<int> dp(x+1); // dp[prices[i]] -> Max pages I can buy
        with price i

    dp[0] = 0;
    for(int i=1; i<=n; i++){
        for(int j = x; j>=prices[i]; j--){
```

```
            dp[j] = max(dp[j],dp[j-prices[i]]+pages[i]);
        }
    }

    //for(int i=1; i<=x; i++) cout<<dp[i]<<" ";
    //cout<<endl;

    cout<<dp[x]<<endl;
}
```

11.5 LCS

```
vector<int> lcs(vector<int> const& a, vector<int> const &b, int n,
    int m){
    vector<vector<int>> dp (n+1,vector<int> (m+1,0));

    for(int i=1; i<=n; i++){
        for(int j=1;j<=m; j++){
            if(a[i-1] == b[j-1]) dp[i][j] = dp[i-1][j-1]+1;
            else dp[i][j] = max(dp[i-1][j], dp[i][j-1]);
        }
    }

    int ind = dp[n][m];
    // Note: if we have characters, we must set ind+1 size
    // and set the last character as terminal '\0'
    vector<int> arns(ind);
    int i = n, j = m;

    while(i>0 and j>0){
        if(a[i-1] == b[j-1]){
            arns[ind-1] = a[i-1];
            i--;
            j--;
            ind--;
        }else if(dp[i-1][j] > dp[i][j-1]) i--;
        else j--;
    }

    return arns;
}
```

11.6 LIS

```
int lis(vector<ll> const& a) {
    int n = a.size();
    const int INF = 1e9;
    vector<int> d(n+1, INF);
    d[0] = -INF;

    for (int i = 0; i < n; i++) {
        int l = upper_bound(d.begin(), d.end(), a[i]) - d.begin();
        if (d[l-1] < a[i] && a[i] < d[l])
            d[l] = a[i];
    }

    int ans = 0;
    for (int l = 0; l <= n; l++) {
        if (d[l] < INF)
            ans = l;
    }
    return ans;
}
```

11.7 Two Sets

```
void solve(){
    int n; cin>>n;

    int tw = n*(n+1)/2;
    if(tw%2 == 1){
        cout<<0<<endl;
        return;
    }
    tw = tw/2;

    vector<int> dp(tw+1,0);

    dp[0] = 1;

    for(int i=1; i<n; i++){
        for(int j=tw; j>=i; j--){
            dp[j] = (dp[j] + dp[j-i])%MOD;
        }
    }

    cout<<dp[tw]<<endl;

}
```

11.8 Bitmask DP

```
const int limit=20;
vector<pair<ll,ll>> dp((1<<limit));
```

```cpp
//ascii https://elcodigoascii.com.ar/

inline void solve()
{
    int n; cin>>n;
    ll x; cin>>x;
    vector<ll> weight(n);
    dp[0]={1,0};
    F0(i,n) cin>>weight[i];
    for(ll i=1;i<(1<<n);i++)
    {
        dp[i]={n+1,0};
        for(int j=0;j<n;j++)
        {
            if(i&(1<<j))
            {
                pair<ll,ll> aux=dp[i^(1<<j)];
                if(aux.second+weight[j]<=x){
                    aux.second+=weight[j];
                }
                else{
                    aux.first++;
                    aux.second=weight[j];
                }
                dp[i]=min(dp[i],aux);
            }
        }
    }
    cout<<dp[(1<<n)-1].first<<endl;
}
```

11.9 Digit DP

```cpp
const ll mod=1e9+7;
ll dp[20][10][2][2];
//ascii https://elcodigoascii.com.ar/

inline void init() {
    for(int i=0;i<20;i++)
        for(int j=0;j<10;j++)
            for(int k=0;k<2;k++)
                for(int l=0;l<2;l++)
                    dp[i][j][k][l] = -1;
}

ll mem(int idx,int tight,int prev,int ld,string s)
{
    if(idx==0)
    {
        return 1;
    }
    static bool initialized = false;
    if(!initialized) {
        memset(dp, -1, sizeof(dp));
        initialized = true;
    }
    if(dp[idx][prev][ld][tight]!=-1){
        return dp[idx][prev][ld][tight];
    }
    int k=9;
    if(tight) k=s[s.size()-idx]-'0';
    ll sum=0;
    for(int i=0;i<=k;i++)
    {
        if(ld || prev!=i)
        {
            int new_ld,new_tight;
            if(i==0 && ld) new_ld=1;
            else new_ld=0;
            if(tight && k==i) new_tight=1;
            else new_tight=0;
            sum+=mem(idx-1,new_tight,i,new_ld,s);
        }
    }
    dp[idx][prev][ld][tight]=sum;
    return sum;
}
```

11.10 Edit Distance

```cpp
int editDistance(string &s1, string &s2) {

    int m = s1.length();
    int n = s2.length();

    // Create a table to store results of subproblems
    vector<vector<int>> dp(m + 1, vector<int>(n + 1));

    // Fill the known entries in dp[][]
    // If one string is empty, then answer
    // is length of the other string
    for (int i = 0; i <= m; i++)
        dp[i][0] = i;
    for (int j = 0; j <= n; j++)
        dp[0][j] = j;

    // Fill the rest of dp[][]
    for (int i = 1; i <= m; i++) {
        for (int j = 1; j <= n; j++) {
            if (s1[i - 1] == s2[j - 1])
```

```
                dp[i][j] = dp[i - 1][j - 1];
            else
                dp[i][j] = 1 + min({dp[i][j - 1],
                                    dp[i - 1][j],
                                    dp[i - 1][j - 1]});
        }
    }

    return dp[m][n];
}
```

11.11 Retrieve LIS

```
vector<int> lis(vector<int> const& a) {
    int n = a.size();
    vector<int> d(n, 1), p(n, -1);
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < i; j++) {
            if (a[j] < a[i] && d[i] < d[j] + 1) {
                d[i] = d[j] + 1;
                p[i] = j;
            }
        }
    }

    int ans = d[0], pos = 0;
    for (int i = 1; i < n; i++) {
        if (d[i] > ans) {
            ans = d[i];
            pos = i;
        }
    }

    vector<int> subseq;
    while (pos != -1) {
        subseq.push_back(a[pos]);
        pos = p[pos];
    }
    reverse(subseq.begin(), subseq.end());
    return subseq;
}
```

11.12 SOS DP

```
typedef long long ll;
typedef vector<int> vec;
const ll mod=1e9+7;
const int NMAX=2e5+3;
const int XMAX=1e6+1e5;
const int BT=20;

//ascii https://elcodigoascii.com.ar/


inline void solve()
{
    int n; cin>>n;
    vector<int> arr(n);
    vector<ll> r(XMAX),d(XMAX),zero(XMAX);
    vector<vector<int>> res(n,vector<int>(3));
    int maxi=0;
    FO(i,n){
         cin>>arr[i];
         r[arr[i]]++;
         d[arr[i]]++;

    }
    for(int i=0;i<BT;i++){
        for(int j=0;j<1<<BT;j++)
        {
            if(j&(1<<i)){
                r[j]+=r[j^(1<<i)];
            }
        }
        for(int j=(1<<BT)-1;j>=0;j--){
            if(j&(1<<i)){
                d[j^(1<<i)]+=d[j];
            }
        }
    }


    FO(i,n){
        cout<<r[arr[i]]<<" "<<d[arr[i]]<<" "<<n-r[(1<<BT)-1-arr[i
            ]]<<endl;
    }

}
```

11.13 XOR Subset DP

```
for(int dim=0;dim<MAXDIM;dim++)
{
    for(int i=0;i<n;i++)
    {
        if(i&(1<<dim)){
            F[i]^=F[i^(1<<dim)];
```

```
        }
    }
}
FO(i,n){
    cout<<F[i]<<" ";
}
```

11.14 Convex Hull Trick

```
#include <deque>
#include <algorithm>
typedef long long ll;

const ll INF = 1e18;

// LineContainer: deque-based Convex Hull Trick for monotonic slopes
// Maintains y = m*x + b and supports query(x) = max y at x
struct Line {
    ll m, b;  // y = m*x + b
    mutable ll p; // intersection with next line (x-coordinate)
    // Sort by slope for insertion
    bool operator<(const Line& o) const { return m < o.m; }
};

// For max query with binary search on intersection x
struct LineContainer : std::deque<Line> {
    // Intersection x-coordinate of lines l1 and l2: (b2-b1)/(m1-m2)
    // l1 has smaller slope than l2 (m1 < m2 for max hull, since m1*x+b1 > m2*x+b2 for large negative x)
    static ll intersectX(const Line& l1, const Line& l2) {
        if (l1.m == l2.m) return l1.b > l2.b ? INF : -INF;
        // floor division for ll, works correctly for positive/negative results
        return (l2.b - l1.b) / (l1.m - l2.m);
    }

    // Check if middle line l2 is unnecessary between l1 and l3
    static bool isBad(const Line& l1, const Line& l2, const Line& l3) {
        // l2 is unnecessary if intersection(l1,l3) <= intersection(l1,l2)
        // (b3-b1)/(m1-m3) <= (b2-b1)/(m1-m2)
        // cross-multiply (watch sign of denominator)
        return (l3.b - l1.b) * (l1.m - l2.m) <= (l2.b - l1.b) * (l1.m - l3.m);
    }

    // Add line y = m*x + b (slopes added in monotonic increasing order)
    void add(ll m, ll b) {
        Line line = {m, b, 0};
        // Find position based on slope
        auto it = std::lower_bound(begin(), end(), line);

        // If same slope exists, keep the one with larger b (higher line)
        if (it != end() && it->m == m) {
            if (b <= it->b) return; // existing line is better
            it->b = b;
            it = std::next(it);
        } else {
            it = insert(it, line);
        }

        // Remove lines made obsolete by the new line
        while (it != begin() && std::prev(it, 1) != end()) {
            auto l1 = std::prev(it, 2);
            auto l2 = std::prev(it, 1);
            if (l1->m == l2->m || l2->m == line.m) {
                // Same slope adjacent - remove the middle one with worse b
                if (l2->b < line.b) {
                    erase(l2);
                    break;
                }
            }
            if (isBad(*l1, *l2, line)) {
                erase(l2);
            } else {
                break;
            }
        }

        // Remove lines made obsolete by the new line to the right
        while (std::next(it) != end()) {
            auto l2 = it;
            auto l3 = std::next(it, 1);
            if (isBad(*l2, *l3, line)) {
                erase(l3);
            } else {
                break;
            }
        }

        // Update intersection points
        if (size() >= 2) {
            auto cur = begin();
            auto nxt = std::next(cur);
            cur->p = intersectX(*cur, *nxt);
        }
    }

    // Query maximum y at x
```

```
    ll query(ll x) {
        if (empty()) return -INF;
        // Binary search for rightmost line whose p >= x
        int l = 0, r = size() - 1;
        while (l < r) {
            int m = (l + r + 1) / 2;
            if (at(m).p >= x) l = m;
            else r = m - 1;
        }
        Line& ln = at(l);
        return ln.m * x + ln.b;
    }
};
```

11.15 Knuth's Optimization

```
#include <vector>
#include <climits>

// Knuth's Optimization for DP with quadrangle inequality
// dp[i][j] = min over k of (dp[i][k] + dp[k][j]) + C[i][j]
// Reduces O(N^3) to O(N^2) when cost matrix C satisfies quadrangle inequality
// and monotonicity of optimal split points.

using namespace std;

vector<vector<long long>> knuth_dp(int n, const vector<vector<long long>>& cost) {
    vector<vector<long long>> dp(n, vector<long long>(n, 0));
    vector<vector<int>> opt(n, vector<int>(n, 0));

    // Base case: opt[i][i] = i
    for (int i = 0; i < n; i++) {
        opt[i][i] = i;
    }

    // Increasing interval length
    for (int len = 2; len <= n; len++) {
        for (int i = 0; i + len - 1 < n; i++) {
            int j = i + len - 1;

            // Search for optimal k in [opt[i][j-1], opt[i+1][j]]
            int start = opt[i][j - 1];
            int end = (i + 1 < n) ? opt[i + 1][j] : j;

            long long best = LLONG_MAX;
            int bestK = start;

            for (int k = start; k <= end; k++) {
                long long val = dp[i][k] + dp[k][j] + cost[i][j];
                if (val < best) {
                    best = val;
                    bestK = k;
                }
            }

            dp[i][j] = best;
            opt[i][j] = bestK;
        }
    }

    return dp;
}
```

11.16 Divide and Conquer DP

```
#include <bits/stdc++.h>
using namespace std;

typedef long long ll;
const ll INF = 1e18;

/**
 * Divide and Conquer DP Optimization
 *
 * For DP of the form: dp[j] = min_{k < j} (dp[k] + cost(k, j))
 * where the optimal k for each j is monotonic (opt[j] <= opt[j+1]).
 *
 * Time complexity: O(n log n) instead of O(n^2)
 *
 * Usage:
 *   auto cost = [&](int k, int j) { return dp[k] + transCost(k, j); };
 *   vector<ll> ans = divide_conquer_dp(n, cost);
 */
vector<ll> divide_conquer_dp(int n, function<ll(int k, int j)> cost) {
    vector<ll> dp(n, INF);
    dp[0] = 0;

    // Recursive D&C: compute dp[l..r] given that dp[0..l-1] is computed
    function<void(int, int, int, int)> compute = [&](int l, int r,
        int optL, int optR) {
        if (l > r) return;
        int mid = (l + r) >> 1;
        pair<ll, int> best = {INF, -1};

        // Search for optimal k in [optL, min(mid-1, optR)]
```

```
        int upper = min(mid - 1, optR);
        for (int k = optL; k <= upper; ++k) {
            ll val = dp[k] + cost(k, mid);
            if (val < best.first) {
                best = {val, k};
            }
        }

        dp[mid] = best.first;
        int optK = best.second;

        // Left half: optimal k is in [optL, optK]
        compute(l, mid - 1, optL, optK);
        // Right half: optimal k is in [optK, optR]
        compute(mid + 1, r, optK, optR);
    };

    compute(1, n - 1, 0, n - 1);
    return dp;
}
```

```
    for(int i=1; i<=n; i++)
        vis[i] = false;
}

void solve(){
    int n; cin>>n;
    for(int i=1; i<=n-1; i++){
        int u,v; cin>>u>>v;
        adj[u].push_back(v);
        adj[v].push_back(u);
    }

    int max_count = INT_MIN;
    dfs(1,max_count,0);
    resetVisited(n);
    dfs(far,max_count,0);
    if(max_count == INT_MIN){
        cout<<0<<endl;
        return;
    }

    cout<<max_count<<endl;
}
```

12 Trees

12.1 Diameter

```
const int MAX = 2e5+3;

#define INF LLONG_MAX

vector<vector<int>> adj(MAX);
bool vis[MAX];
vector<int> dist(MAX);

int far;
void dfs(int s, int &max_count, int c){
    vis[s] = true;
    c++;
    for(auto u : adj[s]){
        if(!vis[u]){
            if(c >= max_count){
                max_count = c;
                far = u;
            }
            dfs(u,max_count, c);
        }
    }
}

void resetVisited(int n){
```

12.2 Euler Tour

```
vector<vector<int>> adj(MAX);
vector<int> in(MAX);
vector<int> out(MAX);
vector<ll> values(MAX);
vector<ll> flat(MAX);
int timer = 1;

ll t[2*MAX];

void build(int n){
    for(int i=1; i<=n; i++)
        t[n+i] = flat[i];

    for(int i=n; i>=1; i--)
        t[i] = t[i<<1] + t[i<<1 | 1];

}

void update(int idx, int v, int n){
    for(t[idx += n] = v; idx > 1; idx >>= 1)
        t[idx >> 1] = t[idx ^ 1] + t[idx];

}

```

```cpp
// Query [l,r)

ll query(int l, int r, int n){
    ll arns = 0;
    for(l+=n, r+=n; l<r; l>>=1, r>>=1){
        if (l & 1) arns += t[l++];
        if (r & 1) arns += t[--r];
    }
    return arns;
}

void eulerTour(int s, int e){
    in[s] = timer;
    flat[timer++] = values[s];
    for(auto u : adj[s]){
        if (u == e) continue;
        eulerTour(u,s);
    }
    out[s] = timer;
}

void solve(){
    int n,q; cin>>n>>q;
    for(int i=1; i<=n; i++){
        cin>>values[i];
    }

    for(int i=1; i<=n-1; i++){
        int u,v; cin>>u>>v;
        adj[u].push_back(v);
        adj[v].push_back(u);
    }

    eulerTour(1,0);
    /*
    for(int i=1; i<=n; i++){
        cout<<in[i]<<endl;
    }

    for(int i=1; i<=n; i++){
        cout<<out[i]<<endl;
    }
        */

    build(n);

    for(int i=1; i<=q; i++){
        int a; cin>>a;
        if(a == 1){
            int v; int x; cin>>v>>x;
            update(in[v],x,n);
        }else{
            int v; cin>>v;
            cout<<query(in[v],out[v],n)<<endl;
        }
    }
}
```

12.3 Tree Isomorphism

```cpp
vector<ull> h_map;
vector<ull> t2_map;
vector<ull> d; // Random variable per depth
vector<vector<int>> tree;
vector<vector<int>> tree2;
int arns = 0;

ull mix(ull x){
    x += 0x9e3779b97f4a7c15;
    x = (x ^ (x >> 30)) * 0xbf58476d1ce4e5b9;
    x = (x ^ (x >> 27)) * 0x94d049bb133111eb;
    return x ^ (x >> 31);
}

void init_rng(int depth){
    uint64_t seed = chrono::steady_clock::now().time_since_epoch().
        count();
    mt19937_64 rng(seed);
    d.resize(depth+1);
    for(int i=0; i<=depth; i++){
        d[i] = rng();
    }
}

void hash_tree1(int u, int prev, int depth){
    h_map[u] = 1;
    for(int v : tree[u]){
        if(v!=prev){
            hash_tree1(v,u,depth+1);
            h_map[u] += mix(h_map[v]) ;
        }
    }
}

void hash_tree2(int u, int prev, int depth){
    t2_map[u] = 1;
    for(int v : tree2[u]){
        if(v!=prev){
            hash_tree2(v,u,depth+1);
        t2_map[u] += mix(t2_map[v]);
        }
    }
}
```

12.4 Tree Isomorphism (Centroid Hash)

```
ll shift(ll x) {
    x ^= x << 13;
    x ^= x >> 7;
    x ^= x << 17;
    return x;
}

void sztree(int s,vector<vector<int>> &adj,vector<int> &sz,int par
    =-1)
{
    sz[s]=1;
    for(int ch:adj[s])
    {
        if(ch==par) continue;
        sztree(ch,adj,sz,s);
        sz[s]+=sz[ch];
    }
}

int getRoot(int s,vector<vector<int>> &adj,vector<int> &sz,int n,int
     par=-1){
    for(int ch:adj[s]){
        if(ch==par) continue;
        if(sz[ch]>n/2){
            return getRoot(ch,adj,sz,n,s);
        }
    }
    return s;
}

ll treeHash(int s,vector<vector<int>> &adj,int par=-1){
    ll h=1;
    for(int ch:adj[s]){
        if(ch==par) continue;
        h+=shift(treeHash(ch,adj,s));
    }
    return h;
}

pair<ll,ll> rootHash(int n)
{
    vector<vector<int>> adj(n+1);
    vector<int> sz(n+1);
    FO(i,n-1){
        int a,b; cin>>a>>b;
        adj[a].push_back(b);
        adj[b].push_back(a);
    }
    sztree(1,adj,sz);
    pair<ll,ll> candidates;
    int centroid1=getRoot(1,adj,sz,n);
    sztree(centroid1,adj,sz);
    int centroid2=-1;
    for(int ch:adj[centroid1]){
        if(n-sz[ch]<=n/2){
            centroid2=ch;
            break;
        }
    }
    candidates.first=treeHash(centroid1,adj);
    if(centroid2!=-1){
        candidates.second=treeHash(centroid2,adj);
    }
    else{
        candidates.second=-1;
    }
    return candidates;
}
```

12.5 Max Distance

```
#define INF LLONG_MAX

vector<vector<int>> adj(MAX);
vector<int> f(MAX,0); // f(x) = maximum distance from node x to any
    node of its subtree
vector<int> g(MAX,0); // g(x) = maximum distance from any node to x
     outside its subtree
vector<int> arns(MAX,0); // arns(x) = max(f(x),g(x))


void dfsA(int s, int e){
    for(auto u : adj[s]){
        if(u == e) continue;
        dfsA(u,s);
        if(f[u]+1 > f[s]){ // Maximum distance + 1 of u > Max dist
            of predecessor
            g[s] = f[s];   // Maximum distance from any node outside
                of s subtree is f[s]
            f[s] = f[u] + 1;
        }else if(f[u]+1>g[s]){
            g[s] = f[u]+1; // Maximum distance from any node outside
                of s subtree is f[u]+1
        }
    }
}

void dfsB(int s, int e, int to){
    arns[s] = max(to, f[s]);
    for(auto u : adj[s]){
```

```
        if(u == e) continue;
        if(f[u]+1 == f[s]) dfsB(u,s, max(to,g[s])+1);
        else dfsB(u,s, arns[s]+1);
    }
}

void solve(){
    int n; cin>>n;
    for(int i=1; i<=n-1; i++){
        int u,v; cin>>u>>v;
        adj[u].push_back(v);
        adj[v].push_back(u);
    }

    dfsA(1,0);
    dfsB(1,0,0);

    for(int i=1;i<=n; i++)
        cout<<arns[i]<<" ";
    cout<<endl;
}
```

12.6 Binary Lifting

```
int n, l;
vector<vector<int>> adj;

int timer;
vector<int> tin, tout;
vector<vector<int>> up;

void dfs(int v, int p)
{
    tin[v] = ++timer;
    up[v][0] = p;
    for (int i = 1; i <= l; ++i)
        up[v][i] = up[up[v][i-1]][i-1];

    for (int u : adj[v]) {
        if (u != p)
            dfs(u, v);
    }

    tout[v] = ++timer;
}

bool is_ancestor(int u, int v)
{
    return tin[u] <= tin[v] && tout[u] >= tout[v];
}

int lca(int u, int v)
{
    if (is_ancestor(u, v))
        return u;
    if (is_ancestor(v, u))
        return v;
    for (int i = l; i >= 0; --i) {
        if (!is_ancestor(up[u][i], v))
            u = up[u][i];
    }
    return up[u][0];
}

void preprocess(int root) {
    tin.resize(n);
    tout.resize(n);
    timer = 0;
    l = ceil(log2(n));
    up.assign(n, vector<int>(l + 1));
    dfs(root, root);
}
```

12.7 Centroid Decomposition

```
bool v[MAX];
int dst[MAX];
int sz[MAX];
vector<int> adj[MAX];

// Buffer estatico global para recolectar profundidades rapidisimo
int depth_buffer[MAX];

class CentroidDecomposition {
    private:
        int k;
        ll res=0;

        void sztree(int s, int par=-1) {
            sz[s]=1;
            for(int ch : adj[s]){
                if(ch==par || v[ch]) continue;
                sztree(ch, s);
                sz[s] += sz[ch];
            }
        }

        // Un solo DFS por subarbol: guarda los resultados en el
            buffer continuo
```

```
        void get_depths(int s, int par, int depth, int &tail, int &
            max_d) {
            if(depth > k) return;
            depth_buffer[tail++] = depth;
            max_d = max(max_d, depth);
            for(int ch : adj[s]){
                if(ch==par || v[ch]) continue;
                get_depths(ch, s, depth+1, tail, max_d);
            }
        }

    public:
        CentroidDecomposition(int _k): k(_k) {
            res=0;
        }

        void addEdge(int a, int b){
            adj[a].push_back(b);
            adj[b].push_back(a);
        }

        // VERSION ITERATIVA: A prueba de Stack Overflow (evita TLE/
            RTE en casos borde)
        int getCentroid(int s, int n) {
            int par = -1;
            bool found = true;
            while(found) {
                found = false;
                for(int ch : adj[s]) {
                    if(ch != par && !v[ch] && sz[ch] > n/2) {
                        par = s;
                        s = ch;
                        found = true;
                        break;
                    }
                }
            }
            return s;
        }

        void Decompose(int s, int par=-1) {
            sztree(s, par);
            s = getCentroid(s, sz[s]); // Encontramos centroide de
                forma segura
            v[s] = true;

            dst[0] = 1;
            int current_max_depth = 0;
            sort(all(adj[s]),[](int a,int b){
                return sz[a]<sz[b];
            });

            for(int ch : adj[s]){
                if(v[ch]) continue; // Solo nos importa no cruzar
                    componentes visitados

                int tail = 0;
                int sub_max_depth = 0;

                // 1. Recolectar (1 solo DFS)
                get_depths(ch, s, 1, tail, sub_max_depth);

                // 2. Contar caminos cruzados (bucle cache-friendly)
                for(int i = 0; i < tail; i++) {
                    res += dst[k - depth_buffer[i]];
                }

                // 3. Agregar subarbol actual al arreglo maestro
                for(int i = 0; i < tail; i++) {
                    dst[depth_buffer[i]]++;
                }

                current_max_depth = max(current_max_depth,
                    sub_max_depth);
            }

            // Limpieza ultra rapida usando memset implicito
            fill(dst, dst + current_max_depth + 1, 0);
            dst[0] = 1; // restore centroid contribution for
                subsequent children

            // Recursion para el resto de los componentes
            for(int ch : adj[s]){
                if(v[ch]) continue;
                Decompose(ch, s);
            }
        }

        ll getRes(){ return res; }
};
```

12.8 Heavy-Light Decomposition

```
vector<int> parent, depth, heavy, head, pos;
int cur_pos;

int dfs(int v, vector<vector<int>> const& adj) {
    int size = 1;
    int max_c_size = 0;
    for (int c : adj[v]) {
        if (c != parent[v]) {
            parent[c] = v, depth[c] = depth[v] + 1;
            int c_size = dfs(c, adj);
```

```
            size += c_size;
            if (c_size > max_c_size)
                max_c_size = c_size, heavy[v] = c;
        }
    }
    return size;
}

void decompose(int v, int h, vector<vector<int>> const& adj) {
    head[v] = h, pos[v] = cur_pos++;
    if (heavy[v] != -1)
        decompose(heavy[v], h, adj);
    for (int c : adj[v]) {
        if (c != parent[v] && c != heavy[v])
            decompose(c, c, adj);
    }
}

void init(vector<vector<int>> const& adj) {
    int n = adj.size();
    parent = vector<int>(n);
    depth = vector<int>(n);
    heavy = vector<int>(n, -1);
    head = vector<int>(n);
    pos = vector<int>(n);
    cur_pos = 0;

    dfs(0, adj);
    decompose(0, 0, adj);
}

int query(int a, int b) {
    int res = 0;
    for (; head[a] != head[b]; b = parent[head[b]]) {
        if (depth[head[a]] > depth[head[b]])
            swap(a, b);
        int cur_heavy_path_max = segment_tree_query(pos[head[b]],
            pos[b]);
        res = max(res, cur_heavy_path_max);
    }
    if (depth[a] > depth[b])
        swap(a, b);
    int last_heavy_path_max = segment_tree_query(pos[a], pos[b]);
    res = max(res, last_heavy_path_max);
    return res;
}
```

12.9 Lowest Common Ancestor

```
struct LCA {
    vector<int> height, euler, first, segtree;
    vector<bool> visited;
    int n;

    LCA(vector<vector<int>> &adj, int root = 0) {
        n = adj.size();
        height.resize(n);
        first.resize(n);
        euler.reserve(n * 2);
        visited.assign(n, false);
        dfs(adj, root);
        int m = euler.size();
        segtree.resize(m * 4);
        build(1, 0, m - 1);
    }

    void dfs(vector<vector<int>> &adj, int node, int h = 0) {
        visited[node] = true;
        height[node] = h;
        first[node] = euler.size();
        euler.push_back(node);
        for (auto to : adj[node]) {
            if (!visited[to]) {
                dfs(adj, to, h + 1);
                euler.push_back(node);
            }
        }
    }

    void build(int node, int b, int e) {
        if (b == e) {
            segtree[node] = euler[b];
        } else {
            int mid = (b + e) / 2;
            build(node << 1, b, mid);
            build(node << 1 | 1, mid + 1, e);
            int l = segtree[node << 1], r = segtree[node << 1 | 1];
            segtree[node] = (height[l] < height[r]) ? l : r;
        }
    }

    int query(int node, int b, int e, int L, int R) {
        if (b > R || e < L)
            return -1;
        if (b >= L && e <= R)
            return segtree[node];
        int mid = (b + e) >> 1;

        int left = query(node << 1, b, mid, L, R);
        int right = query(node << 1 | 1, mid + 1, e, L, R);
        if (left == -1) return right;
        if (right == -1) return left;
        return height[left] < height[right] ? left : right;
```

```
    }

    int lca(int u, int v) {
        int left = first[u], right = first[v];
        if (left > right)
            swap(left, right);
        return query(1, 0, euler.size() - 1, left, right);
    }
};
```

12.10 Binary Lifting LCA

```
void dfs(int s,vector<vector<pair<int,int>>> &adj,int par=-1, int
    weight=0)
{
    tin[s]=++tim;
    wlca[s][0]=weight;
    //cout<<weight<<endl;
    lca[s][0]=par;
    for(auto [ch,w]:adj[s]){
        if(ch!=par){
            dfs(ch,adj,s,w);
        }
    }
    tout[s]=++tim;
}

bool ancestor(int a,int b){
    if(a==-1) return true;
    if(b==-1) return false;
    return tin[a]<=tin[b] && tout[b]<=tout[a];
}

int getLca(int a,int b)
{
    if(ancestor(a,b)) return a;
    if(ancestor(b,a)) return b;
    for(int i=MDEPTH-1;i>=0;i--)
    {
        //cout<<lca[a][i]<<endl;
        if(!ancestor(lca[a][i],b)){
            a=lca[a][i];
        }
    }

    return lca[a][0];
}
```

12.11 Successor DP

```
const int m=30;
int succesorM[limit][m];
//ascii https://elcodigoascii.com.ar/

inline void solve()
{
    int n,q; cin>>n>>q;
    int res,aux;
    ll k;
    lFOR(i,n){
        cin>>succesorM[i][0];
    }
    FOR(j,1,m)
    {
        lFOR(i,n)
        {
            succesorM[i][j]=succesorM[succesorM[i][j-1]][j-1];
        }
    }
    FO(i,q)
    {
        cin>>res>>k;
        aux=0;
        while(k)
        {
            if(k%2){
                res=succesorM[res][aux];
            }
            k/=2;
            aux++;
        }
        cout<<res<<endl;
    }
}
```

13 Miscellaneous

13.1 2D Prefix Sum

```
void solve(){
    int n,q; cin>>n>>q;
    char forest[n+1][n+1];

    for(int i=1; i<=n; i++){
        for(int j=1; j<=n; j++){
            cin>>forest[i][j];
```

```
        }
    }

    vector<vector<int>> dp(n+1, vector<int> (n+1,0));


    for(int i=1; i<=n; i++){
        for(int j=1; j<=n; j++){
            dp[i][j] =  dp[i][j-1] + dp[i-1][j] - dp[i-1][j-1];
            if(forest[i][j] == '*') dp[i][j]++;
        }
    }



    for(int i=0; i<q; i++){
        int a,b,c,d;
        cin>>a>>b>>c>>d;
        cout<<dp[c][d] - dp[a-1][d] - dp[c][b-1] + dp[a-1][b-1]<<
            endl;
    }

}
```

13.2 Kadane

```
ll kadane(vector<int> &a, int n){
    ll arns = a[0], zum = 0;

    for (int i = 0; i < n; i++){
        zum += a[i];
        arns = max(arns, zum);
        zum = max(zum, 0LL);
    }

    return arns;
}
```

13.3 Index Compression

```
sort(all(a));

auto compression = [&](int x) {
    return (int)(lower_bound(all(a), x) - a.begin());
};
```

13.4 Berlekamp-Massey

```
vector<T> berlekampMassey(const vector<T> &s) {
    vector<T> c;    // the linear recurrence sequence we are
        building
    vector<T> oldC; // the best previous version of c to use (the
        one with the rightmost left endpoint)
    int f = -1;     // the index at which the best previous version
        of c failed on
    for (int i=0; i<(int)s.size(); i++) {
        // evaluate c(i)
        // delta = s_i - \sum_{j=1}^n c_j s_{i-j}
        // if delta == 0, c(i) is correct
        T delta = s[i];
        for (int j=1; j<=(int)c.size(); j++)
            delta -= c[j-1] * s[i-j];   // c_j is one-indexed, so we
                actually need index j - 1 in the code
        if (delta == 0)
            continue;   // c(i) is correct, keep going
        // now at this point, delta != 0, so we need to adjust it
        if (f == -1) {
            // this is the first time we're updating c
            // s_i was the first non-zero element we encountered
            // we make c of length i + 1 so that s_i is part of the
                base case
            c.assign(i + 1, T(0));
            c[i] = T(1);
            f = i;
        } else {
            // we need to use a previous version of c to improve on
                this one
            // apply the 5 steps to build d
            // 1. set d equal to our chosen sequence
            vector<T> d = oldC;
            // 2. multiply the sequence by -1
            for (T &x : d)
                x = -x;
            // 3. insert a 1 on the left
            d.insert(d.begin(), 1);
            // 4. multiply the sequence by delta / d(f + 1)
            T df1 = 0;  // d(f + 1)
            for (int j=1; j<=(int)d.size(); j++)
                df1 += d[j-1] * s[f+1-j];
            assert(df1 != 0);
            T coef = delta / df1;   // storing this in outer
                variable so it's O(n^2) instead of O(n^2 log MOD)
            for (T &x : d)
                x *= coef;
            // 5. insert i - f - 1 zeros on the left
            vector<T> zeros(i - f - 1);
            zeros.insert(zeros.end(), d.begin(), d.end());
            d = zeros;
```

```
            // now we have our new recurrence: c + d
            vector<T> temp = c; // save the last version of c
                because it might have a better left endpoint
            c.resize(max(c.size(), d.size()));
            for (int j=0; j<(int)d.size(); j++)
                c[j] += d[j];
            // finally, let's consider updating oldC
            if (i - (int) temp.size() > f - (int) oldC.size()) {
                // better left endpoint, let's update!
                oldC = temp;
                f = i;
            }
        }
    }
    return c;
}
```

13.5 Moore's Voting

```
int majorityElement(vector<int>& nums) {
    int vote = 0, r = 0;
    for(int i=0; i<nums.size();i++){
        if(nums[i] == nums[r])
            vote++;
        else
            vote--;
        if(vote == 0){
            r = i;
            vote = 1;
        }
    }

    int cnt = 0;
    int goal = (nums.size())/2;
    for(int i=0; i<nums.size(); i++){
        if(nums[i] == nums[r]){
            cnt++;
            if(cnt > goal){
                break;
            }
        }
    }

    return nums[r];
}
```

13.6 Compiler Optimizations

```
// C++ compiler optimizations for competitive programming.
// Use in submission files when heavy loops / math / bit operations
    are bottlenecks.
// Avoid for interactive/debug-heavy tasks because aggressive
    optimizations can make debugging harder.

#pragma GCC optimize("O3,unroll-loops")
#pragma GCC target("avx2,bmi,bmi2,lzcnt,popcnt")

// Optional fast I/O toggle:
// ios::sync_with_stdio(false);
// cin.tie(nullptr);

// Notes:
// - O3 + unroll-loops: usually safe and useful in most non-
    interactive problems.
// - target(...): can speed up bitset/popcount-heavy code, but may
    fail on judges without these CPU features.
// - If unsure about judge hardware, remove target(...) and keep O3
    only.
```

13.7 Sprague-Grundy

```
#include <vector>
#include <functional>
#include <unordered_map>
using namespace std;

int mex(vector<int>& s) {
    if (s.empty()) return 0;
    int n = s.size();
    int lo = 0, hi = n;
    while (lo < hi) {
        int mid = (lo + hi) / 2;
        int cnt = 0;
        for (int x : s) if (x <= mid) cnt++;
        if (cnt > mid) hi = mid;
        else lo = mid + 1;
    }
    return lo;
}

int grundy(int state, function<vector<int>(int)> moves) {
    static unordered_map<int, int> memo;
    if (memo.count(state)) return memo[state];
    vector<int> g;
    for (int nxt : moves(state))
        g.push_back(grundy(nxt, moves));
    return memo[state] = mex(g);
```

```
}
```